HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO BÀI TẬP LỚN MÔN HỌC IOT VÀ ỨNG DỤNG

Đề tài: Xây dựng hệ thống IoT giám sát môi trường và điều khiển thiết bị từ xa sử dụng ESP32

Giảng viên hướng dẫn: TS. Nguyễn Quốc Uy

Họ và tên:	**Nguyễn Chí Hùng**
Mã sinh viên:	**B23DCCN359**
Lớp:	**D23CNPM04**
Nhóm lớp:	**10**

Hà Nội - 2026

Lời cảm ơn

Trước hết, em xin gửi lời cảm ơn chân thành đến Thầy TS.Nguyễn Quốc Uy, giảng viên môn IoT, đã tận tình giảng dạy, hướng dẫn và truyền đạt cho em những kiến thức quý báu trong suốt quá trình học tập và thực hiện bài tập lớn với đề tài "Xây dựng hệ thống IoT giám sát môi trường và điều khiển thiết bị từ xa sử dụng ESP32".

Nhờ sự hướng dẫn và những góp ý tận tình của Thầy, em đã có cơ hội vận dụng các kiến thức về Internet of Things, vi điều khiển ESP32, cảm biến, kết nối mạng và điều khiển thiết bị từ xa vào một bài toán thực tế. Qua quá trình thực hiện, em không chỉ củng cố kiến thức chuyên môn mà còn rèn luyện được khả năng nghiên cứu, lập trình và giải quyết các vấn đề phát sinh trong quá trình xây dựng hệ thống.

Em cũng xin bày tỏ lòng biết ơn sâu sắc tới Học viện Công nghệ Bưu chính Viễn thông, nơi đã tạo môi trường học tập, cung cấp cơ sở vật chất và những điều kiện cần thiết để em có thể học tập, nghiên cứu và hoàn thành bài tập lớn này.

Mặc dù đã cố gắng hoàn thiện bài tập và báo cáo một cách tốt nhất, nhưng do kiến thức và kinh nghiệm thực tế của bản thân còn hạn chế nên bài làm không tránh khỏi những thiếu sót. Em rất mong nhận được sự nhận xét và góp ý của Thầy để có thể hoàn thiện hơn kiến thức và kỹ năng của mình.

Cuối cùng, em xin kính chúc Thầy TS.Nguyễn Quốc Uy luôn mạnh khỏe, hạnh phúc và thành công trong công tác giảng dạy, đồng thời tiếp tục truyền đạt những kiến thức quý báu và cảm hứng học tập đến các thế hệ sinh viên.

Em xin chân thành cảm ơn!

MỤC LỤC

DANH MỤC HÌNH ẢNH 6
CHƯƠNG 1. TỔNG QUAN DỰ ÁN 8
1.1. Mục tiêu dự án 8
1.2. Phạm vi dự án 8
1.3. Đối tượng sử dụng 8
1.4. Các chức năng chính 9
1.5. Các công nghệ, thiết bị sử dụng 9
CHƯƠNG 2. THIẾT KẾ KIẾN TRÚC TỔNG THỂ 11
CHƯƠNG 3. THIẾT KẾ CHI TIẾT HỆ THỐNG 12
3.1. Thiết kế cơ sở dữ liệu 12
3.1.1. Lược đồ ERD 12
3.1.2. Lược đồ quan hệ 13
3.2. Use case 14
3.2.1. Sơ đồ Use case tổng quan 14
3.2.2. Use case 1: View Data On Dashboard 15
3.2.3. Use case 2: Control Device 17
3.2.4. Use case 3: View Sensor Data History 20
3.2.5. Use case 4: View Device Activity History 22
3.2.6. Use case 5: View Profile 24
CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG 26
4.1. Cài đặt mosquitto 26
4.2. Lắp đặt thiết bị 27
4.2.1. Danh sách các thiết bị 27
4.2.2 Lắp đặt 29
4.3. Lập trình phần cứng 30
4.3.1. Khai báo thư viện, các biến và nguyên mẫu hàm 30
4.3.2. Khởi tạo hệ thống và kết nối mạng 32
4.3.3. Vòng lặp hoạt động chính 36
4.3.4. Đọc và gửi dữ liệu cảm biến 37

4.3.5. Nhận lệnh và điều khiển thiết bị 39
4.3.6. Gửi trạng thái thiết bị 41
4.4. Xây dựng front-end 42
4.4.1. Cấu trúc thư mục 43
4.4.2. Thư mục assets 43
4.4.3. Thư mục js 44
4.4.4. Các file HTML gốc 47
4.4.5. Thư mục css 48
4.5. Xây dựng back-end 51
4.5.1. Cấu trúc thư mục 52
4.5.2. Thư mục config 53
4.5.3. Thư mục controller 53
4.5.4. Thư mục dto 54
4.5.5. Thư mục entity 55
4.5.6. Thư mục exception 55
4.5.7. Thư mục mqtt 55
4.5.8. Thư mục repository 56
4.5.9. Thư mục security 56
4.5.10. Thư mục service 56
4.5.11. Thư mục resources 57
4.5.12. File pom.xml 58
4.6. API documentation 58
4.6.1. Sensor APIs 58
4.6.2. Device (LED) APIs 64
4.6.3. History APIs 67
CHƯƠNG 5. KẾT QUẢ ĐẠT ĐƯỢC 70
5.1. Trang Dashboard 70
5.2. Trang Data sensor 72
5.3. Trang History 73
5.4. Trang Profile 75

5.5. Đánh giá kết quả 75
5.5.1. Ưu điểm 76
5.5.2. Nhược điểm 77
5.5.3. Hướng khắc phục và phát triển 78
CHƯƠNG 6. KẾT LUẬN 80
PHỤ LỤC 81

DANH MỤC HÌNH ẢNH

Hình 2.1. Kiến trúc tổng quan hệ thống 11
Hình 3.1. Lược đồ ERD 12
Hình 3.2. Lược đồ quan hệ 13
Hình 3.3. Sơ đồ use case tổng quan 14
Hình 3.4. Giao diện cho use case 1 15
Hình 3.5. Biểu đồ tuần tự use case 1 16
Hình 3.6. Use case điều khiển thiết bị 17
Hình 3.7. Biểu đồ tuần tự use case 2 18
Hình 3.8. Giao diện cho use case 3 20
Hình 3.9. Biểu đồ tuần tự use case 3 21
Hình 3.10. Giao diện cho use case 4 22
Hình 3.11. Biểu đồ tuần tự use case 4 23
Hình 3.12. Giao diện use case 5 24
Hình 3.13. Biểu đồ tuần tự use case 5 25
Hình 4.1. Trang web download mosquitto 26
Hình 4.2. File cấu hình mosquitto.conf 26
Hình 4.3. Kit ESP32 + cáp 27
Hình 4.5. Breadboard 27
Hình 4.6. Cảm biến ánh sáng 27
Hình 4.7. Cảm biến nhiệt độ và độ ẩm 28
Hình 4.8. Đèn led 5mm 28
Hình 4.9. Điện trở 220R 28
Hình 4.10. Dây đực - đực 28
Hình 4.11. Dây đực - cái 29
Hình 4.12. Phần cứng sau khi lắp đặt hoàn chỉnh 29
Hình 4.13. Khai báo thư viện, các biến, nguyên hàm 30
Hình 4.14. Hàm setup() 32
Hình 4.15. Hàm connectWifi() 34
Hình 4.16. Hàm connectMQTT() 35
Hình 4.17. Hàm loop() 36
Hình 4.18. Hàm publishSensorData() 37
Hình 4.19. Hàm readLightLux() 38
Hình 4.20. Hàm onMqttMessage() 39
Hình 4.21. Hàm publishDeviceStatus() 41
Hình 4.22. Cấu trúc thư mục front-end 43
Hình 4.23. Cấu trúc thư mục back-end 52

Hình 4.24. Test api/sensors/latest với postman 59
Hình 4.25. Test api/sensors/chart với postman 60
Hình 4.26. Test api/sensors/chart với postman (tiếp) 60
Hình 4.27. Test api/sensors/chart với postman (tiếp) 61
Hình 4.28. Test api/sensors/chart với postman (tiếp) 61
Hình 4.29. Test api/sensors với postman 63
Hình 4.30. Test api/sensors với postman (tiếp) 63
Hình 4.31. Test api/sensors với postman (tiếp) 64
Hình 4.32. Test api/devices/status với postman 65
Hình 4.33. Test api/devices/{deviceId}/control với postman 66
Hình 4.34. Test api/devices/history/{historyId}/status với postman 67
Hình 4.35. Test api/devices/history với postman 68
Hình 4.36. Test api/devices/history với postman (tiếp) 69
Hình 4.37. Test api/devices/history với postman (tiếp) 69
Hình 5.1. Kết quả trang Dashboard 70
Hình 5.2. Giao diện khi bật đèn 2 70
Hình 5.3. Thiết bị khi bật đèn 2 71
Hình 5.4. Kết quả trang Data sensor 72
Hình 5.5. Kết quả trang History 73
Hình 5.6. Kết quả trang profile 75

CHƯƠNG 1. TỔNG QUAN DỰ ÁN

1.1. Mục tiêu dự án

Dự án nhằm xây dựng một hệ thống Internet of Things (IoT) đơn giản sử dụng vi điều khiển ESP32 để thu thập dữ liệu từ các cảm biến môi trường và điều khiển thiết bị từ xa thông qua giao diện Web.

Hệ thống sử dụng các cảm biến để thu thập ba thông số môi trường gồm nhiệt độ, độ ẩm và cường độ ánh sáng. Dữ liệu sau khi được ESP32 thu thập sẽ được truyền thông qua giao thức MQTT đến máy chủ MQTT. Website có nhiệm vụ nhận dữ liệu và hiển thị các thông số cảm biến theo thời gian thực, với chu kỳ cập nhật khoảng 2 giây.

Bên cạnh chức năng giám sát, hệ thống cho phép người dùng điều khiển bật/tắt hai đèn LED trực tiếp từ giao diện Web. Lệnh điều khiển từ Web được truyền thông qua MQTT đến ESP32, sau đó ESP32 thực hiện thay đổi trạng thái của các LED tương ứng.

Thông qua dự án, hệ thống hướng đến việc minh họa mô hình hoạt động cơ bản của một hệ thống IoT, trong đó thiết bị cảm biến, máy chủ trung gian MQTT và ứng dụng Web có thể trao đổi dữ liệu hai chiều.

1.2. Phạm vi dự án

Phạm vi của dự án tập trung vào việc xây dựng một mô hình IoT quy mô nhỏ phục vụ mục đích học tập và nghiên cứu.

Dự án tập trung vào các chức năng giám sát và điều khiển cơ bản, chưa hướng đến việc triển khai hệ thống IoT ở quy mô lớn, quản lý nhiều thiết bị hoặc xây dựng các cơ chế bảo mật phức tạp.

1.3. Đối tượng sử dụng

Đối tượng sử dụng chính của hệ thống là người dùng có nhu cầu theo dõi các thông số môi trường và điều khiển thiết bị từ xa thông qua Website.

Ngoài ra, hệ thống cũng hỗ trợ phục vụ trong quá trình học tập và thực hành của sinh viên ngành công nghệ thông tin, kỹ thuật máy tính, điện - điện tử hoặc các ngành liên quan đến IoT.

1.4. Các chức năng chính

- Thu thập và hiển thị dữ liệu cảm biến: Thu thập dữ liệu nhiệt độ, độ ẩm và ánh sáng từ ESP32, truyền qua MQTT và hiển thị trên Website theo thời gian thực.

- Điều khiển thiết bị: Cho phép người dùng bật/tắt các thiết bị LED trực tiếp từ Website thông qua giao thức MQTT.

- Quản lý lịch sử dữ liệu cảm biến: Cho phép xem, tìm kiếm và sắp xếp lịch sử dữ liệu cảm biến theo thời gian.

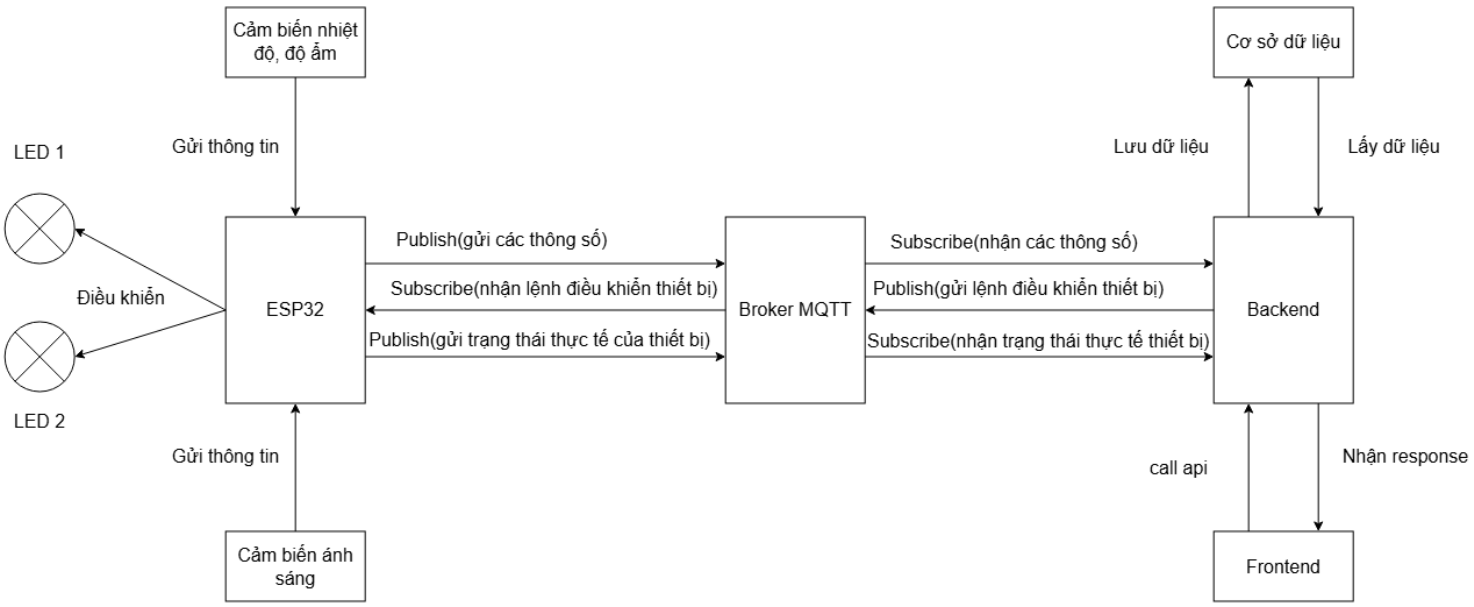
- Quản lý lịch sử hoạt động thiết bị: Cho phép xem, tìm kiếm và sắp xếp lịch sử bật/tắt của các thiết bị theo thời gian.

1.5. Các công nghệ, thiết bị sử dụng

Thành phần	**Công nghệ/Thiết bị**	**Vai trò**
Vi điều khiển	ESP32	Đọc dữ liệu từ cảm biến, kết nối Wi-Fi và điều khiển LED
Lập trình ESP32	Arduino IDE / Arduino Framework	Lập trình và nạp chương trình cho ESP32
Cảm biến	Cảm biến nhiệt độ, độ ẩm (DHT 11), ánh sáng (photodiode TH373)	Thu thập dữ liệu môi trường
Thiết bị đầu ra	2 LED 5mm	Thực hiện chức năng bật/tắt theo yêu cầu
Giao thức	MQTT	Trao đổi dữ liệu giữa ESP32 và hệ thống Web
MQTT Broker	MQTT Broker	Trung gian nhận và phân phối dữ liệu MQTT
Backend	Spring boot, RESTful API	Xử lý dữ liệu và cung cấp API cho Website
Frontend	HTML, CSS, JavaScript	Xây dựng giao diện giám sát và điều khiển

Thành phần	Công nghệ/Thiết bị	Vai trò
Cơ sở dữ liệu	MySQL	Lưu trữ, quản lý và truy xuất dữ liệu

CHƯƠNG 2. THIẾT KẾ KIẾN TRÚC TỔNG THỂ

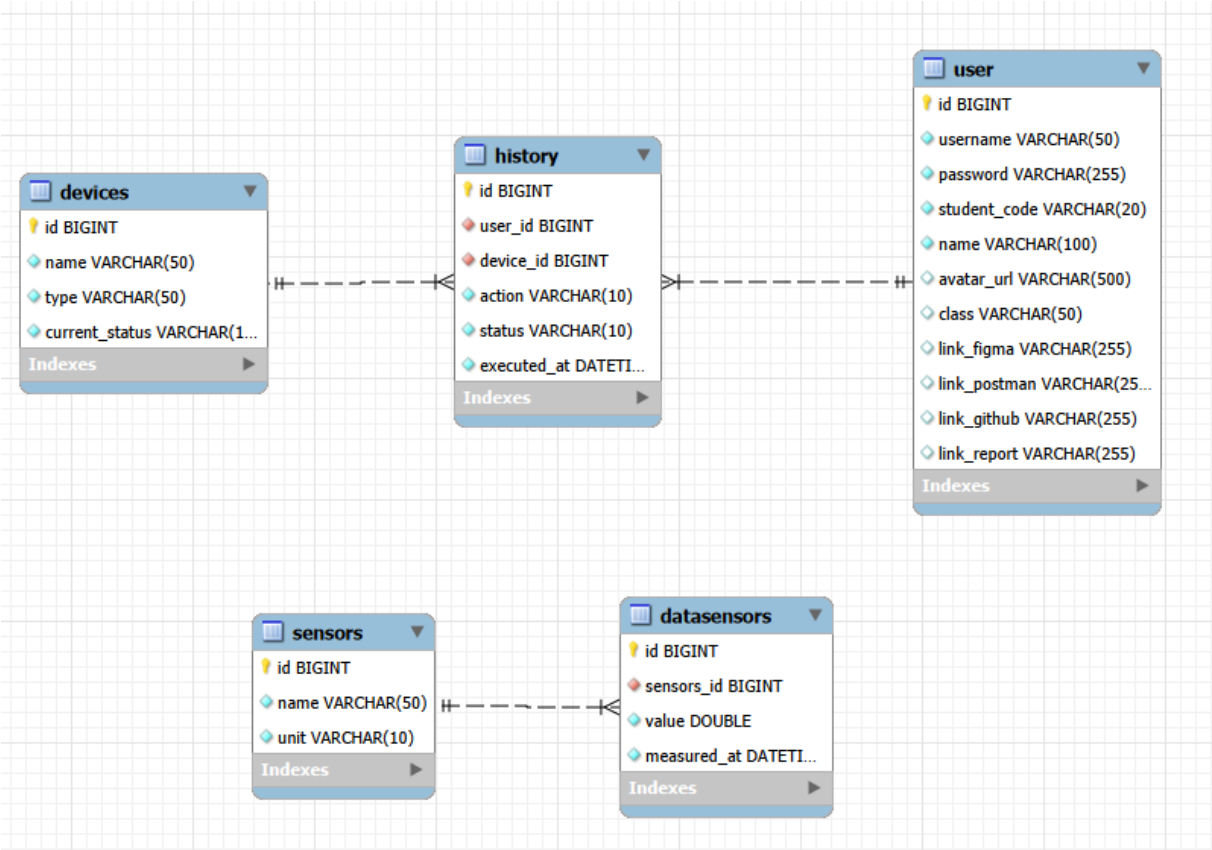


Hình 2.1. Kiến trúc tổng quan hệ thống

CHƯƠNG 3. THIẾT KẾ CHI TIẾT HỆ THỐNG

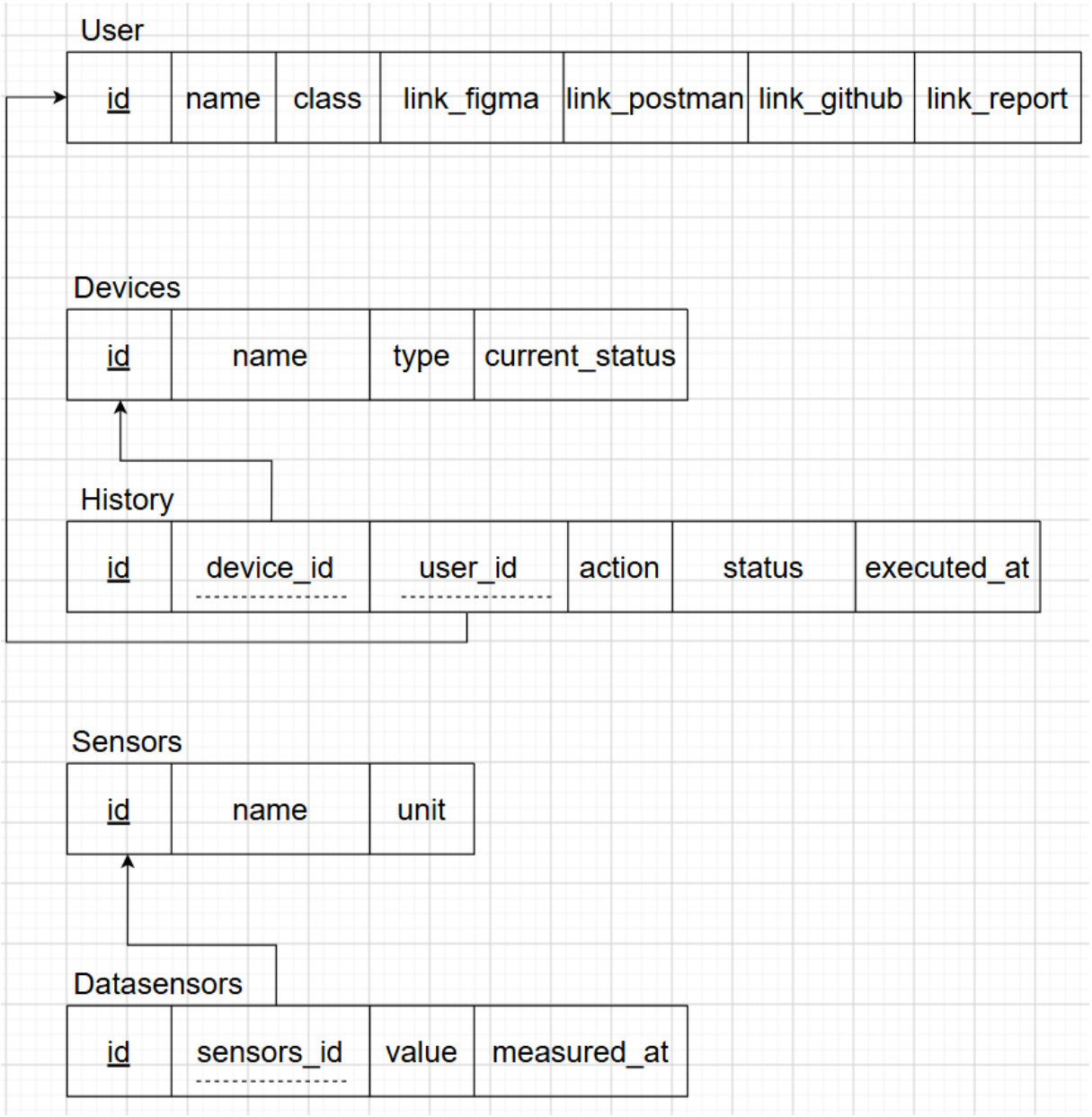
3.1. Thiết kế cơ sở dữ liệu

3.1.1. Lược đồ ERD



Hình 3.1. Lược đồ ERD

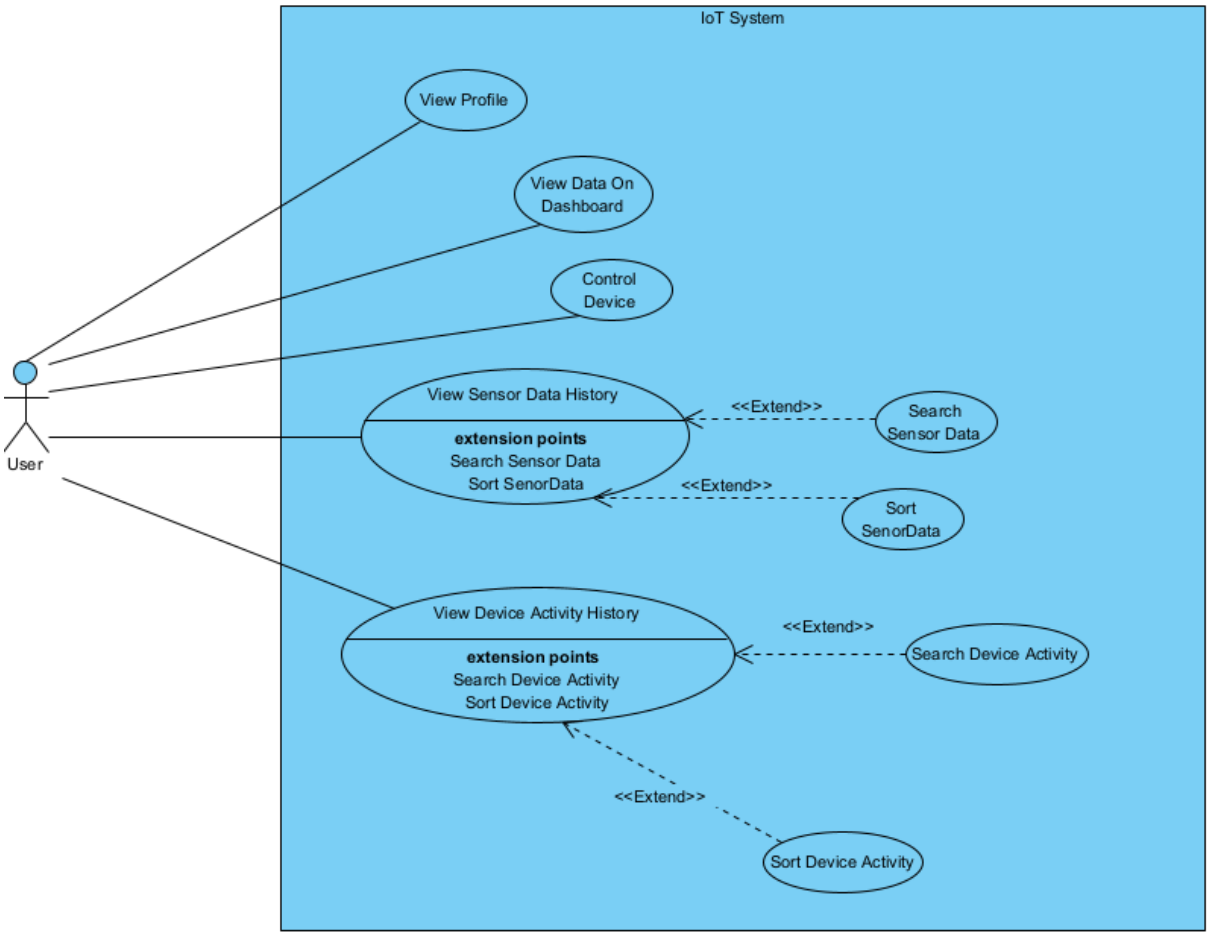
3.1.2. Lược đồ quan hệ



Hình 2.2. Lược đồ quan hệ

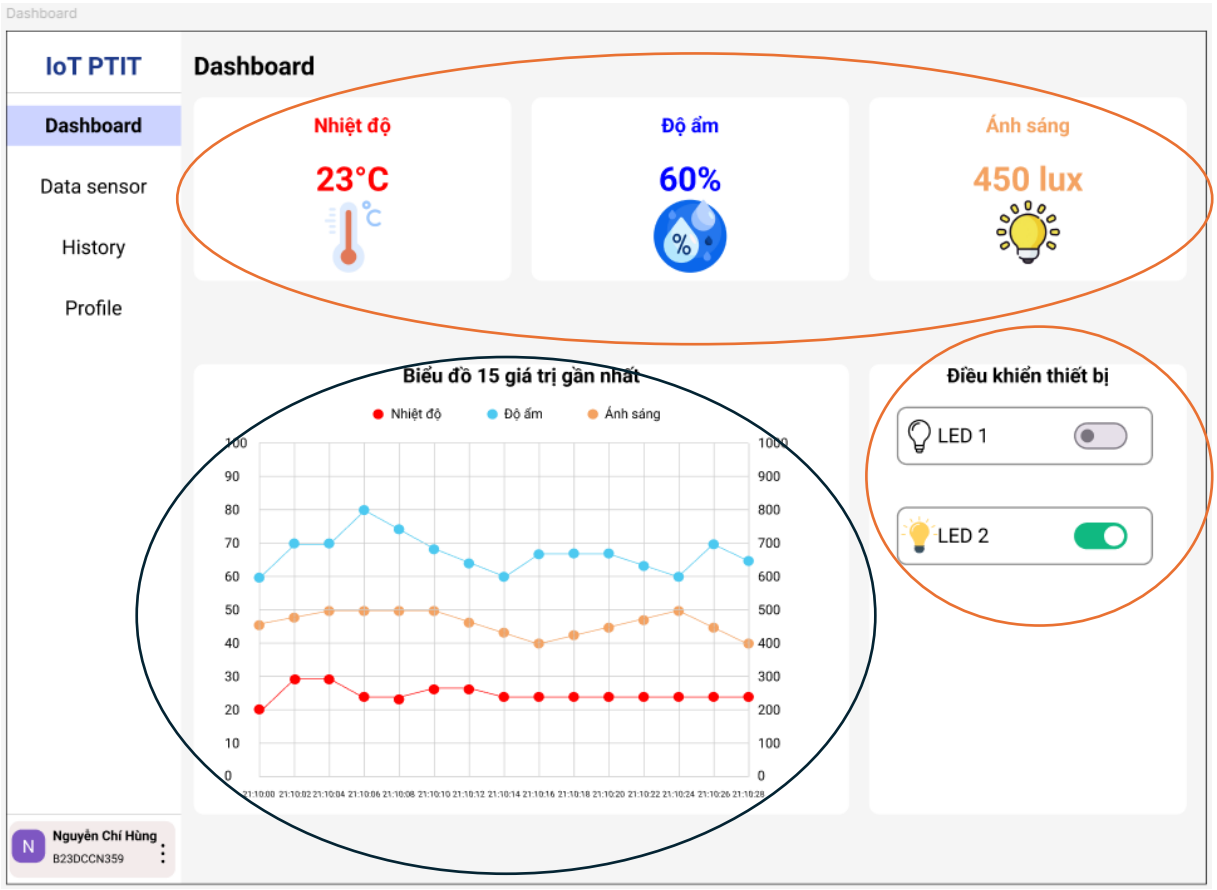
3.2. Use case

3.2.1. Sơ đồ Use case tổng quan



Hình 3.3. Sơ đồ use case tổng quan

3.2.2. Use case 1: View Data On Dashboard



Hình 3.4. Giao diện cho use case 1

a. Mô tả use case

- Use Case Description: Người dùng truy cập trang Dashboard để xem thông số hiện tại của 3 loại cảm biến (nhiệt độ, độ ẩm, ánh sáng), biểu đồ 15 giá trị gần nhất và trạng thái hoạt động (ON/OFF) của 2 thiết bị LED. Dữ liệu được cập nhật tự động theo thời gian thực (2 giây/lần).

- Actor:

- User (người dùng hệ thống)

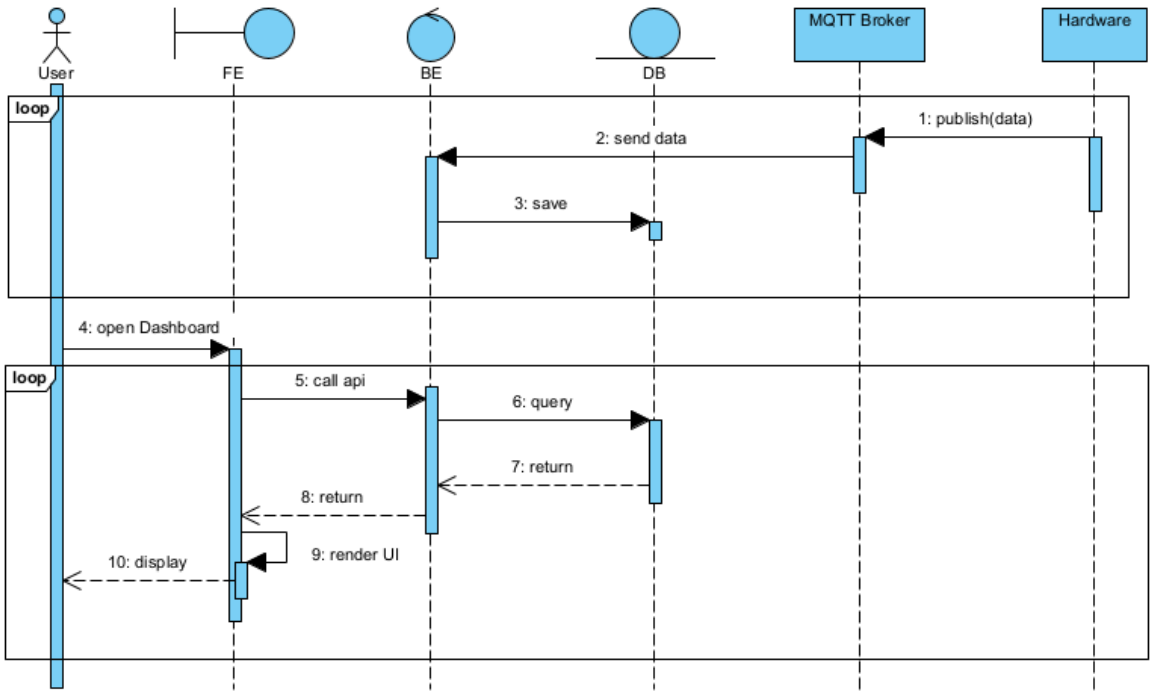
- Pre-Condition:

- User đã đăng nhập thành công vào hệ thống (nếu hệ thống yêu cầu xác thực)
- ESP32 đang hoạt động, kết nối MQTT Broker và gửi dữ liệu cảm biến ổn định
- Backend đã kết nối thành công tới MQTT Broker và Database
- Có ít nhất dữ liệu cảm biến trong Database để hiển thị

- Post-Condition:

- Trang Dashboard hiển thị đầy đủ: giá trị hiện tại của 3 cảm biến, biểu đồ 15 giá trị gần nhất, trạng thái ON/OFF của LED1 và LED2
- Dữ liệu trên giao diện được tự động làm mới mỗi 2 giây mà không cần user thao tác lại
- Nếu không có dữ liệu hoặc mất kết nối, hệ thống hiển thị trạng thái phù hợp (ví dụ: "Không có dữ liệu" hoặc "Mất kết nối")

b. Sequence diagram

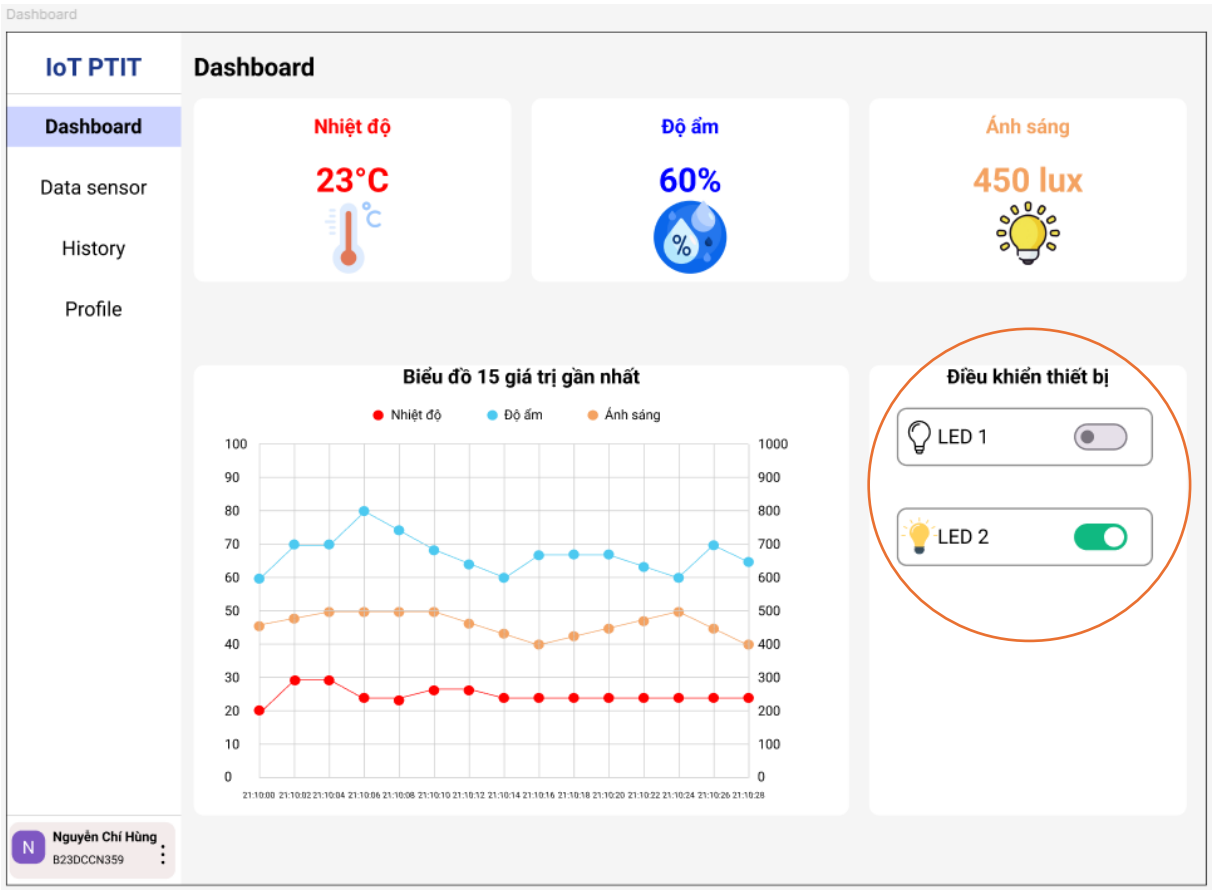


Hình 3.5. Biểu đồ tuần tự use case 1

1. **Hardware (HW)** đọc các thông số môi trường thực tế (nhiệt độ, độ ẩm, ánh sáng) và tiến hành publish (gửi) dữ liệu lên **MQTT Broker** mỗi 2 giây.
2. **MQTT Broker** đưa message chứa dữ liệu này đến **Backend (BE)** (do BE đã subscribe sẵn topic cảm biến từ trước).
3. **BE** lưu dữ liệu vào **Database (DB)**.
4. **User** mở trình duyệt web và truy cập vào trang Dashboard.
5. **Frontend (FE)** gửi các request HTTP GET đến **BE** thông qua các API:
    - /api/sensors/latest (lấy giá trị thực tế hiện tại).
    - /api/sensors/chart?limit=15 (lấy 15 bản ghi gần nhất cho biểu đồ).
    - /api/devices/status (lấy trạng thái bật/tắt của đèn LED).
6. **BE** tiếp nhận request và tiến hành truy vấn **DB**.
7. **DB** thực thi truy vấn và trả kết quả về cho **BE**.
8. **BE** đóng gói các kết quả và trả về cho **FE**

9. **FE** nhận kết quả, tiến hành render giao diện: hiển thị số liệu lên 3 ô thông số lớn, vẽ đồ thị 15 điểm bằng thư viện Chart.js, và cài đặt đúng trạng thái cho nút toggle của LED 1 và LED 2.
10. **FE** hiển thị giao diện cho người dùng.

3.2.3. Use case 2: Control Device



Hình 3.6. Use case điều khiển thiết bị

a. Mô tả use case

- Use Case Description: Người dùng bật hoặc tắt thiết bị LED (LED1 hoặc LED2) thông qua giao diện Dashboard bằng nút toggle. Yêu cầu điều khiển được gửi từ Frontend đến Backend, sau đó Backend gửi lệnh qua MQTT Broker xuống ESP32 để thực thi, và trạng thái thực tế được ghi nhận lại vào hệ thống.

- Actor:

- User (người thực hiện thao tác điều khiển)
- Hardware/ESP32 (thiết bị thực thi lệnh và phản hồi trạng thái)

- Pre-Condition:

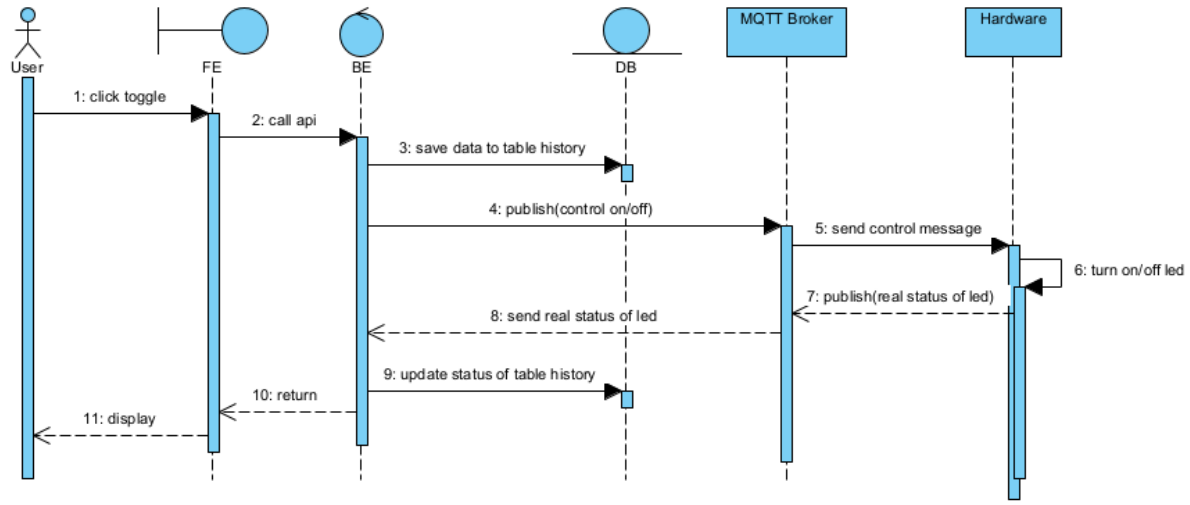
- User đã đăng nhập thành công vào hệ thống

- ESP32 đang kết nối với MQTT Broker và sẵn sàng nhận lệnh điều khiển
- Backend đã subscribe đúng topic điều khiển và topic trạng thái thiết bị
- Trang Dashboard đã load và hiển thị đúng trạng thái hiện tại của LED trước khi thao tác

- Post-Condition:

- Một bản ghi mới được tạo trong bảng History với action (ON/OFF) và status ban đầu là "WAITING"
- ESP32 nhận lệnh và thực thi bật/tắt LED thực tế
- Trạng thái thực tế (ON/OFF) được ESP32 gửi ngược về, cập nhật vào bản ghi History (thay thế "WAITING")
- Giao diện toggle switch trên Dashboard hiển thị đúng trạng thái thực tế sau khi thiết bị phản hồi
- Trường hợp thiết bị không phản hồi sau thời gian timeout, trạng thái được đánh dấu là lỗi (ví dụ: "FAILED")

b. Sequence diagram

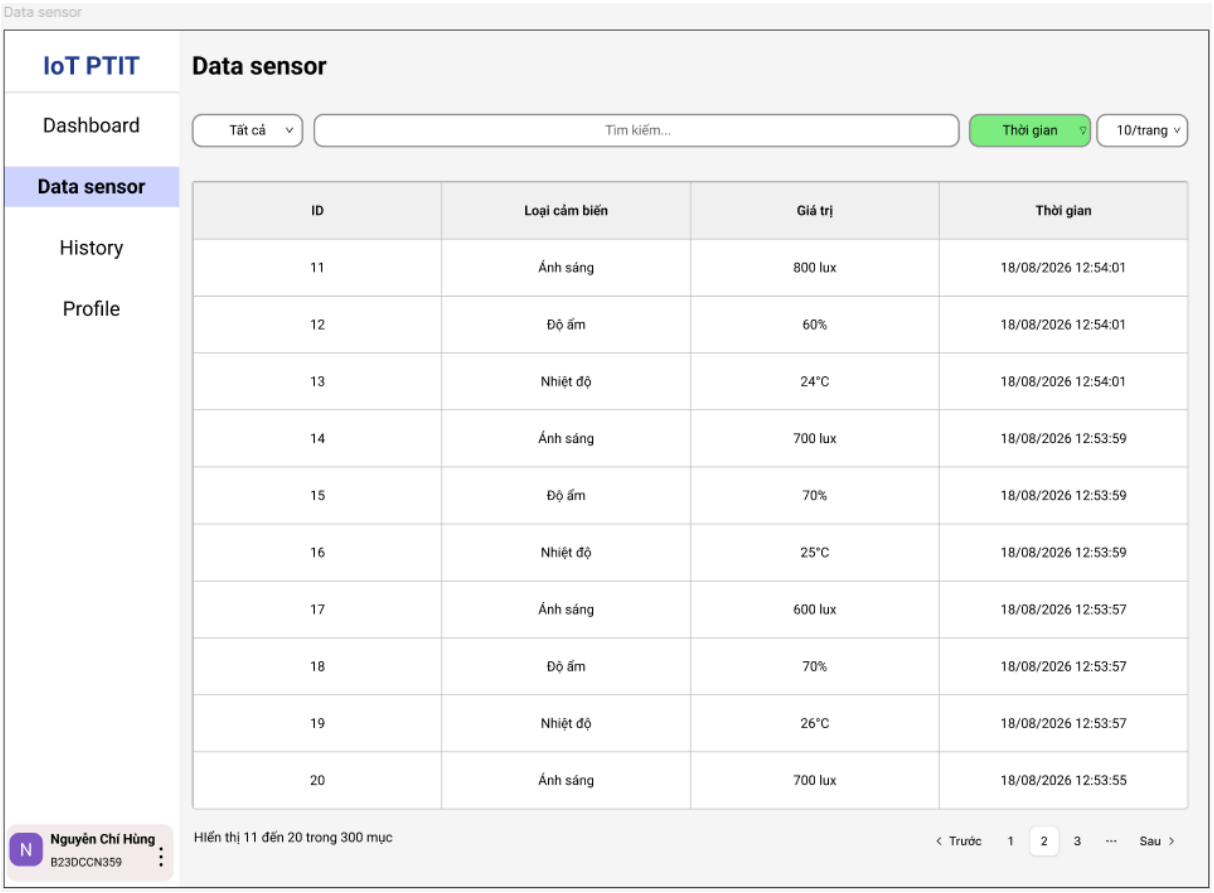


Hình 3.7. Biểu đồ tuần tự use case 2

1. **User** tương tác với giao diện bằng cách click vào nút gạt (toggle) để ra lệnh điều khiển thiết bị (ví dụ: yêu cầu bật LED 1).
2. **Frontend (FE)** gửi một request HTTP POST đến **Backend (BE)** thông qua API /api/devices/{id}/control kèm theo dữ liệu payload chứa hành động mong muốn (ví dụ: action = ON).

3. **BE** tiếp nhận request và thực thi lệnh INSERT vào bảng history trong **Database (DB)** một bản ghi mới để ghi nhận thao tác, thiết lập trạng thái ban đầu là chờ xử lý (status = WAITING).
4. **BE** đóng gói lệnh điều khiển thành bản tin JSON và publish (gửi) lên **MQTT Broker** vào topic điều khiển tương ứng (ví dụ: iot/led1/control với nội dung {"action":"ON"}).
5. **MQTT Broker** nhận bản tin và chuyển tiếp (forward) gói tin xuống **Hardware (HW - ESP32)** (do ESP32 đã đăng ký/subscribe topic này từ trước).
6. **HW** nhận lệnh điều khiển, giải mã JSON và thực thi thay đổi mức logic trên vi điều khiển (sử dụng hàm digitalWrite) để thực sự bật/tắt đèn LED vật lý.
7. **HW** đóng gói trạng thái thực tế của thiết bị và publish ngược lại lên **MQTT Broker** vào topic trạng thái (ví dụ: iot/led1/status với nội dung {"status":"ON"}).
8. **MQTT Broker** tiếp nhận và chuyển tiếp bản tin trạng thái này về cho **BE** (do BE đã subscribe topic trạng thái của thiết bị).
9. **BE** phân tích gói tin nhận được và thực thi lệnh UPDATE vào **DB** để cập nhật trạng thái thực (ON hoặc OFF) cho bản ghi lịch sử ở bước 3, thay thế cho trạng thái WAITING.
10. **BE** trả về trạng thái bật/tắt có thành công không để **FE** hiển thị thông báo cho người dùng.

3.2.4. Use case 3: View Sensor Data History



Hình 3.8. Giao diện cho use case 3

a. Mô tả use case

- Use Case Description: Người dùng truy cập trang Data sensor để xem toàn bộ lịch sử dữ liệu đã ghi nhận từ các cảm biến, có thể tìm kiếm theo thời gian, các loại cảm biến, sắp xếp theo thời gian và có phân trang kết quả.

- Actor:

- User (người xem và tra cứu dữ liệu)

- Pre-Condition:

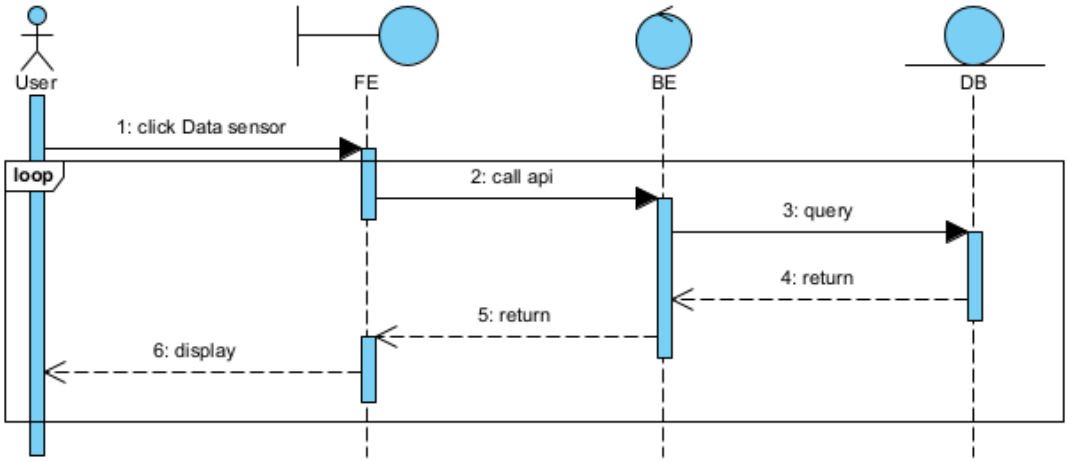
- User đã đăng nhập thành công vào hệ thống
- Database đã có dữ liệu cảm biến được lưu trữ từ trước (thông qua quá trình thu thập dữ liệu nền)
- Backend cung cấp đúng API hỗ trợ filter, sort, phân trang

- Post-Condition:

- Bảng dữ liệu hiển thị đúng danh sách bản ghi (ID, loại cảm biến, giá trị, thời gian) phù hợp với điều kiện tìm kiếm/lọc/sắp xếp mà user đã chọn

- Khi user thay đổi điều kiện tìm kiếm, lọc, sắp xếp hoặc chuyển trang, bảng dữ liệu được cập nhật tương ứng mà không mất các điều kiện đã áp dụng trước đó
- Nếu không có dữ liệu phù hợp, hệ thống hiển thị thông báo "Không tìm thấy dữ liệu"

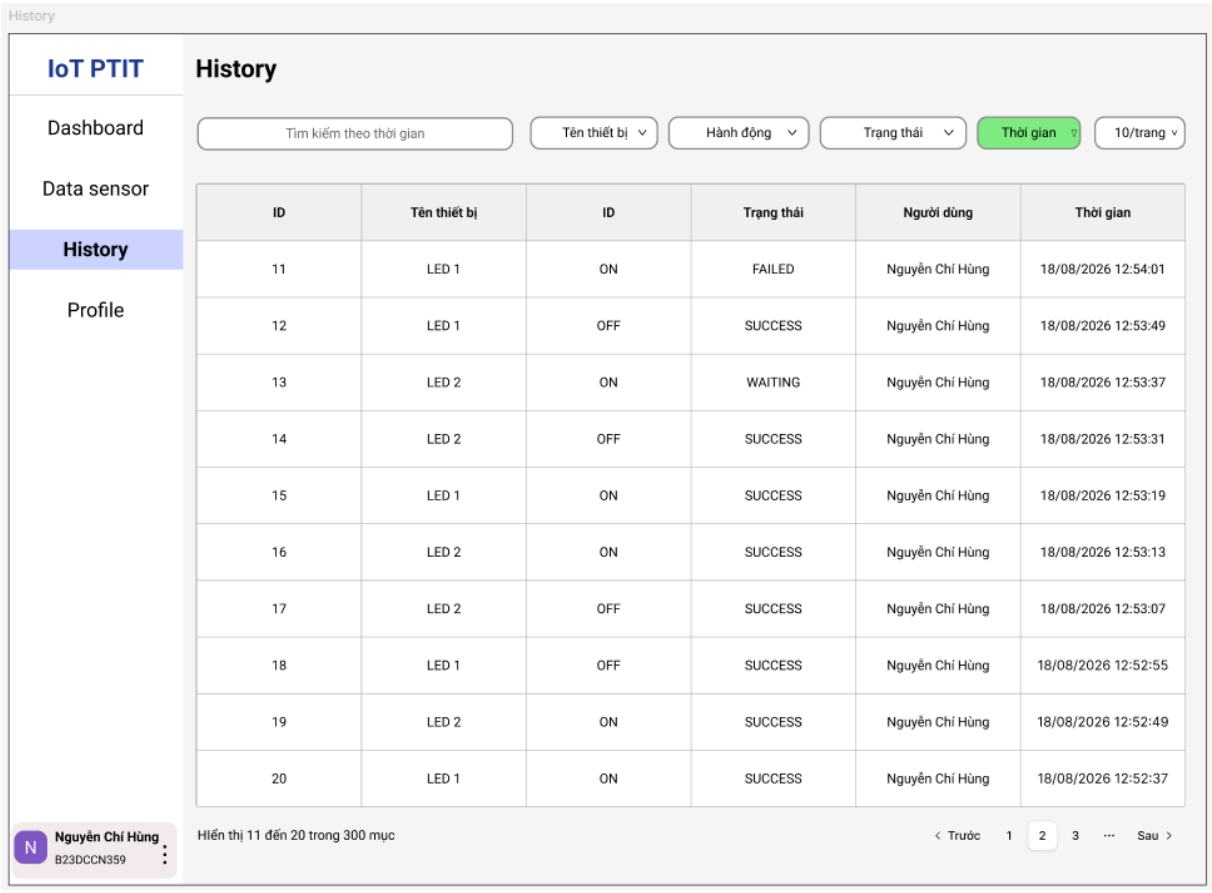
b. Sequence diagram



Hình 3.9. Biểu đồ tuần tự use case 3

1. **User** chọn mục "Data sensor".
2. **Frontend (FE)** tiếp nhận sự kiện và gửi một request HTTP GET đến **Backend (BE)** thông qua API /api/sensors?page=1&size=10&sort=desc để lấy dữ liệu mặc định.
3. **BE** tiếp nhận request và thực thi truy vấn vào bảng sensor_data trong **Database (DB)**
4. **DB** xử lý và trả về cho **BE** danh sách bản ghi cùng với tổng số lượng bản ghi hiện có.
5. **BE** đóng gói dữ liệu thành định dạng JSON và trả về cho **FE**.
6. **FE** nhận JSON, tiến hành render (kết xuất) dữ liệu thành dạng bảng và hiển thị thanh phân trang cho **User**.

3.2.5. Use case 4: View Device Activity History



Hình 3.10. Giao diện cho use case 4

a. Mô tả use case

- Use Case Description: Người dùng truy cập trang History để xem lại toàn bộ lịch sử các thao tác điều khiển thiết bị (LED1, LED2), bao gồm hành động được yêu cầu và trạng thái thực tế sau khi thực thi. Người dùng có thể tìm kiếm theo thời gian, lọc theo tên thiết bị, lọc theo hành động, lọc theo trạng thái, sắp xếp theo thời gian và có phân trang kết quả.

- Actor:

- User (người xem và tra cứu lịch sử hoạt động thiết bị)

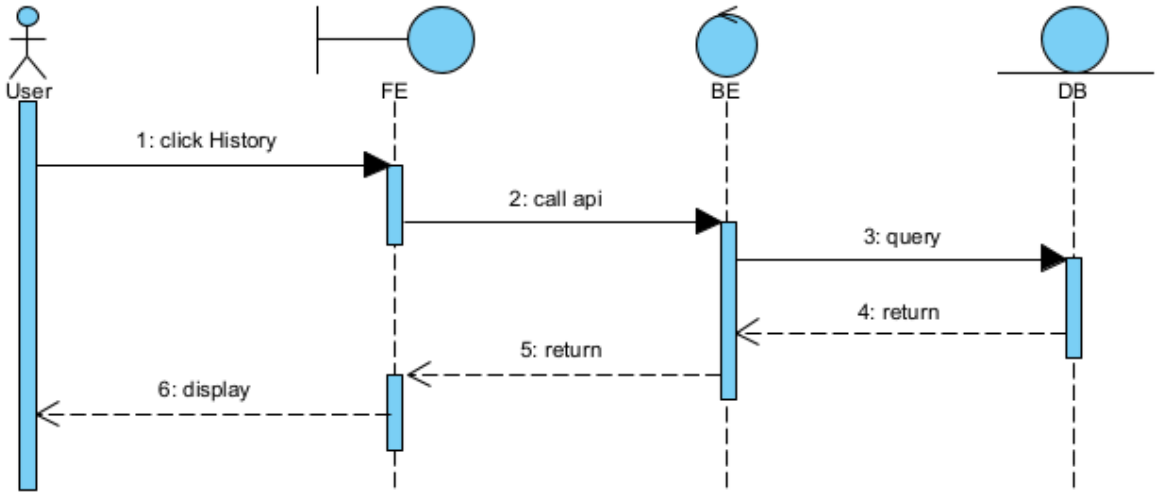
- Pre-Condition:

- User đã đăng nhập thành công vào hệ thống
- Database đã có ít nhất một bản ghi lịch sử điều khiển thiết bị (được tạo ra từ use case Control Device)
- Backend cung cấp đúng API hỗ trợ filter, sort, phân trang cho bảng History

- Post-Condition:

- Bảng lịch sử hiển thị đúng danh sách bản ghi (ID, tên thiết bị, hành động, trạng thái, thời gian) phù hợp với điều kiện tìm kiếm/lọc/sắp xếp mà user đã chọn
- Khi user thay đổi điều kiện tìm kiếm, lọc, sắp xếp hoặc chuyển trang, bảng dữ liệu được cập nhật tương ứng mà không mất các điều kiện đã áp dụng trước đó
- Các bản ghi có trạng thái "WAITING" (đang chờ thiết bị phản hồi) vẫn hiển thị đúng thực trạng tại thời điểm truy vấn
- Nếu không có dữ liệu phù hợp, hệ thống hiển thị thông báo "Không tìm thấy dữ liệu"

b. Sequence diagram

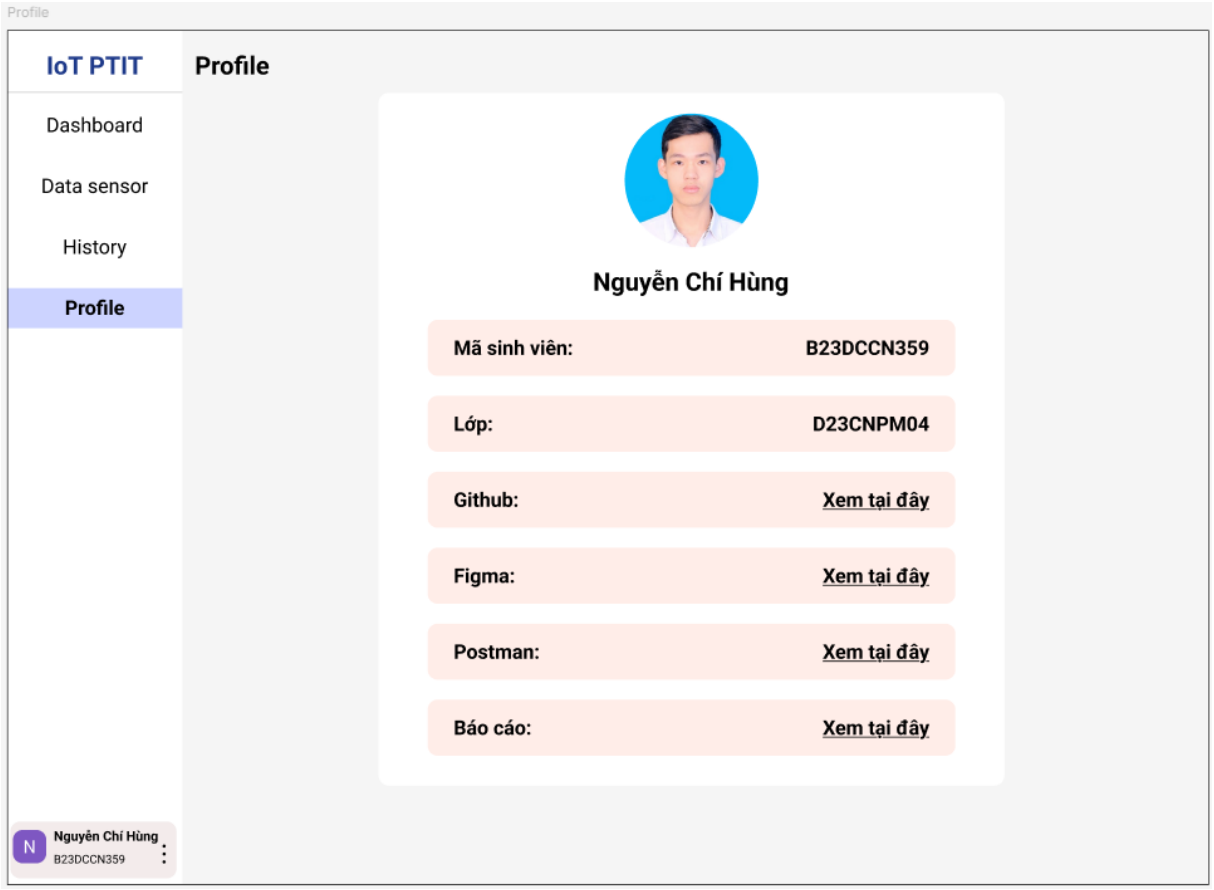


Hình 3.11. Biểu đồ tuần tự use case 4

1. User chọn mục "History".
2. **Frontend (FE)** tiếp nhận sự kiện và gửi một request HTTP GET đến **Backend (BE)** thông qua API /api/history?page=1&size=10&sort=desc để lấy dữ liệu mặc định.
3. **BE** tiếp nhận request và thực thi truy vấn vào bảng history trong **Database (DB)**.
4. **DB** xử lý và trả về cho **BE** danh sách bản ghi cùng với tổng số lượng bản ghi hiện có.
5. **BE** đóng gói dữ liệu thành định dạng JSON và trả về cho **FE**.

6. **FE** nhận JSON, tiến hành render (kết xuất) dữ liệu thành dạng bảng và hiển thị thanh phân trang cho **User**.

3.2.6. Use case 5: View Profile



Hình 3.12. Giao diện use case 5

a. Mô tả use case

- Use Case Description: Người dùng truy cập trang Profile để xem các thông tin: họ tên, mã sinh viên, lớp, github, figma, postman, báo cáo.

- Actor:

- User (người dùng hệ thống)

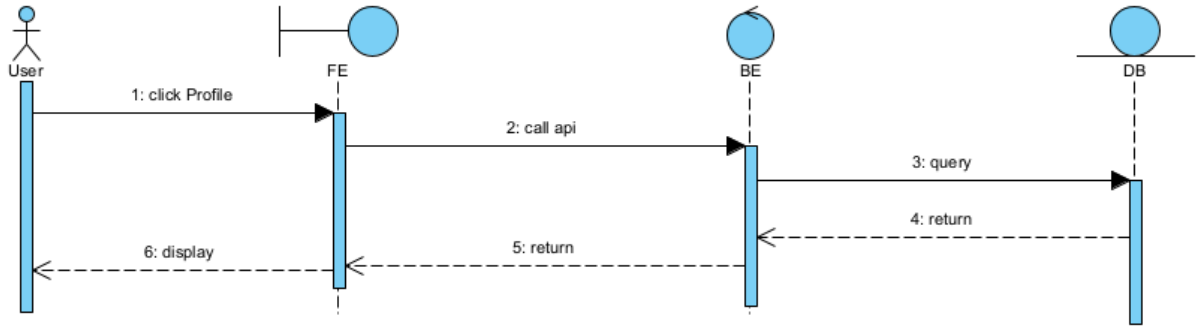
- Pre-Condition:

- User đã đăng nhập thành công vào hệ thống (nếu hệ thống yêu cầu xác thực)
- Có dữ liệu trong Database để hiển thị

- Post-Condition:

- Người dùng xem được các thông tin cá nhân

b. Sequence diagram



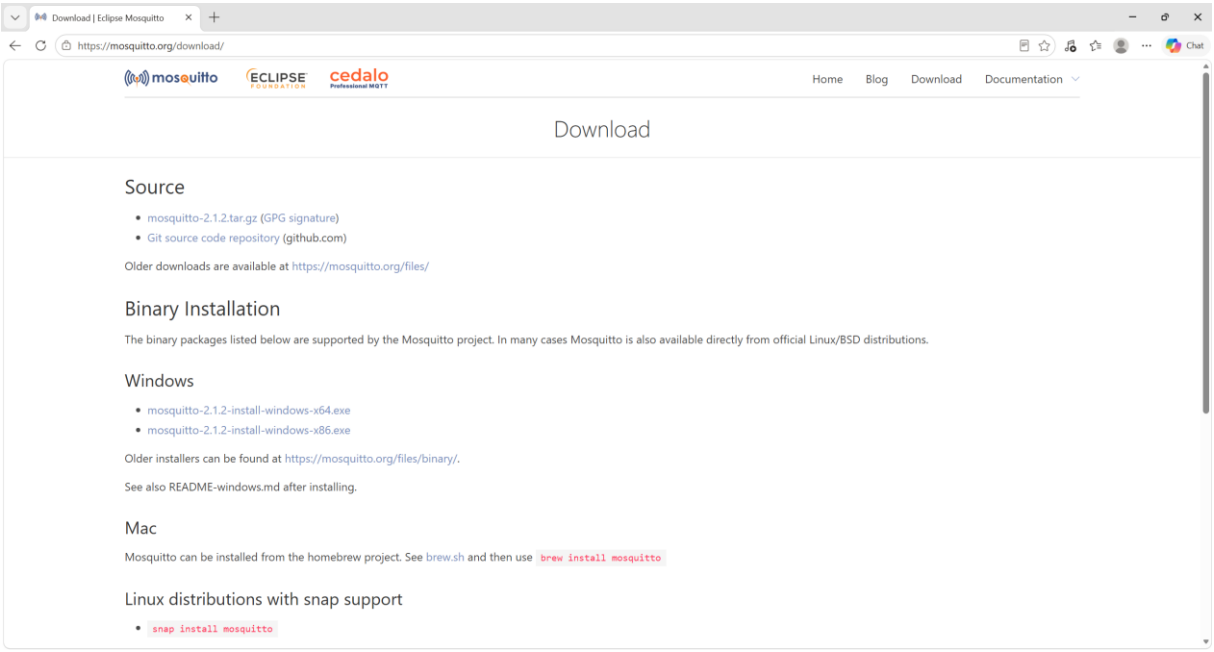
Hình 3.13. Biểu đồ tuần tự use case 5

1. User chọn mục "Profile".
2. **Frontend (FE)** tiếp nhận sự kiện và gửi một request HTTP GET đến **Backend (BE)** thông qua API /api/user để lấy dữ liệu mặc định.
3. **BE** tiếp nhận request và thực thi truy vấn vào bảng user trong **Database (DB)**.
4. **DB** xử lý và trả về cho **BE** các thông tin cá nhân.
5. **BE** đóng gói dữ liệu thành định dạng JSON và trả về cho **FE**.
6. **FE** nhận JSON, tiến hành render (kết xuất) dữ liệu và hiển thị cho **User**.

CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG

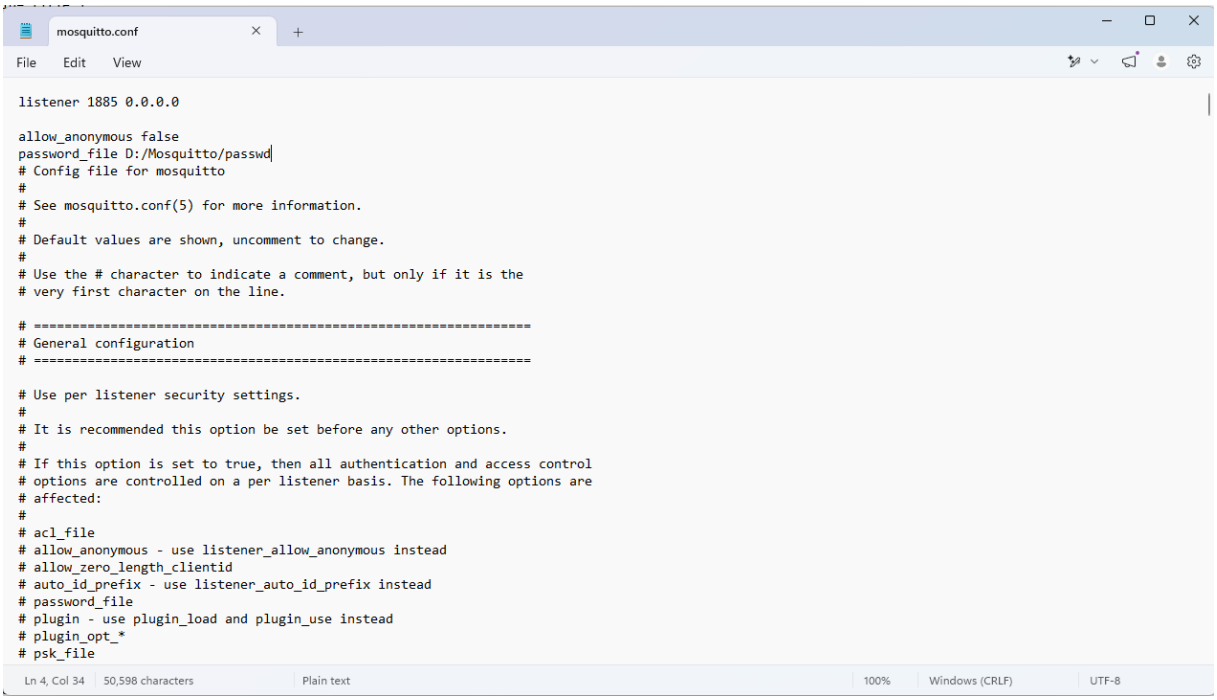
4.1. Cài đặt mosquitto

- Tải mosquitto từ trang web: https://mosquitto.org/download/



Hình 4.1. Trang web download mosquitto

- Chỉnh cấu hình trong file mosquitto.conf:

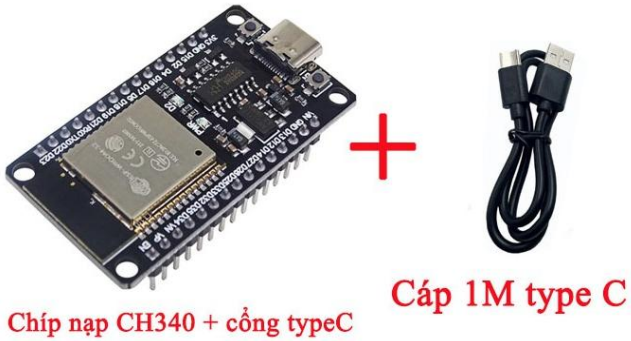
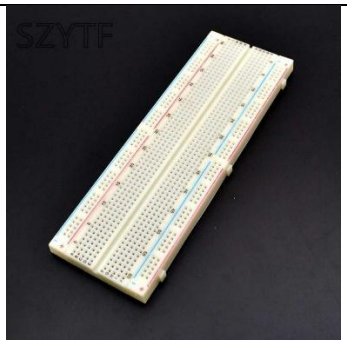


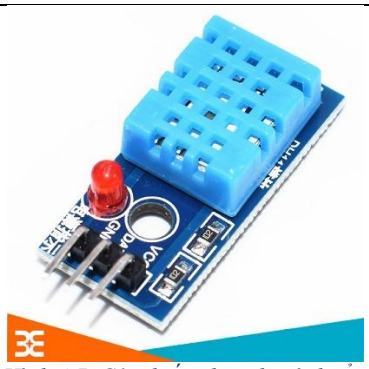
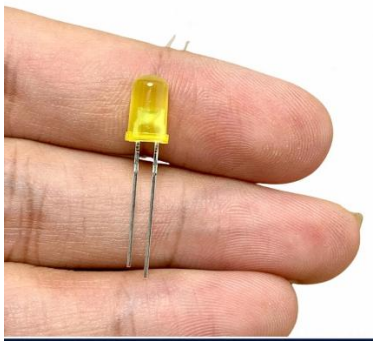
Hình 4.2. File cấu hình mosquitto.conf

- listener 1885 0.0.0.0: Cấu hình Mosquitto lắng nghe kết nối MQTT trên cổng 1885 và tất cả các địa chỉ IPv4 của máy chủ.
- allow_anonymous false: Không cho phép kết nối ẩn danh, client phải xác thực trước khi kết nối.
- password_file D:/Mosquitto/passwd: Chỉ định đường dẫn đến tệp chứa thông tin tài khoản và mật khẩu để xác thực client MQTT.

4.2. Lắp đặt thiết bị

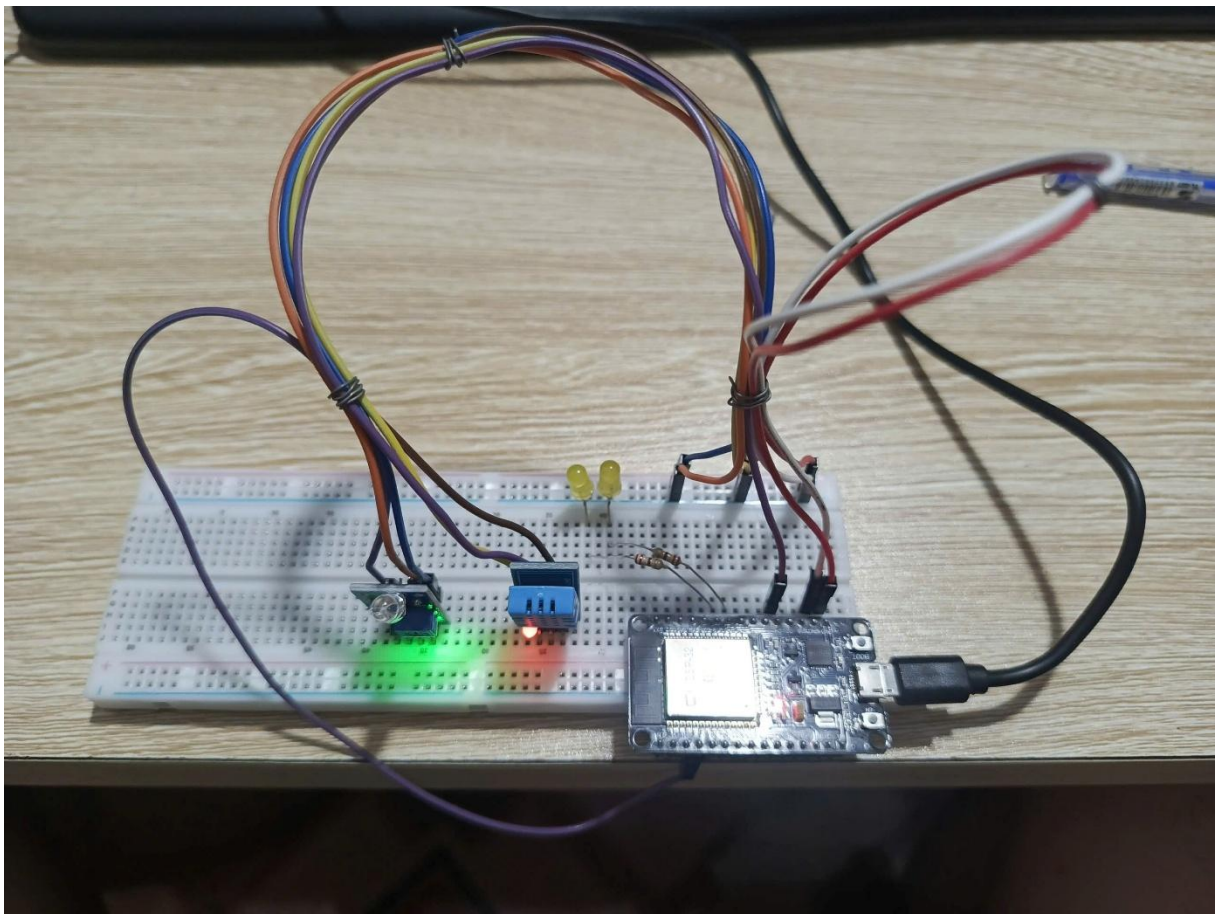
4.2.1. Danh sách các thiết bị

Tên thiết bị	Hình ảnh
ESP 32 CH340 + cáp để cấp nguồn và truyền dữ liệu x 1 cái	 *Hình 4.3. Kit ESP32 + cáp*
Breadboard x 1 cái	 *Hình 4.5. Breadboard*
Cảm biến ánh sáng Photodiode-TH373 x 1 cái	 *Hình 4.6. Cảm biến ánh sáng*

Cảm biến nhiệt độ, độ ẩm DHT11 -TH024 x 1 cái	 *Hình 4.7. Cảm biến nhiệt độ và độ ẩm*
Led 5mm x 2 cái	 *Hình 4.8. Đèn led 5mm*
Điền trở 220R x 2 cái	 *Hình 4.9. Điện trở 220R*
Dây đực – đực x 7 cái	 *Hình 4.10. Dây đực - đực*

Dây đực cái x 1 cái	 *Hình 4.11. Dây đực - cái*

4.2.2 Lắp đặt



Hình 4.12. Phần cứng sau khi lắp đặt hoàn chỉnh

4.3. Lập trình phần cứng

4.3.1. Khai báo thư viện, các biến và nguyên mẫu hàm

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>
#include <ArduinoJson.h>
#include <Preferences.h>

// ── CẤU HÌNH WIFI ──────────────────────────────────────────
const char* WIFI_SSID     = "LAPTOP-QSELFC71 1279";       // TODO: đổi thành SSID thực tế
const char* WIFI_PASSWORD = "7y|2737U";   // TODO: đổi thành mật khẩu thực tế

// ── CẤU HÌNH MQTT BROKER ──────────────────────────────────
const char* MQTT_BROKER    = "192.168.137.1";  // TODO: đổi thành IP máy chạy backend/broker
const int   MQTT_PORT      = 1885;
const char* MQTT_USER      = "mqttuser";      // TODO: đổi thành username MQTT
const char* MQTT_PASSWORD  = "hung123";       // TODO: đổi thành password MQTT

// Client ID duy nhất dựa trên MAC address (tránh conflict với client khác)
char MQTT_CLIENT_ID[32];   // sẽ điền trong setup()
// ── MQTT TOPICS (KHÔNG THAY ĐỔI) ──────────────────────────
const char* TOPIC_SENSOR_DATA    = "data/sensors";
const char* TOPIC_DEVICE_CONTROL = "device/control";
const char* TOPIC_DEVICE_STATUS  = "device/status";

// ── CẤU HÌNH CHÂN GPIO ────────────────────────────────────
#define DHT_PIN         4       // Chân DATA của DHT11
#define DHT_TYPE        DHT11   // Loại cảm biến
#define LIGHT_SENSOR_PIN 34     // Chân ADC đọc quang trở/photodiode (ADC1 channel)
#define LED1_PIN        18      // LED 1 cắm chân D18
#define LED2_PIN        19      // LED 2 cắm chân D19

// ── HẰNG SỐ ───────────────────────────────────────────────
const long PUBLISH_INTERVAL_MS = 2000;   // Publish dữ liệu mỗi 2 giây
const int  ADC_MAX             = 4095;   // ESP32 ADC 12-bit

// ── BIẾN TRẠNG THÁI ───────────────────────────────────────
bool led1State = false;
bool led2State = false;
unsigned long lastPublishTime = 0;

// ── KHỞI TẠO ĐỐI TƯỢNG ────────────────────────────────────
DHT dht(DHT_PIN, DHT_TYPE);
WiFiClient wifiClient;
PubSubClient mqttClient(wifiClient);
Preferences preferences;

// ── FORWARD DECLARATIONS ──────────────────────────────────
void connectWiFi();
void connectMQTT();
void publishSensorData();
void publishDeviceStatus(int deviceId, const char* deviceKey, bool isOn);
void onMqttMessage(char* topic, byte* payload, unsigned int length);
float readLightLux();
```

Hình 4.13. Khai báo thư viện, các biến, nguyên hàm

- **Khai báo thư viện:** Sử dụng các thư viện WiFi.h, PubSubClient.h, DHT.h, ArduinoJson.h và Preferences.h để hỗ trợ kết nối Wi-Fi, giao tiếp MQTT, đọc dữ liệu từ cảm biến DHT11, xử lý dữ liệu JSON và lưu trữ dữ liệu cấu hình trong bộ nhớ không mất dữ liệu của ESP32.
- **Cấu hình Wi-Fi:** Khai báo tên mạng và mật khẩu Wi-Fi để ESP32 kết nối với mạng nội bộ.
- **Cấu hình MQTT Broker:** Khai báo địa chỉ IP, cổng kết nối và thông tin xác thực để ESP32 giao tiếp với MQTT Broker.
- **Thiết lập Client ID:** Khai báo biến lưu mã định danh của ESP32, dự kiến được tạo dựa trên địa chỉ MAC để tránh trùng lặp với các thiết bị khác.
- **Khai báo các topic MQTT:** Sử dụng ba topic gồm data/sensors để gửi dữ liệu cảm biến, device/control để nhận lệnh điều khiển và device/status để gửi trạng thái thiết bị.
- **Cấu hình chân GPIO:** Khai báo chân GPIO 4 kết nối cảm biến DHT11, GPIO 34 đọc tín hiệu cảm biến ánh sáng, GPIO 18 điều khiển LED 1 và GPIO 19 điều khiển LED 2.
- **Thiết lập thông số hoạt động:** Quy định chu kỳ gửi dữ liệu cảm biến là 2 giây và giá trị ADC tối đa là 4095 tương ứng với độ phân giải 12 bit.
- **Quản lý trạng thái thiết bị:** Sử dụng các biến led1State và led2State để lưu trạng thái bật/tắt của hai đèn LED.
- **Khởi tạo đối tượng:** Tạo các đối tượng để giao tiếp với cảm biến DHT11, kết nối Wi-Fi và thực hiện truyền nhận dữ liệu MQTT.
- **Khai báo các hàm chức năng:** Xây dựng các hàm dự kiến phục vụ kết nối Wi-Fi, kết nối MQTT, gửi dữ liệu cảm biến, gửi trạng thái thiết bị, xử lý lệnh MQTT và đọc giá trị cảm biến ánh sáng.

4.3.2. Khởi tạo hệ thống và kết nối mạng

```
57  void setup() {
58    Serial.begin(115200);
59    Serial.println("\n[ESP32] IoT Dashboard Firmware Starting...");
60
61    // Tạo client ID duy nhất từ MAC address (6 bytes cuối)
62    uint8_t mac[6];
63    WiFi.macAddress(mac);
64    snprintf(MQTT_CLIENT_ID, sizeof(MQTT_CLIENT_ID),
65             "esp32-%02X%02X%02X", mac[3], mac[4], mac[5]);
66    Serial.printf("[MQTT] Client ID: %s\n", MQTT_CLIENT_ID);
67
68    // Khởi tạo chân GPIO
69    pinMode(LED1_PIN, OUTPUT);
70    pinMode(LED2_PIN, OUTPUT);
71
72    // Đọc trạng thái đèn đã lưu trong Flash NVS trước khi ngắt kết nối / mất điện
73    preferences.begin("led_state", false);
74    led1State = preferences.getBool("led1", false);
75    led2State = preferences.getBool("led2", false);
76
77    // Khôi phục ngay lập tức trạng thái đèn
78    digitalWrite(LED1_PIN, led1State ? HIGH : LOW);
79    digitalWrite(LED2_PIN, led2State ? HIGH : LOW);
80    Serial.printf("[RESTORE] LED1: %s, LED2: %s\n", led1State ? "ON" : "OFF", led2State ? "ON" : "OFF");
81
82    // Khởi tạo cảm biến DHT
83    dht.begin();
84
85    // Kết nối WiFi
86    connectWiFi();
87
88    // Cấu hình MQTT
89    mqttClient.setServer(MQTT_BROKER, MQTT_PORT);
90    mqttClient.setCallback(onMqttMessage);
91    mqttClient.setBufferSize(512);  // Tăng buffer nếu payload lớn
92
93    // Kết nối MQTT
94    connectMQTT();
95
96    Serial.println("[ESP32] Setup complete.");
97  }
```

Hình 4.14. Hàm setup()

- Hàm setup():

- **Khởi tạo Serial:** Sử dụng Serial.begin(115200) để thiết lập giao tiếp Serial với tốc độ 115200 baud, phục vụ việc hiển thị thông báo, theo dõi quá trình khởi động và kiểm tra trạng thái hoạt động của ESP32 thông qua Serial Monitor.
- **Tạo mã định danh MQTT:** Sử dụng WiFi.macAddress(mac) để lấy địa chỉ MAC của ESP32 và lưu vào mảng mac gồm 6 byte. Sau đó, hàm snprintf() lấy ba byte cuối của địa chỉ MAC để tạo mã định danh có dạng esp32-XXXXXX, lưu vào biến MQTT_CLIENT_ID. Cách làm này giúp giảm nguy cơ trùng lặp Client ID khi có nhiều thiết bị cùng kết nối đến MQTT Broker. Mã định danh sau đó được hiển thị trên Serial Monitor để thuận tiện theo dõi.

- **Khởi tạo chân GPIO:** Sử dụng hàm pinMode() để cấu hình hai chân LED1_PIN (GPIO 18) và LED2_PIN (GPIO 19) ở chế độ đầu ra (OUTPUT), cho phép ESP32 điều khiển trạng thái bật/tắt của hai đèn LED.
- **Khởi tạo bộ nhớ lưu trạng thái:** Sử dụng preferences.begin("led_state", false) để mở vùng lưu trữ có tên led_state trong bộ nhớ Flash NVS của ESP32. Tham số false cho phép đọc và ghi dữ liệu. Tiếp theo, hai hàm preferences.getBool("led1", false) và preferences.getBool("led2", false) đọc trạng thái đã lưu của LED 1 và LED 2. Nếu chưa có dữ liệu được lưu trước đó, trạng thái mặc định của mỗi đèn là false (tắt).
- **Khôi phục trạng thái đèn LED:** Sử dụng hàm digitalWrite() để thiết lập lại trạng thái của hai đèn theo dữ liệu đã đọc từ bộ nhớ. Nếu biến led1State hoặc led2State là true, chân GPIO tương ứng được đặt ở mức HIGH; nếu là false, chân GPIO được đặt ở mức LOW. Đồng thời, hàm Serial.printf() hiển thị trạng thái khôi phục của từng đèn trên Serial Monitor. Nhờ đó, chương trình có thể khôi phục trạng thái đèn đã lưu sau khi ESP32 khởi động lại hoặc mất nguồn, với điều kiện trạng thái đã được lưu trước đó và mức logic phù hợp với mạch LED.
- **Khởi tạo cảm biến:** Sử dụng hàm dht.begin() để khởi tạo cảm biến DHT11, chuẩn bị cho quá trình đọc dữ liệu nhiệt độ và độ ẩm trong các phần tiếp theo của chương trình.
- **Kết nối Wi-Fi:** Gọi hàm connectWiFi() để thiết lập kết nối giữa ESP32 và mạng Wi-Fi đã được cấu hình thông qua các biến WIFI_SSID và WIFI_PASSWORD. Kết nối này là điều kiện cần để thiết bị có thể giao tiếp với MQTT Broker qua mạng.
- **Cấu hình MQTT:** Sử dụng mqttClient.setServer(MQTT_BROKER, MQTT_PORT) để thiết lập địa chỉ IP và cổng của MQTT Broker. Hàm mqttClient.setCallback(onMqttMessage) đăng ký hàm xử lý thông điệp nhận được từ MQTT, cho phép ESP32 tiếp nhận và xử lý các lệnh điều khiển từ hệ thống. Ngoài ra, mqttClient.setBufferSize(512) thiết lập kích thước bộ đệm MQTT là 512 byte, hỗ trợ xử lý các thông điệp có kích thước lớn hơn mức mặc định của thư viện.
- **Kết nối MQTT:** Gọi hàm connectMQTT() để thiết lập kết nối giữa ESP32 và MQTT Broker. Trong quá trình kết nối, chương trình sử dụng các thông số đã cấu hình như Client ID, tên đăng nhập và mật khẩu MQTT. Hàm này cũng được dự kiến thực hiện đăng ký nhận lệnh từ topic device/control, tùy theo nội dung triển khai bên trong hàm.
- **Thông báo hoàn tất:** Sử dụng Serial.println("[ESP32] Setup complete.") để hiển thị thông báo cho biết quá trình khởi tạo đã hoàn tất. Đây là dấu

hiệu giúp người phát triển theo dõi tiến trình khởi động và kiểm tra chương trình trong Serial Monitor.

```
126  void connectWiFi() {
127    Serial.printf("[WiFi] Connecting to %s", WIFI_SSID);
128    WiFi.mode(WIFI_STA);
129    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
130
131    int attempt = 0;
132    while (WiFi.status() != WL_CONNECTED && attempt < 30) {
133      delay(500);
134      Serial.print(".");
135      attempt++;
136    }
137
138    if (WiFi.status() == WL_CONNECTED) {
139      Serial.printf("\n[WiFi] Connected. IP: %s\n", WiFi.localIP().toString().c_str());
140    } else {
141      Serial.println("\n[WiFi] Failed to connect. Will retry in loop.");
142    }
143  }
```

Hình 4.15. Hàm connectWifi()

- Hàm connectWiFi():

- **Thiết lập chế độ Wi-Fi:** Đặt ESP32 hoạt động ở chế độ trạm (Station) thông qua WiFi.mode(WIFI_STA).
- **Bắt đầu kết nối:** Sử dụng WiFi.begin() với tên mạng và mật khẩu đã khai báo để kết nối Wi-Fi.
- **Kiểm tra trạng thái kết nối:** Chương trình kiểm tra trạng thái Wi-Fi và thử kết nối tối đa 30 lần, mỗi lần cách nhau 500 ms.
- **Thông báo kết nối thành công:** Khi kết nối thành công, chương trình hiển thị địa chỉ IP của ESP32 trên Serial Monitor.
- **Xử lý kết nối thất bại:** Nếu không kết nối được sau số lần thử quy định, chương trình thông báo lỗi và dự kiến thực hiện kết nối lại trong vòng lặp chính.

```
148  void connectMQTT() {
149    int attempt = 0;
150    while (!mqttClient.connected() && attempt < 5) {
151      Serial.printf("[MQTT] Connecting to %s:%d ...\n", MQTT_BROKER, MQTT_PORT);
152      if (mqttClient.connect(MQTT_CLIENT_ID, MQTT_USER, MQTT_PASSWORD)) {
153        Serial.println("[MQTT] Connected!");
154        // Subscribe với QoS 1 để không mất lệnh khi vừa reconnect
155        mqttClient.subscribe(TOPIC_DEVICE_CONTROL, 1);
156        Serial.printf("[MQTT] Subscribed to %s (QoS 1)\n", TOPIC_DEVICE_CONTROL);
157
158        // Đồng bộ ngay trạng thái thực tế lên server sau khi kết nối/kết nối lại
159        publishDeviceStatus(1, "led1", digitalRead(LED1_PIN) == HIGH);
160        publishDeviceStatus(2, "led2", digitalRead(LED2_PIN) == HIGH);
161      } else {
162        Serial.printf("[MQTT] Failed (rc=%d). Retry in 3s...\n", mqttClient.state());
163        delay(3000);
164        attempt++;
165      }
166    }
167  }
```

Hình 4.16. Hàm connectMQTT()

- Hàm connectMQTT():

- **Thực hiện kết nối MQTT:** Sử dụng Client ID, tên đăng nhập và mật khẩu để kết nối ESP32 với MQTT Broker.
- **Giới hạn số lần thử:** Chương trình thử kết nối tối đa 5 lần; nếu thất bại, mỗi lần thử lại cách nhau 3 giây.
- **Đăng ký topic điều khiển:** Khi kết nối thành công, ESP32 đăng ký nhận thông điệp từ topic device/control với QoS 1 nhằm tăng độ tin cậy khi truyền nhận lệnh.
- **Thông báo trạng thái:** Hiển thị kết quả kết nối MQTT trên Serial Monitor; nếu kết nối thất bại, chương trình hiển thị mã lỗi để hỗ trợ kiểm tra và xử lý sự cố.
- **Mục đích:** Đảm bảo ESP32 có thể giao tiếp với MQTT Broker và sẵn sàng tiếp nhận các lệnh điều khiển thiết bị từ hệ thống IoT.

4.3.3. Vòng lặp hoạt động chính

```
102  void loop() {
103    // Đảm bảo WiFi vẫn kết nối
104    if (WiFi.status() != WL_CONNECTED) {
105      Serial.println("[WiFi] Disconnected. Reconnecting...");
106      connectWiFi();
107    }
108
109    // Đảm bảo MQTT vẫn kết nối
110    if (!mqttClient.connected()) {
111      connectMQTT();
112    }
113    mqttClient.loop();
114
115    // Publish dữ liệu cảm biến định kỳ
116    unsigned long now = millis();
117    if (now - lastPublishTime >= PUBLISH_INTERVAL_MS) {
118      lastPublishTime = now;
119      publishSensorData();
120    }
121  }
```

Hình 4.17. Hàm loop()

- Hàm loop():

- **Kiểm tra kết nối Wi-Fi:** Kiểm tra trạng thái kết nối Wi-Fi của ESP32. Nếu mất kết nối, chương trình gọi hàm connectWiFi() để thực hiện kết nối lại.
- **Kiểm tra kết nối MQTT:** Kiểm tra trạng thái kết nối với MQTT Broker. Nếu kết nối bị gián đoạn, chương trình gọi hàm connectMQTT() để thiết lập lại kết nối.
- **Duy trì giao tiếp MQTT:** Sử dụng hàm mqttClient.loop() để duy trì kết nối MQTT, xử lý các thông điệp nhận được và tiếp nhận lệnh điều khiển từ topic đã đăng ký.
- **Gửi dữ liệu cảm biến định kỳ:** Sử dụng hàm millis() để xác định thời gian đã trôi qua kể từ lần gửi dữ liệu gần nhất. Khi đạt khoảng thời gian 2 giây theo cấu hình PUBLISH_INTERVAL_MS, chương trình cập nhật thời điểm gửi và gọi hàm publishSensorData() để thu thập, gửi dữ liệu cảm biến lên MQTT Broker.

4.3.4. Đọc và gửi dữ liệu cảm biến

```
172  void publishSensorData() {
173    float temperature = dht.readTemperature();
174    float humidity    = dht.readHumidity();
175    float light       = readLightLux();
176
177    // Kiểm tra lỗi DHT
178    if (isnan(temperature) || isnan(humidity)) {
179      Serial.println("[DHT] Read error! Check sensor wiring.");
180      return;
181    }
182
183    // Tạo JSON payload (không gửi timestamp, server tự lấy thời gian thực)
184    StaticJsonDocument<256> doc;
185    doc["temperature"] = round(temperature * 10) / 10.0;
186    doc["humidity"]    = round(humidity);
187    doc["light"]       = (int)light;
188
189    char payload[256];
190    serializeJson(doc, payload);
191
192    bool ok = mqttClient.publish(TOPIC_SENSOR_DATA, payload);
193    if (ok) {
194      Serial.printf("[MQTT] Published sensors: %s\n", payload);
195    } else {
196      Serial.println("[MQTT] Publish failed!");
197    }
198  }
```

Hình 4.18. Hàm publishSensorData()

- Hàm publishSensorData():

- **Đọc dữ liệu cảm biến:** Thu thập nhiệt độ và độ ẩm từ cảm biến DHT11, đồng thời gọi hàm readLightLux() để lấy giá trị ánh sáng.
- **Kiểm tra dữ liệu:** Kiểm tra giá trị nhiệt độ và độ ẩm. Nếu cảm biến DHT11 đọc lỗi, chương trình hiển thị thông báo và dừng quá trình gửi dữ liệu.
- **Định dạng dữ liệu JSON:** Sử dụng thư viện ArduinoJson để đóng gói dữ liệu cảm biến thành JSON, bao gồm ba thông số: nhiệt độ, độ ẩm và ánh sáng.
- **Xử lý giá trị đo:** Làm tròn nhiệt độ đến một chữ số thập phân, độ ẩm đến số nguyên và chuyển giá trị ánh sáng sang số nguyên.
- **Gửi dữ liệu qua MQTT:** Gửi dữ liệu JSON lên topic data/sensors để hệ thống giám sát tiếp nhận và hiển thị.

- **Thông báo kết quả:** Hiển thị dữ liệu đã gửi hoặc thông báo lỗi nếu quá trình gửi dữ liệu thất bại.

```
200  // ============================================================
201  // ĐỌC CẢM BIẾN ÁNH SÁNG
202  // ============================================================
203  float readLightLux() {
204    // Lấy trung bình 5 lần đọc để giảm nhiễu ADC
205    long sum = 0;
206    for (int i = 0; i < 5; i++) {
207      sum += analogRead(LIGHT_SENSOR_PIN);
208      delay(2);
209    }
210    int raw = sum / 5;
211
212    // Quang trở (LDR): điện trở TĂNG khi TỐI → điện áp trên LDR THẤP → ADC THẤP
213    // → Cần ĐẢO: raw thấp = tối = lux thấp
214    float lux = (float)(ADC_MAX - raw) / ADC_MAX * 1000.0f;
215    if (lux < 0)    lux = 0;
216    if (lux > 1000) lux = 1000;
217    return lux;
218  }
```

Hình 4.19. Hàm readLightLux()

- Hàm readLightLux():

- **Đọc dữ liệu cảm biến ánh sáng:** Sử dụng chân GPIO 34 để đọc tín hiệu analog từ cảm biến ánh sáng.
- **Lấy giá trị trung bình:** Thực hiện 5 lần đọc ADC, mỗi lần cách nhau 2 ms, sau đó tính giá trị trung bình nhằm giảm ảnh hưởng của nhiễu tín hiệu.
- **Chuyển đổi giá trị ánh sáng:** Sử dụng giá trị ADC trung bình và giá trị ADC tối đa 4095 để tính giá trị ánh sáng theo công thức tuyến tính, với phạm vi quy ước từ 0 đến 1000 lux.
- **Giới hạn giá trị đầu ra:** Đảm bảo giá trị ánh sáng không nhỏ hơn 0 và không vượt quá 1000.
- **Trả về kết quả:** Trả về giá trị ánh sáng kiểu float để hàm publishSensorData() sử dụng khi đóng gói và gửi dữ liệu lên MQTT.

4.3.5. Nhận lệnh và điều khiển thiết bị

```
224  void onMqttMessage(char* topic, byte* payload, unsigned int length) {
225    // Chuyển payload thành chuỗi
226    char msg[512];
227    if (length >= sizeof(msg)) length = sizeof(msg) - 1;
228    memcpy(msg, payload, length);
229    msg[length] = '\0';
230
231    Serial.printf("[MQTT] Message on [%s]: %s\n", topic, msg);
232
233    if (strcmp(topic, TOPIC_DEVICE_CONTROL) == 0) {
234      StaticJsonDocument<256> doc;
235      DeserializationError err = deserializeJson(doc, msg);
236      if (err) {
237        Serial.printf("[JSON] Parse error: %s\n", err.c_str());
238        return;
239      }
240
241      long   deviceId  = doc["deviceId"].as<long>();
242      const char* action = doc["action"].as<const char*>();
243
244      if (!action) return;
245
246      bool turnOn = (strcmp(action, "ON") == 0);
247
248      // Thực hiện digitalWrite theo deviceId
249      if (deviceId == 1) {
250        led1State = turnOn;
251        digitalWrite(LED1_PIN, turnOn ? HIGH : LOW);
252        preferences.putBool("led1", turnOn); // Lưu trạng thái vào Flash NVS
253        Serial.printf("[LED1] Set to %s (saved to flash)\n", turnOn ? "ON" : "OFF");
254
255        // Thêm delay nhỏ mô phỏng độ trễ thực tế
256        delay(50);
257
258        // Đọc lại trạng thái thực tế từ chân GPIO
259        bool actual = (digitalRead(LED1_PIN) == HIGH);
260        publishDeviceStatus(1, "led1", actual);
261
262      } else if (deviceId == 2) {
263        led2State = turnOn;
264        digitalWrite(LED2_PIN, turnOn ? HIGH : LOW);
265        preferences.putBool("led2", turnOn); // Lưu trạng thái vào Flash NVS
266        Serial.printf("[LED2] Set to %s (saved to flash)\n", turnOn ? "ON" : "OFF");
267
268        delay(50);
269
270        bool actual = (digitalRead(LED2_PIN) == HIGH);
271        publishDeviceStatus(2, "led2", actual);
272
273      } else {
274        Serial.printf("[CTRL] Unknown deviceId: %ld\n", deviceId);
275      }
276    }
277  }
```

Hình 4.20. Hàm onMqttMessage()

- Hàm onMqttMessage():

- **Tiếp nhận thông điệp MQTT:** Hàm onMqttMessage(char* topic, byte* payload, unsigned int length) được gọi khi ESP32 nhận được thông điệp từ MQTT Broker. Trong đó, topic chứa tên topic nhận dữ liệu, payload chứa nội dung thông điệp và length là độ dài của dữ liệu nhận được.
- **Chuyển payload thành chuỗi:** Khai báo mảng ký tự msg[512] để lưu nội dung thông điệp nhận được. Nếu độ dài dữ liệu lớn hơn hoặc bằng kích thước mảng, biến length được giới hạn ở mức 511 byte nhằm dành một vị trí cho ký tự kết thúc chuỗi '\0'. Sau đó, hàm memcpy() sao chép dữ liệu từ payload sang msg và thêm ký tự kết thúc chuỗi, giúp chương trình có thể xử lý nội dung MQTT dưới dạng chuỗi ký tự thông thường.
- **Hiển thị thông điệp nhận được:** Sử dụng hàm Serial.printf() để hiển thị topic và nội dung thông điệp lên Serial Monitor. Điều này giúp người phát triển theo dõi dữ liệu được gửi từ hệ thống điều khiển và thuận tiện kiểm tra lỗi trong quá trình vận hành.
- **Kiểm tra topic điều khiển:** Sử dụng hàm strcmp(topic, TOPIC_DEVICE_CONTROL) == 0 để kiểm tra thông điệp có được gửi đến topic device/control hay không. Chỉ những thông điệp thuộc topic này mới được xử lý theo logic điều khiển đèn LED; các topic khác sẽ bị bỏ qua.
- **Phân tích dữ liệu JSON:** Khởi tạo đối tượng StaticJsonDocument<256> doc để lưu trữ dữ liệu JSON và sử dụng hàm deserializeJson(doc, msg) để phân tích nội dung thông điệp. Nếu dữ liệu JSON không hợp lệ, chương trình hiển thị thông báo lỗi thông qua Serial.printf() và kết thúc xử lý bằng lệnh return, tránh tiếp tục điều khiển thiết bị với dữ liệu không đúng định dạng.
- **Trích xuất thông tin điều khiển:** Sử dụng doc["deviceId"].as<long>() để lấy mã định danh thiết bị và doc["action"].as<const char*>() để lấy lệnh điều khiển. Biến deviceId xác định đèn LED cần điều khiển, trong khi action chứa trạng thái yêu cầu như "ON" hoặc "OFF". Nếu không tìm thấy trường action hợp lệ, chương trình kết thúc xử lý để tránh thao tác với dữ liệu không đầy đủ.
- **Xác định trạng thái bật/tắt:** Sử dụng biểu thức strcmp(action, "ON") == 0 để xác định lệnh điều khiển. Nếu giá trị action là "ON", biến turnOn được đặt thành true, yêu cầu bật đèn; ngược lại, biến này nhận giá trị false, tương ứng với yêu cầu tắt đèn. Do đó, trong đoạn code hiện tại, các giá trị khác "ON" cũng được hiểu là lệnh tắt đèn.
- **Điều khiển đèn LED 1:** Nếu deviceId == 1, chương trình cập nhật biến led1State theo giá trị turnOn và sử dụng digitalWrite(LED1_PIN, turnOn ? HIGH : LOW) để bật hoặc tắt đèn LED 1 thông qua GPIO 18. Sau đó, hàm

preferences.putBool("led1", turnOn) lưu trạng thái mới vào bộ nhớ Flash NVS, giúp thiết bị có thể khôi phục trạng thái đèn sau khi khởi động lại. Hàm Serial.printf() hiển thị trạng thái điều khiển trên Serial Monitor.

- **Điều khiển đèn LED 2:** Nếu deviceId == 2, chương trình thực hiện quy trình tương tự đối với LED 2 thông qua GPIO 19. Biến led2State được cập nhật, chân GPIO được thiết lập ở mức HIGH hoặc LOW, và trạng thái mới được lưu bằng preferences.putBool("led2", turnOn). Thông báo về trạng thái đèn cũng được hiển thị để hỗ trợ giám sát.
- **Kiểm tra trạng thái thực tế của đèn:** Sau khi điều khiển mỗi đèn, chương trình sử dụng delay(50) để tạo khoảng trễ 50 mili giây, sau đó gọi digitalRead() để đọc mức logic hiện tại tại chân GPIO tương ứng. Kết quả được lưu vào biến actual, rồi truyền cùng mã thiết bị và tên đèn vào hàm publishDeviceStatus() để gửi trạng thái thiết bị lên hệ thống thông qua MQTT. Lưu ý, việc đọc lại GPIO chỉ xác nhận mức logic tại chân điều khiển, không trực tiếp xác nhận đèn LED thực sự phát sáng.
- **Xử lý mã thiết bị không hợp lệ:** Nếu deviceId không bằng 1 hoặc 2, chương trình hiển thị thông báo Unknown deviceId kèm mã thiết bị nhận được. Điều này giúp phát hiện các lệnh điều khiển gửi đến thiết bị không được hỗ trợ.

4.3.6. Gửi trạng thái thiết bị

```
282  void publishDeviceStatus(int deviceId, const char* deviceKey, bool isOn) {
283    StaticJsonDocument<128> doc;
284    doc["deviceId"] = deviceId;
285    doc["device"]   = deviceKey;
286    doc["status"]   = isOn ? "ON" : "OFF";
287
288    char payload[128];
289    serializeJson(doc, payload);
290
291    // Log xác nhận GPIO thực tế (debug phần cứng)
292    int pin = (deviceId == 1) ? LED1_PIN : LED2_PIN;
293    Serial.printf("[GPIO] Pin %d = %d (expected %d)\n", pin, digitalRead(pin), isOn ? HIGH : LOW);
294
295    bool ok = mqttClient.publish(TOPIC_DEVICE_STATUS, payload);
296    Serial.printf("[MQTT] Status published: %s (ok=%d)\n", payload, ok);
297  }
```

Hình 4.21. Hàm publishDeviceStatus()

- Hàm publishDeviceStatus():

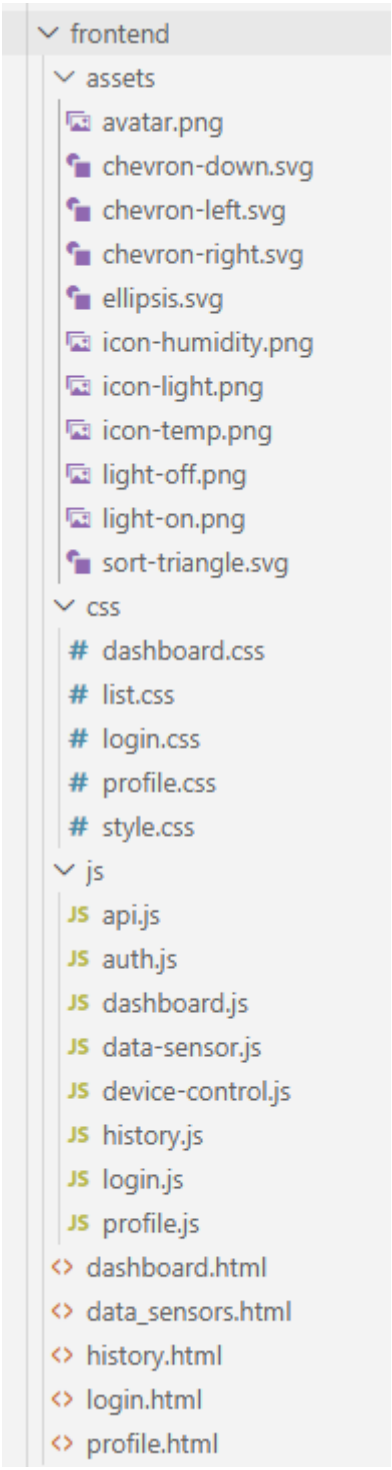
- **Tạo dữ liệu trạng thái thiết bị:** Sử dụng thư viện ArduinoJson để tạo đối tượng JSON chứa mã thiết bị (deviceId), tên thiết bị (deviceKey) và trạng thái hoạt động (status).
- **Xác định trạng thái thiết bị:** Chuyển đổi giá trị Boolean isOn thành chuỗi "ON" hoặc "OFF" để biểu thị trạng thái bật hoặc tắt của thiết bị.

- **Đóng gói dữ liệu JSON:** Chuyển đổi đối tượng JSON thành chuỗi ký tự để chuẩn bị gửi qua giao thức MQTT.
- **Kiểm tra trạng thái GPIO:** Xác định chân GPIO tương ứng với thiết bị và đọc trạng thái thực tế bằng digitalRead(), sau đó hiển thị kết quả trên Serial Monitor để hỗ trợ kiểm tra phần cứng.
- **Gửi trạng thái qua MQTT:** Gửi dữ liệu JSON lên topic device/status để ứng dụng giám sát nhận được thông tin trạng thái của thiết bị.
- **Thông báo kết quả gửi:** Hiển thị nội dung JSON và kết quả gửi dữ liệu trên Serial Monitor, giúp kiểm tra quá trình truyền trạng thái thiết bị.

4.4. Xây dựng front-end

Phần Frontend của hệ thống được xây dựng bằng ba công nghệ web cốt lõi là HTML5, CSS3 và JavaScript. Trong đó, HTML5 đảm nhận vai trò định hình cấu trúc các trang giao diện; CSS3 chịu trách nhiệm tạo kiểu dáng, bố cục và đảm bảo tính responsive trên nhiều kích thước màn hình; JavaScript xử lý toàn bộ logic tương tác phía client, bao gồm gọi API, cập nhật dữ liệu tự động và thao tác DOM để hiển thị thông tin lên giao diện.

4.4.1. Cấu trúc thư mục



Hình 4.22. Cấu trúc thư mục front-end

4.4.2. Thư mục assets

Thư mục assets chứa toàn bộ tài nguyên tĩnh, bao gồm hình ảnh và biểu tượng (icon), được sử dụng chung cho giao diện người dùng của ứng dụng.

4.4.3. Thư mục js

Thư mục js chứa toàn bộ mã JavaScript chịu trách nhiệm xử lý logic nghiệp vụ, giao tiếp với API và tương tác với người dùng trên từng trang của ứng dụng.

- File api.js: File api.js cung cấp các hàm dùng chung để giao tiếp với backend, bao gồm:

- Định nghĩa hằng số BASE_URL trỏ đến địa chỉ backend.
- Cung cấp hàm apiFetch dùng chung để gửi yêu cầu đến API, tự động đính kèm JWT token, phân tích dữ liệu JSON, xử lý lỗi và chuyển hướng về login.html khi nhận được mã lỗi 401.
- Cung cấp các hàm xác thực người dùng: apiLogin, apiRegister, apiChangePassword và apiGetMe.
- Cung cấp các hàm lấy dữ liệu cảm biến: fetchLatestSensors, fetchChartData và fetchSensorList.
- Cung cấp các hàm liên quan đến thiết bị: fetchDeviceStatus, postDeviceControl, fetchHistoryStatus và fetchDeviceHistory.
- Cung cấp các hàm tiện ích: buildQueryString dùng để tạo chuỗi truy vấn, formatDateTime dùng để định dạng thời gian hiển thị và convertTimeInput dùng để chuyển đổi dữ liệu thời gian tìm kiếm thành mẫu LIKE phù hợp với backend.

- File auth.js: File auth.js đảm nhiệm việc xác thực quyền truy cập và quản lý thông tin người dùng trên giao diện, bao gồm:

- Kiểm tra trạng thái đăng nhập và bảo vệ các trang nội bộ. Nếu người dùng chưa đăng nhập, hệ thống sẽ chặn truy cập. Nếu người dùng đã đăng nhập nhưng truy cập login.html, hệ thống sẽ chuyển hướng về dashboard.html.
- Tạo widget thông tin người dùng ở cuối sidebar, bao gồm avatar hiển thị chữ cái đầu, tên người dùng và mã sinh viên, kèm theo menu popup với các chức năng Đổi mật khẩu và Đăng xuất.
- Xử lý đăng xuất bằng cách xóa token và thông tin người dùng khỏi localStorage, sau đó chuyển hướng về login.html.
- Tạo và điều khiển cửa sổ modal Đổi mật khẩu, kiểm tra độ dài mật khẩu, xác nhận mật khẩu trùng khớp và bảo đảm mật khẩu mới không trùng với mật khẩu cũ. Sau đó, hệ thống gọi apiChangePassword và hiển thị thông báo thành công hoặc thất bại.

- Cung cấp hàm escapeHtml để mã hóa các ký tự HTML đặc biệt, góp phần ngăn chặn các cuộc tấn công XSS khi hiển thị dữ liệu người dùng.

- File dashboard.js: File dashboard.js xử lý các chức năng chính của trang Dashboard, bao gồm:

- Khởi tạo và điều khiển các thành phần giao diện trên trang tổng quan.
- Tự động cập nhật dữ liệu cảm biến và biểu đồ sau mỗi 2 giây bằng cơ chế setTimeout-chain nhằm hạn chế tình trạng các yêu cầu bị chồng chéo.
- Cập nhật các giá trị nhiệt độ, độ ẩm và ánh sáng trên ba thẻ thông số.
- Khởi tạo và cập nhật biểu đồ đường bằng thư viện Chart.js, hiển thị 15 giá trị gần nhất. Biểu đồ sử dụng hai trục tung để biểu diễn các nhóm dữ liệu nhiệt độ, độ ẩm và ánh sáng.
- Lấy trạng thái thực tế của hai đèn LED từ API.
- Gắn sự kiện cho các công tắc bật/tắt đèn LED và cập nhật biểu tượng tương ứng với trạng thái thiết bị.
- Hiển thị banner cảnh báo khi mất kết nối với máy chủ sau nhiều lần gửi yêu cầu thất bại.

- File device-control.js: File device-control.js xử lý toàn bộ quy trình điều khiển thiết bị LED từ khi người dùng thao tác trên công tắc đến khi nhận được kết quả cuối cùng, bao gồm:

- Gửi yêu cầu POST để điều khiển thiết bị.
- Thực hiện polling trạng thái lịch sử để chờ kết quả điều khiển, bao gồm ON, OFF hoặc FAILED.
- Quản lý trạng thái chờ bằng spinner và hiển thị thông báo thành công hoặc thất bại.
- Tự động khôi phục trạng thái công tắc về trạng thái trước đó nếu thao tác thất bại hoặc vượt quá thời gian chờ.
- Sử dụng Map để quản lý các interval và timeout, hạn chế rò rỉ bộ nhớ khi người dùng thao tác liên tục.

- File data-sensor.js: File data-sensor.js xử lý logic của trang Data Sensor, bao gồm:

- Quản lý các bộ lọc tìm kiếm theo loại cảm biến, giá trị và thời gian.

- Hỗ trợ sắp xếp dữ liệu và lựa chọn số lượng bản ghi hiển thị trên mỗi trang.
- Tự động làm mới dữ liệu sau mỗi 2 giây và tạm dừng quá trình làm mới khi người dùng đang nhập nội dung tìm kiếm.
- Hiển thị bảng dữ liệu cảm biến bằng DocumentFragment nhằm tối ưu hiệu năng. Bảng chỉ được render lại khi danh sách ID bản ghi thay đổi.
- Hiển thị thông tin phân trang và tạo các nút điều hướng giữa các trang dữ liệu.

- File history.js: File history.js xử lý các chức năng trên trang History, bao gồm:

- Quản lý bộ lọc tìm kiếm theo thời gian, tên thiết bị, hành động và trạng thái.
- Hỗ trợ sắp xếp dữ liệu và lựa chọn số lượng bản ghi trên mỗi trang.
- Tự động làm mới dữ liệu lịch sử sau mỗi 2 giây.
- Hiển thị bảng lịch sử điều khiển thiết bị.
- Cập nhật thông tin phân trang và các nút điều hướng giữa các trang.

- File login.js: File login.js xử lý các chức năng đăng nhập và đăng ký tài khoản, bao gồm:

- Cho phép chuyển đổi giữa hai tab Đăng nhập và Đăng ký, đồng thời xóa thông báo lỗi khi chuyển tab.
- Xử lý việc gửi biểu mẫu đăng nhập, kiểm tra dữ liệu đầu vào không được để trống, gọi apiLogin, lưu token và thông tin người dùng vào localStorage, sau đó chuyển hướng về dashboard.html.
- Xử lý việc gửi biểu mẫu đăng ký, kiểm tra độ dài mật khẩu và tính khớp nhau giữa mật khẩu với mật khẩu xác nhận, gọi apiRegister, lưu token và thông tin người dùng vào localStorage, sau đó chuyển hướng về dashboard.html.
- Hiển thị thông báo lỗi tương ứng khi API trả về kết quả thất bại.

- File profile.js: File profile.js chịu trách nhiệm tải và hiển thị thông tin sinh viên trên trang Profile, bao gồm:

- Lấy thông tin sinh viên từ API /auth/me.
- Cập nhật các thông tin cá nhân gồm họ tên, mã sinh viên và lớp.
- Cập nhật các liên kết đến GitHub, Figma, Postman và báo cáo dự án.

- Cập nhật nội dung văn bản thay thế (alt text) cho ảnh đại diện.
- Xử lý các liên kết chưa được cập nhật bằng cách hiển thị thông báo "Chưa cập nhật", làm mờ liên kết và vô hiệu hóa thao tác nhấp chuột.

4.4.4. Các file HTML gốc

Các file HTML gốc là những trang giao diện chính của ứng dụng. Mỗi file tương ứng với một chức năng riêng, đồng thời liên kết với các file CSS và JavaScript cần thiết để xây dựng giao diện và xử lý nghiệp vụ.

- File login.html: Đây là trang đăng nhập và đăng ký tài khoản, bao gồm:

- Hiển thị thẻ giao diện chính ở giữa màn hình.
- Cung cấp hai tab để chuyển đổi giữa biểu mẫu Đăng nhập và Đăng ký.
- Biểu mẫu đăng nhập gồm các trường tên đăng nhập và mật khẩu.
- Biểu mẫu đăng ký gồm các trường tên đăng nhập, mật khẩu, xác nhận mật khẩu, họ tên, mã sinh viên và lớp.
- Cung cấp khu vực hiển thị thông báo lỗi riêng cho từng biểu mẫu.
- Tích hợp các file JavaScript api.js và login.js để xử lý xác thực và đăng ký tài khoản.

- File dashboard.html: Đây là trang tổng quan của ứng dụng, cung cấp các chức năng theo dõi cảm biến và điều khiển thiết bị, bao gồm:

- Hiển thị ba thẻ thông số cảm biến: nhiệt độ, độ ẩm và ánh sáng.
- Hiển thị biểu đồ đường biểu diễn 15 giá trị gần nhất.
- Cung cấp khu vực điều khiển hai đèn LED.
- Hiển thị banner cảnh báo khi mất kết nối với máy chủ.
- Tích hợp các file auth.js, Chart.js, api.js, device-control.js và dashboard.js để xác thực người dùng, hiển thị biểu đồ, giao tiếp với API và điều khiển thiết bị.

- File data_sensors.html: Đây là trang hiển thị danh sách dữ liệu cảm biến dưới dạng bảng, bao gồm:

- Thanh công cụ tìm kiếm với danh sách thả xuống để lựa chọn loại tìm kiếm.
- Ô nhập từ khóa tìm kiếm.
- Nút sắp xếp dữ liệu.

- Danh sách lựa chọn số lượng bản ghi hiển thị trên mỗi trang.
- Tích hợp các file auth.js, api.js và data-sensor.js để xác thực người dùng, lấy dữ liệu và xử lý các chức năng tìm kiếm, sắp xếp, phân trang.

- File history.html: Đây là trang hiển thị lịch sử điều khiển thiết bị, bao gồm:

- Ô tìm kiếm theo thời gian.
- Ba danh sách thả xuống để lọc theo tên thiết bị, hành động và trạng thái.
- Nút sắp xếp dữ liệu.
- Danh sách lựa chọn số lượng bản ghi hiển thị trên mỗi trang.
- Tích hợp các file auth.js, api.js và history.js để xác thực người dùng, lấy dữ liệu lịch sử và xử lý các chức năng lọc, sắp xếp, phân trang.

- File profile.html: Đây là trang hiển thị thông tin sinh viên thực hiện dự án, bao gồm:

- Ảnh đại diện của sinh viên.
- Các thông tin cá nhân gồm họ tên, mã sinh viên và lớp.
- Các liên kết đến GitHub, Figma, Postman và báo cáo dự án.
- Tích hợp các file auth.js, api.js và profile.js để xác thực người dùng, lấy dữ liệu từ API và cập nhật nội dung trang.

4.4.5. Thư mục css

Thư mục css chứa các file định nghĩa giao diện người dùng của ứng dụng. Các file CSS được tổ chức theo từng trang hoặc nhóm chức năng nhằm tăng khả năng tái sử dụng, thuận tiện cho việc bảo trì và chỉnh sửa giao diện.

- File style.css: File style.css là file CSS dùng chung cho toàn bộ ứng dụng, bao gồm:

- Sử dụng các biến CSS trong :root để quản lý màu sắc, kích thước và kiểu chữ thống nhất.
- Định nghĩa bố cục tổng thể với sidebar cố định và vùng nội dung chính linh hoạt.
- Thiết lập giao diện sidebar, logo, thanh điều hướng, trạng thái mục đang được chọn, nút mở menu trên thiết bị di động và lớp nền che.
- Tạo kiểu cho widget người dùng ở cuối sidebar.

- Định nghĩa giao diện modal Đổi mật khẩu, bao gồm lớp phủ, thẻ nội dung, biểu mẫu, nút gửi và các thông báo thành công hoặc lỗi.
- Hỗ trợ thiết kế responsive thông qua các breakpoint 1100px, 900px và 768px. Các breakpoint này giúp điều chỉnh menu trên thiết bị di động và chuyển bố cục nhiều cột thành một cột khi cần thiết.

- File dashboard.css: File dashboard.css định nghĩa giao diện riêng cho trang Dashboard, bao gồm:

- Khai báo các biến màu đại diện cho nhiệt độ, độ ẩm, ánh sáng, trạng thái bật của công tắc và trạng thái tắt của thanh công tắc.
- Thiết lập lưới hiển thị các thẻ thông số thông qua các lớp .stat-grid và .stat-card, với chiều cao linh hoạt theo kích thước vùng hiển thị.
- Tạo bố cục hai cột cho khu vực biểu đồ và khu vực điều khiển thiết bị thông qua các lớp .dashboard-bottom, .chart-card và .control-card.
- Định dạng chú giải biểu đồ và vùng canvas có khả năng co giãn.
- Thiết lập giao diện các thành phần điều khiển thiết bị, bao gồm device-row, lamp-icon, spinner và thông báo trạng thái.
- Tạo kiểu cho công tắc bật/tắt tùy chỉnh và banner cảnh báo mất kết nối.
- Hỗ trợ responsive với breakpoint 1100px để chuyển bố cục về một cột và breakpoint 900px để chuyển các thẻ thông số về một cột.

- File list.css: File list.css là file CSS dùng chung cho các trang danh sách Data Sensor và History, bao gồm:

- Thiết lập giao diện thanh công cụ thống nhất, trong đó ô tìm kiếm có khả năng co giãn, còn danh sách thả xuống và nút sắp xếp có kích thước cố định.
- Định dạng các trường nhập liệu và danh sách lựa chọn có biểu tượng chevron tùy chỉnh.
- Thiết lập giao diện nút sắp xếp, trong đó biểu tượng tam giác thay đổi hướng theo trạng thái được chọn.
- Định dạng bảng dữ liệu bằng thuộc tính table-layout: fixed, cố định phần tiêu đề bảng khi cuộn, điều chỉnh chiều cao hàng theo kích thước vùng hiển thị và cắt bớt nội dung dài bằng dấu ba chấm.

- Thiết lập phần chân bảng, bao gồm thông tin số lượng bản ghi và các nút phân trang. Trang hiện tại được làm nổi bật bằng đường viền và hiệu ứng đổ bóng.
- Bổ sung các quy tắc CSS riêng cho thanh công cụ của trang History, cho phép các thành phần xuống dòng và sử dụng tỷ lệ co giãn phù hợp.
- Hỗ trợ responsive với breakpoint 900px, cho phép thanh công cụ xuống dòng, loại bỏ chiều cao cố định của bảng và chuyển phần chân bảng sang bố cục dọc.

- File login.css: File login.css định nghĩa giao diện riêng cho trang đăng nhập và đăng ký, bao gồm:

- Thiết lập nền chuyển màu (gradient).
- Căn giữa thẻ giao diện chính trên màn hình, kết hợp bo góc và hiệu ứng đổ bóng.
- Định dạng phần tiêu đề, logo và dòng mô tả.
- Thiết lập giao diện hai tab Đăng nhập và Đăng ký. Tab đang hoạt động được đánh dấu bằng đường gạch chân mang màu sắc chủ đạo của ứng dụng.
- Định dạng phần chân trang và liên kết chuyển đổi giữa hai biểu mẫu.

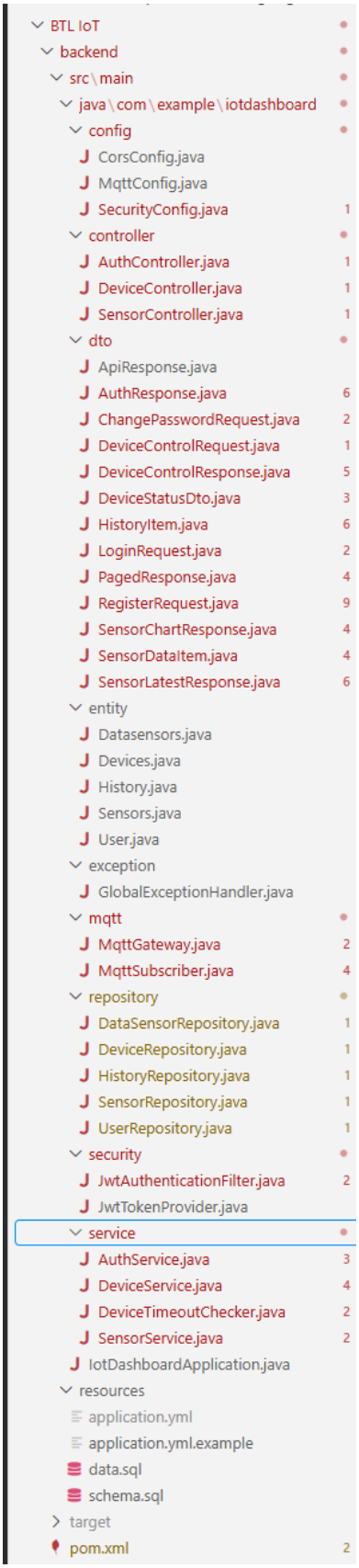
- File profile.css: File profile.css định nghĩa giao diện riêng cho trang Profile, bao gồm:

- Khai báo các biến màu nền dùng cho những hàng thông tin cá nhân.
- Thiết lập vùng chứa profile-wrap với nội dung được căn giữa và hỗ trợ cuộn theo chiều dọc.
- Định dạng thẻ thông tin profile-card với chiều rộng tối đa và chiều cao phù hợp với vùng hiển thị.
- Tạo kiểu cho ảnh đại diện hình tròn và tên sinh viên.
- Định dạng các hàng thông tin thông qua lớp profile-row, sử dụng màu nền riêng và góc bo tròn.
- Hỗ trợ responsive với breakpoint 900px nhằm giảm khoảng đệm và kích thước chữ của các hàng thông tin trên màn hình nhỏ.

4.5. Xây dựng back-end

Phần Backend của hệ thống được xây dựng dựa trên ngôn ngữ Java kết hợp với framework Spring Boot. Trong đó, Java đảm nhận vai trò xử lý toàn bộ logic nghiệp vụ cốt lõi; Spring Boot cung cấp nền tảng để phát triển nhanh các ứng dụng web thông qua hệ sinh thái module phong phú (Spring Web để xây dựng RESTful API, Spring Data JPA để tương tác với cơ sở dữ liệu quan hệ, Spring Security để xác thực và phân quyền). Ngoài ra, hệ thống còn sử dụng thư viện Eclipse Paho để giao tiếp với thiết bị qua giao thức MQTT và MySQL làm hệ quản trị cơ sở dữ liệu lưu trữ thông tin.

4.5.1. Cấu trúc thư mục



Hình 4.23. Cấu trúc thư mục back-end

4.5.2. Thư mục config

Thư mục này chứa các lớp cấu hình cho ứng dụng Spring Boot, bao gồm cấu hình CORS, MQTT và bảo mật.

- CorsConfig.java: Cấu hình CORS (Cross-Origin Resource Sharing) cho phép Frontend từ bất kỳ origin nào có thể gọi tới Backend API. Cho phép các phương thức GET, POST, PUT, DELETE, OPTIONS và mọi header. Cấu hình được áp dụng cho tất cả endpoint bắt đầu bằng /api/**.
- MqttConfig.java: Cấu hình và tạo bean MqttClient kết nối tới MQTT broker. Đọc các thông số từ application.yml (broker URL, client ID, username, password). Thiết lập các tùy chọn kết nối như clean session, tự động reconnect, timeout, keep-alive. Thực hiện kết nối ngay khi khởi tạo và ghi log trạng thái.
- SecurityConfig.java: Cấu hình Spring Security. Vô hiệu hóa CSRF, thiết lập chính sách session stateless (không lưu trạng thái), cấu hình CORS, phân quyền cho các endpoint (cho phép truy cập công khai /api/auth/login, /api/auth/register, /error và các request OPTIONS; các request khác yêu cầu xác thực). Cấu hình JwtAuthenticationFilter chạy trước UsernamePasswordAuthenticationFilter và cung cấp PasswordEncoder (BCrypt).

4.5.3. Thư mục controller

Thư mục này chứa các lớp Controller tiếp nhận HTTP request từ Frontend và trả về response tương ứng.

- AuthController.java: Xử lý các endpoint xác thực: POST /api/auth/login (đăng nhập), POST /api/auth/register (đăng ký), POST /api/auth/change-password (đổi mật khẩu), GET /api/auth/me (lấy thông tin người dùng hiện tại).
- DeviceController.java: Xử lý các endpoint liên quan đến thiết bị: GET /api/devices/status (lấy trạng thái tất cả thiết bị), POST /api/devices/{deviceId}/control (gửi lệnh điều khiển ON/OFF), GET /api/devices/history/{historyId}/status (polling trạng thái lệnh điều khiển), GET /api/devices/history (lấy lịch sử điều khiển có filter và phân trang).
- SensorController.java: Xử lý các endpoint liên quan đến cảm biến: GET /api/sensors/latest (lấy giá trị mới nhất của 3 cảm biến), GET /api/sensors/chart?limit=15 (lấy dữ liệu vẽ biểu đồ), GET /api/sensors (lấy danh sách dữ liệu cảm biến có filter, sort, phân trang).

4.5.4. Thư mục dto

Thư mục này chứa các lớp Data Transfer Object dùng để đóng gói dữ liệu trao đổi giữa các tầng và với Frontend.

- ApiResponse.java: Wrapper chung cho tất cả API response với cấu trúc { success, data, message }. Cung cấp static method ok(data) và error(message).
- AuthResponse.java: Response trả về sau khi đăng nhập/đăng ký thành công, bao gồm token, id, username, name, studentCode, classField.
- ChangePasswordRequest.java: Request body cho API đổi mật khẩu, chứa oldPassword và newPassword.
- DeviceControlRequest.java: Request body cho API điều khiển thiết bị, chứa trường action (ON/OFF).
- DeviceControlResponse.java: Response trả về sau khi gửi lệnh điều khiển, bao gồm historyId, deviceId, action, status (WAITING), time.
- DeviceStatusDto.java: Thông tin trạng thái của một thiết bị: deviceId, deviceName, status (ON/OFF/WAITING).
- HistoryItem.java: Một mục trong lịch sử điều khiển: id, deviceName, action, status, username, time.
- LoginRequest.java: Request body cho API đăng nhập, chứa username và password.
- PagedResponse.java: Response chung cho các API phân trang, chứa items, currentPage, totalPages, totalItems.
- RegisterRequest.java: Request body cho API đăng ký, chứa username, password, name, studentCode, classField và các link (Figma, Postman, GitHub, Report).
- SensorChartResponse.java: Response cho API biểu đồ, chứa labels, temperature, humidity, light.
- SensorDataItem.java: Một mục trong danh sách dữ liệu cảm biến: id, sensorType, value, time.
- SensorLatestResponse.java: Response cho API lấy giá trị mới nhất, chứa 3 đối tượng SensorValueDto (nhiệt độ, độ ẩm, ánh sáng), mỗi đối tượng có value, unit, time.

4.5.5. Thư mục entity

Thư mục này chứa các lớp JPA Entity ánh xạ tới các bảng trong cơ sở dữ liệu.

- Datasensors.java: Entity ánh xạ bảng datasensors, lưu giá trị đo của cảm biến. Quan hệ ManyToOne với Sensors. Gồm id, sensor, value, measuredAt.
- Devices.java: Entity ánh xạ bảng devices, lưu thông tin thiết bị. Gồm id, name, type, currentStatus (ON/OFF/WAITING).
- History.java: Entity ánh xạ bảng history, lưu lịch sử điều khiển. Quan hệ ManyToOne với User và Devices. Gồm id, user, device, action, status, executedAt.
- Sensors.java: Entity ánh xạ bảng sensors, lưu loại cảm biến. Gồm id, name (Nhiệt độ, Độ ẩm, Ánh sáng), unit (°C, %, lux).
- User.java: Entity ánh xạ bảng user, lưu thông tin người dùng/sinh viên. Gồm id, username, password (được ẩn khi serialize JSON), studentCode, name, classField, linkFigma, linkPostman, linkGithub, linkReport.

4.5.6. Thư mục exception

Thư mục này chứa lớp xử lý exception tập trung cho toàn bộ ứng dụng.

- GlobalExceptionHandler.java: Sử dụng @RestControllerAdvice để bắt và xử lý tất cả exception. Trả về response dạng { success: false, message: "..." }. Xử lý riêng IllegalArgumentException (400), NoResourceFoundException (404), và các exception khác (500).

4.5.7. Thư mục mqtt

Thư mục này chứa các lớp xử lý giao tiếp MQTT với ESP32.

- MqttGateway.java: MQTT publisher, chịu trách nhiệm publish lệnh điều khiển lên topic device/control. Payload dạng JSON chứa deviceId, device (led1/led2), action (ON/OFF). QoS 1.
- MqttSubscriber.java: MQTT subscriber, lắng nghe 2 topic: data/sensors (dữ liệu cảm biến) và device/status (trạng thái thiết bị). Sử dụng MqttCallbackExtended để tự động re-subscribe sau khi reconnect. Gọi SensorService.saveSensorData() và DeviceService.handleDeviceStatus() tương ứng.

4.5.8. Thư mục repository

Thư mục này chứa các interface Spring Data JPA Repository, cung cấp phương thức truy vấn cơ sở dữ liệu.

- DataSensorRepository.java: Repository cho entity Datasensors. Cung cấp phương thức findTopBySensorIdOrderByMeasuredAtDesc (lấy bản ghi mới nhất), findWithFilters (tìm kiếm có filter + phân trang, native query), findLatestBySensorId (lấy N bản ghi gần nhất).
- DeviceRepository.java: Repository cho entity Devices. Cung cấp findByCurrentStatus (tìm thiết bị theo trạng thái) và các thao tác CRUD cơ bản.
- HistoryRepository.java: Repository cho entity History. Cung cấp findTopByDeviceIdAndStatusOrderByIdDesc (tìm bản ghi theo trạng thái mới nhất), findByStatusAndExecutedAtBefore (tìm bản ghi quá hạn), existsByDeviceIdAndStatus (kiểm tra tồn tại), markTimedOutAsFailed (bulk update), findWithFilters (tìm kiếm lịch sử có filter + phân trang).
- SensorRepository.java: Repository cho entity Sensors. Kế thừa JpaRepository, cung cấp các thao tác CRUD cơ bản.
- UserRepository.java: Repository cho entity User. Cung cấp findByUsername (tìm người dùng theo username) và existsByUsername (kiểm tra tồn tại).

4.5.9. Thư mục security

Thư mục này chứa các lớp xử lý xác thực và phân quyền bằng JWT.

- JwtAuthenticationFilter.java: Filter chặn mọi request để kiểm tra JWT token trong header Authorization. Nếu token hợp lệ, lấy thông tin username và thiết lập Authentication vào SecurityContext.
- JwtTokenProvider.java: Cung cấp các phương thức tạo JWT token (generateToken), lấy username từ token (getUsernameFromToken), và kiểm tra tính hợp lệ của token (validateToken). Sử dụng thuật toán HMAC-SHA để ký token.

4.5.10. Thư mục service

Thư mục này chứa các lớp Service xử lý logic nghiệp vụ chính của ứng dụng.

- AuthService.java: Xử lý logic đăng nhập (kiểm tra username/password, tạo JWT), đăng ký (mã hóa mật khẩu bằng BCrypt, lưu user, tạo JWT), đổi mật

khẩu (kiểm tra mật khẩu cũ, mã hóa mật khẩu mới), và lấy thông tin người dùng hiện tại.

- DeviceService.java: Xử lý logic điều khiển thiết bị: tạo bản ghi History với status WAITING, cập nhật trạng thái device thành WAITING, publish lệnh MQTT, trả response ngay. Xử lý nhận trạng thái từ ESP32: cập nhật History (thành SUCCESS) và device status. Cung cấp API lấy trạng thái thiết bị, lấy lịch sử điều khiển có filter + phân trang, và polling trạng thái lệnh điều khiển.
- DeviceTimeoutChecker.java: Scheduled job chạy mỗi 2 giây, quét các bản ghi History đang ở trạng thái WAITING đã vượt quá thời gian timeout (mặc định 10 giây), tự động cập nhật thành FAILED, đồng thời khôi phục trạng thái thực tế của thiết bị (dựa trên action) để tránh kẹt trạng thái WAITING. Dọn dẹp các thiết bị bị kẹt WAITING mồ côi.
- SensorService.java: Cache danh sách cảm biến từ DB khi khởi động (sử dụng ApplicationReadyEvent). Xử lý dữ liệu cảm biến từ MQTT: lưu 3 bản ghi (nhiệt độ, độ ẩm, ánh sáng) vào bảng datasensors. Cung cấp API lấy giá trị mới nhất, dữ liệu biểu đồ, danh sách dữ liệu cảm biến có filter + phân trang.
- IotDashboardApplication.java: Lớp main khởi chạy ứng dụng Spring Boot. Kích hoạt scheduling với @EnableScheduling.

4.5.11. Thư mục resources

Thư mục này chứa các file cấu hình và tài nguyên của ứng dụng.

- application.yml: File cấu hình chính của Spring Boot, chứa thông tin kết nối database, MQTT broker, các topic, timeout thiết bị, JWT secret, v.v. **File này không được commit lên Git** để tránh lộ thông tin nhạy cảm.
- application.yml.example: File mẫu của application.yml, chứa đầy đủ cấu trúc cần thiết với các giá trị placeholder. Được commit lên Git để hướng dẫn người phát triển khác biết cần cấu hình những gì.
- data.sql: Script SQL khởi tạo dữ liệu mẫu cho các bảng (sensors, devices, user).
- schema.sql: Script SQL định nghĩa cấu trúc các bảng trong cơ sở dữ liệu.

4.5.12. File pom.xml

File cấu hình Maven, khai báo các dependency cho dự án Spring Boot bao gồm: Spring Web, Spring Data JPA, Spring Security, MySQL Connector, Lombok, Eclipse Paho MQTT Client, JJWT (thư viện JWT), Jackson, và các thư viện liên quan khác. Cấu hình build plugin cho Spring Boot.

4.6. API documentation

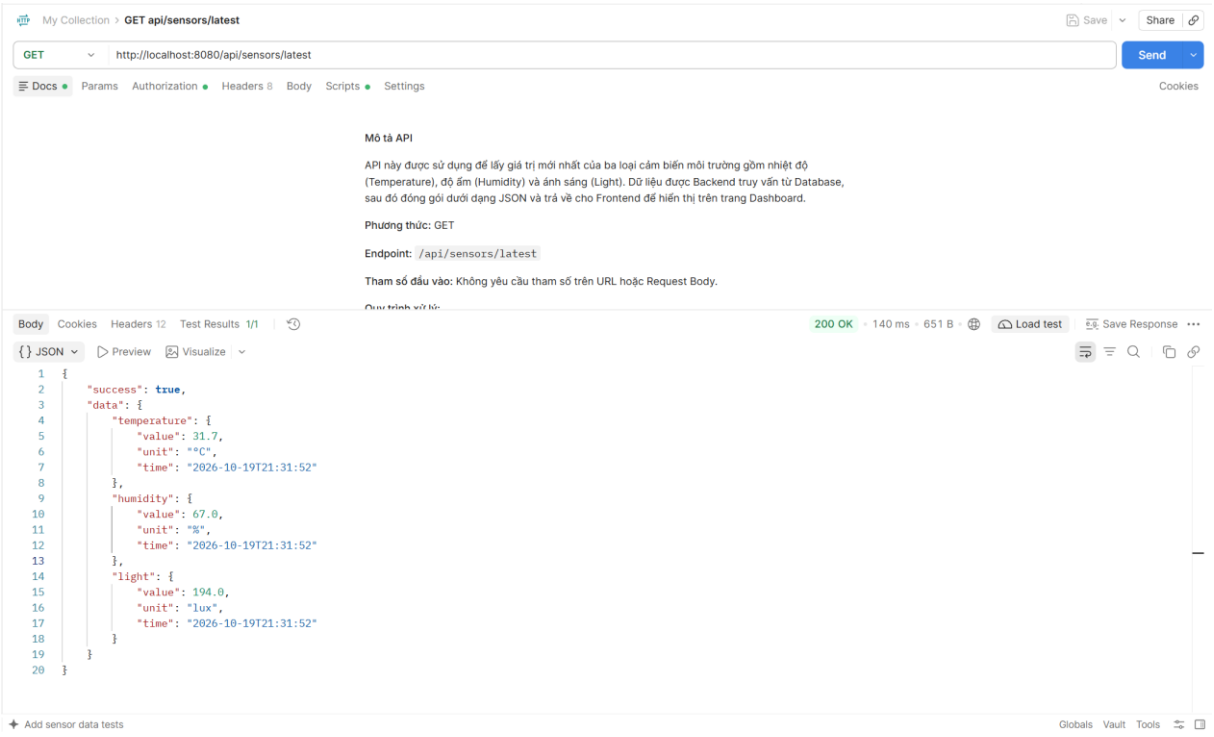
4.6.1. Sensor APIs

a. Lấy giá trị cảm biến mới nhất

GET /api/sensors/latest

Response


```
{
  "success": true,
  "data": {
    "temperature": { "value": 23, "unit": "°C", "time": "2026-08-18T12:54:01" },
    "humidity":    { "value": 60, "unit": "%",  "time": "2026-08-18T12:54:01" },
    "light":       { "value": 450, "unit": "lux", "time": "2026-08-18T12:54:01" }
  }
}
```



Hình 4.24. Test api/sensors/latest với postman

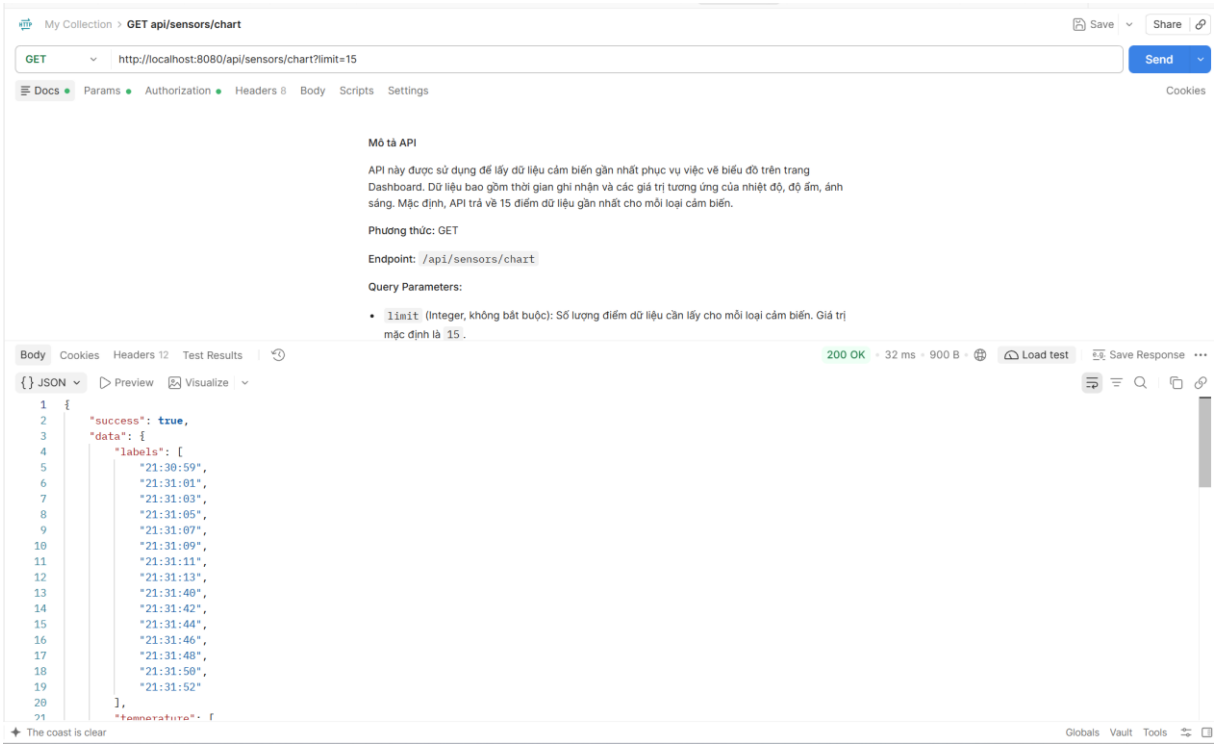
b. Lấy dữ liệu biểu đồ 15 giá trị gần nhất (Dashboard)

GET /api/sensors/chart?limit=15

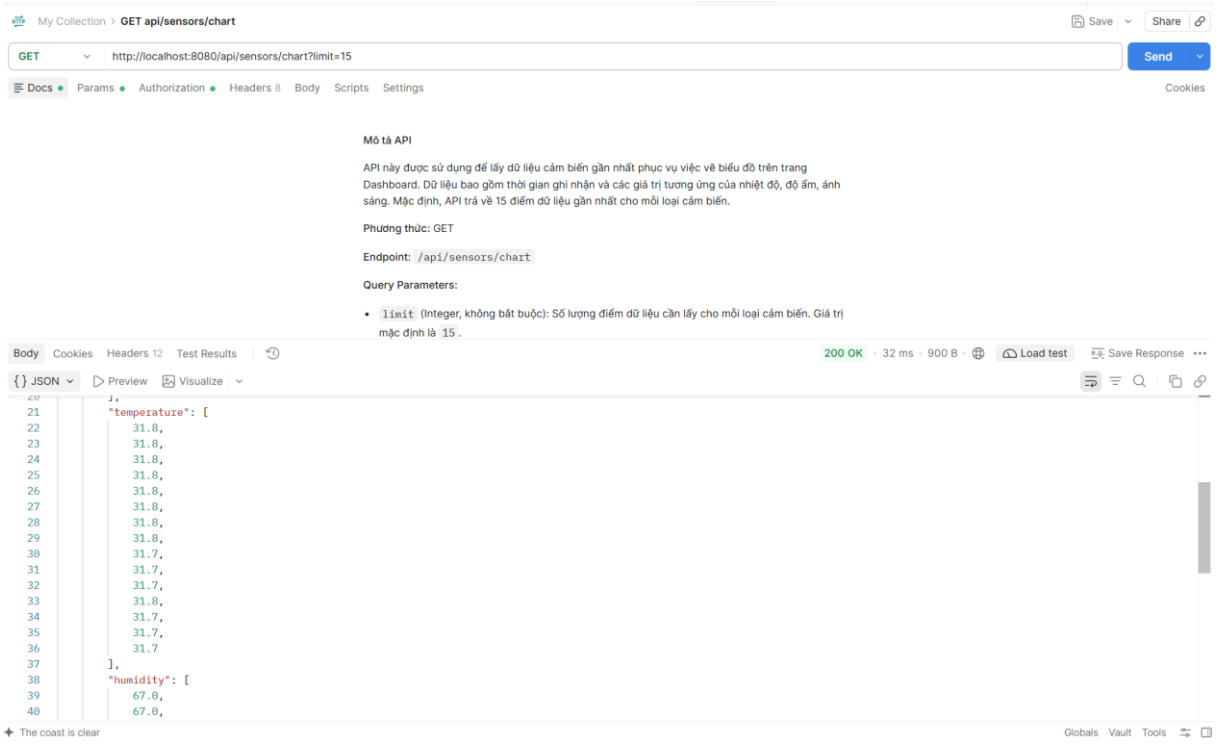
Param	Type	Bắt buộc	Mô tả
limit	int	không (default=15)	số điểm dữ liệu mỗi loại cảm biến

Response

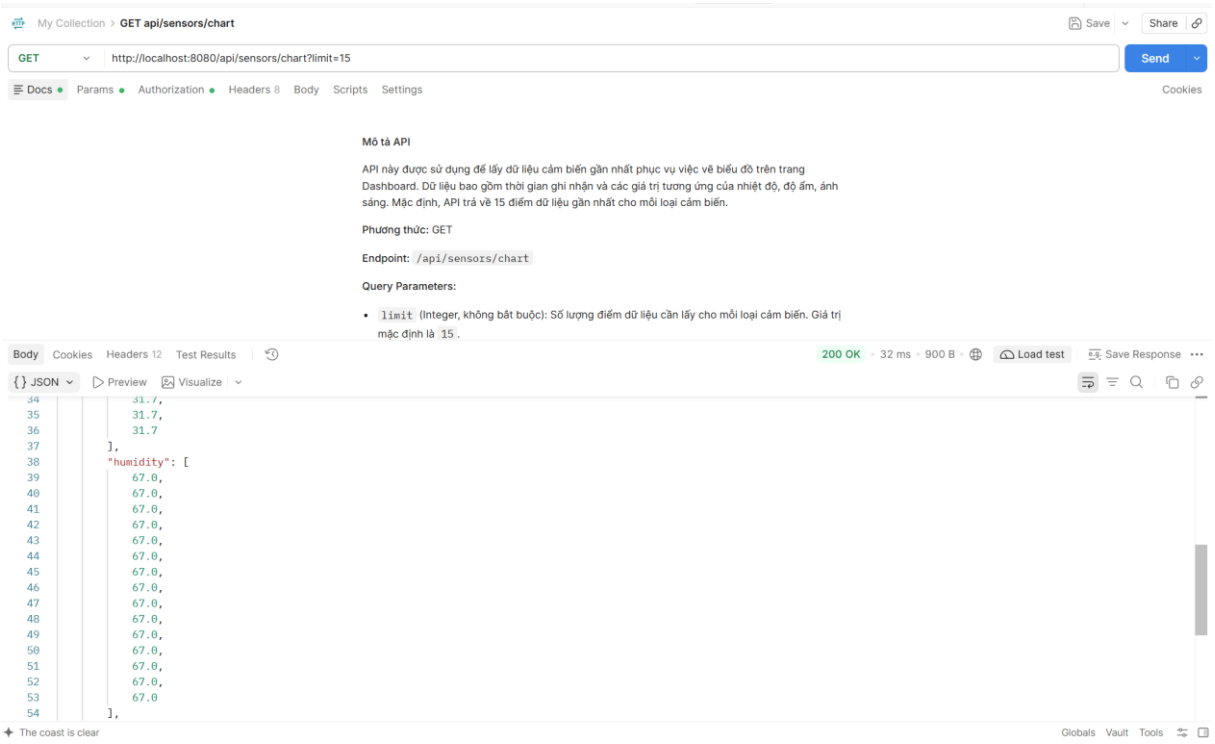
```
{
  "success": true,
  "data": {
    "labels": ["21:10:00","21:10:02","21:10:04", "..."],
    "temperature": [20, 30, 29, 24, "..."],
    "humidity": [60, 70, 70, 80, "..."],
    "light": [450, 480, 500, 500, "..."]
  }
}
```



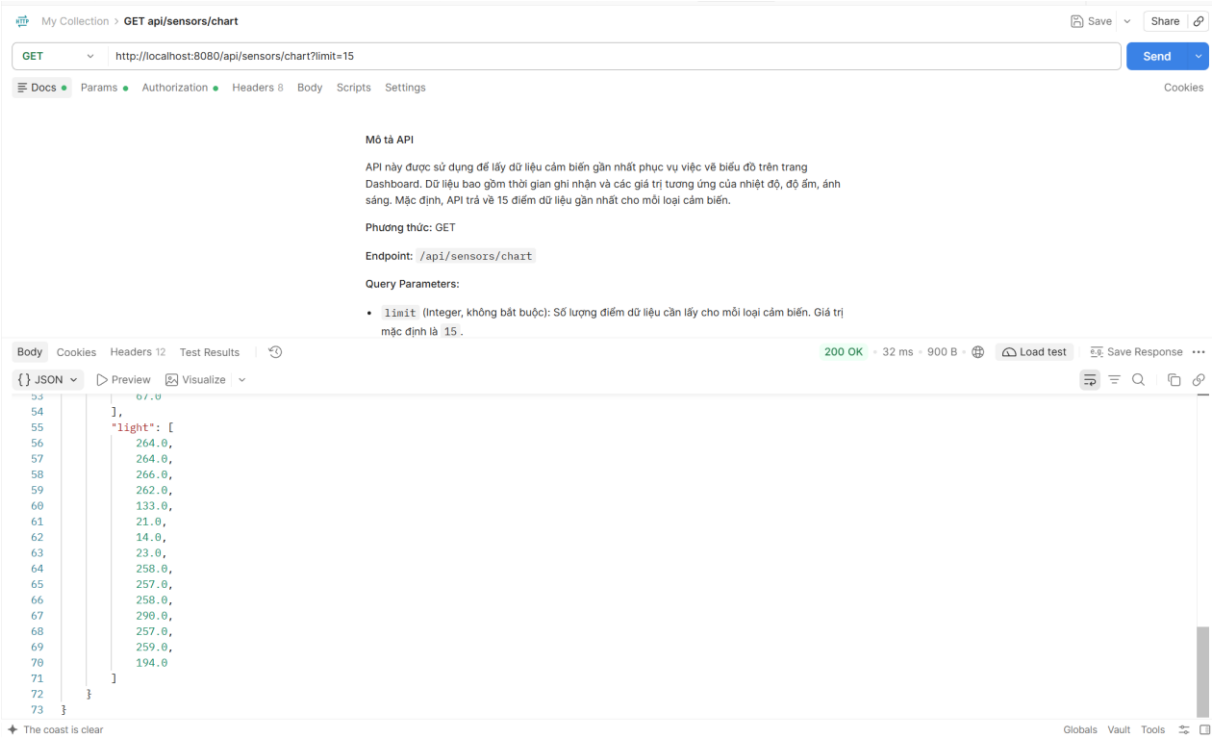
Hình 4.25. Test api/sensors/chart với postman



Hình 4.26. Test api/sensors/chart với postman (tiếp)



Hình 4.27. Test api/sensors/chart với postman (tiếp)



Hình 4.28. Test api/sensors/chart với postman (tiếp)

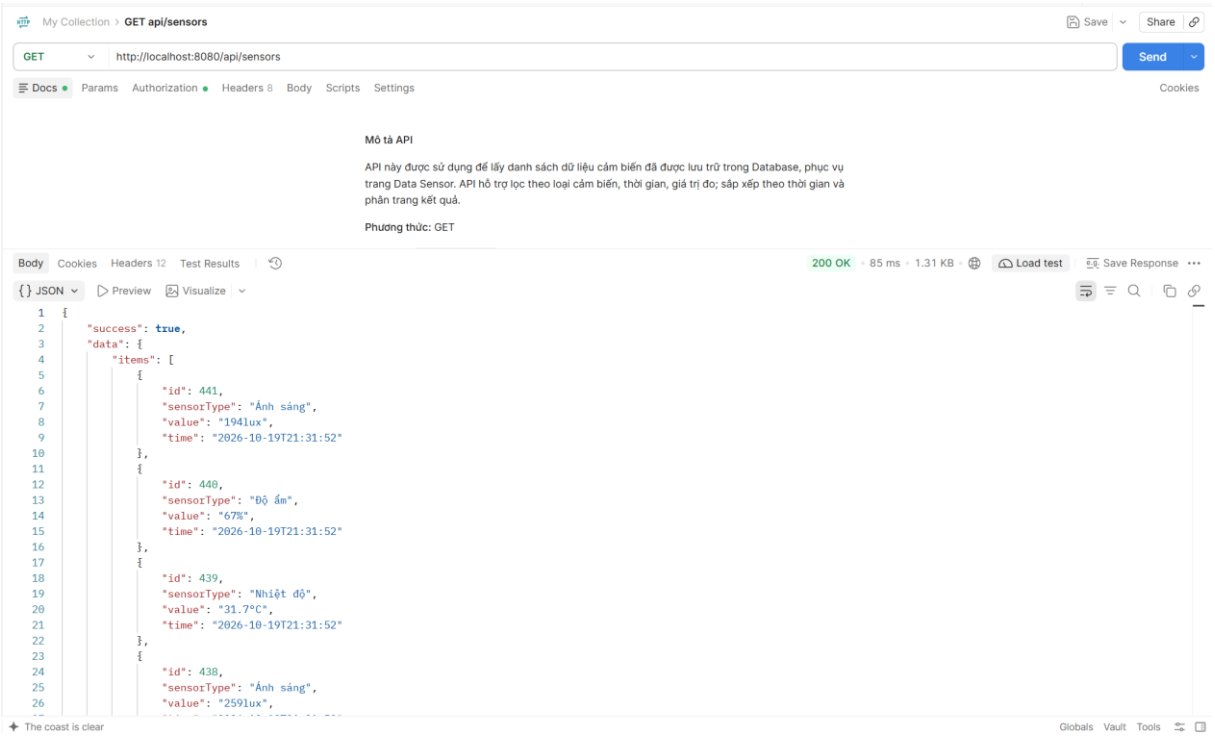
c. Lấy danh sách dữ liệu cảm biến (trang Data sensor - có filter/sort/paging)

GET /api/sensors

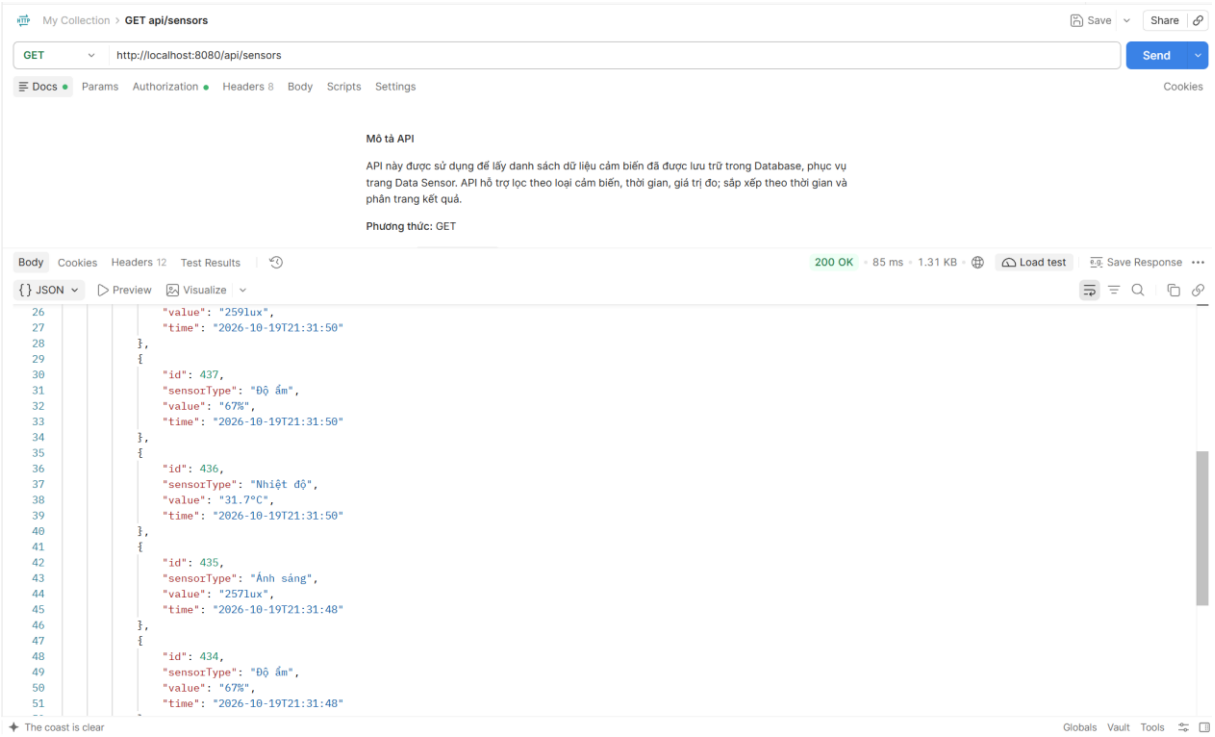
Param	Type	Bắt buộc	Mô tả
type	string	không	TEMPERATURE | HUMIDITY | LIGHT | (bỏ trống = tất cả)
time	string	không	lọc theo 1 mốc thời gian cụ thể
value	string	không	lọc theo giá trị (chứa/so khớp)
sort	string	không (default=desc)	asc | desc theo thời gian
page	int	không (default=1)	số trang
size	int	không (default=10)	số bản ghi/trang

Response

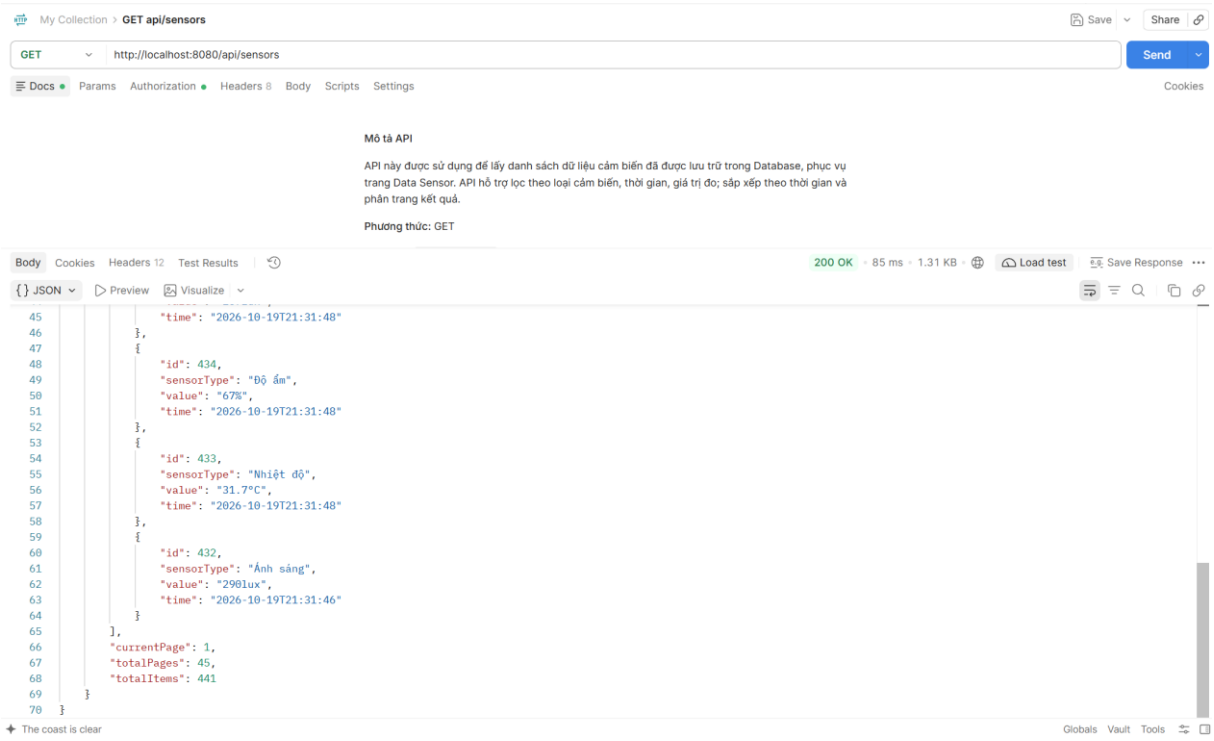

```
{
  "success": true,
  "data": {
    "items": [
      { "id": 20, "sensorType": "Ánh sáng", "value": "800 lux", "time": "2026-08-18T12:54:01" },
      { "id": 21, "sensorType": "Độ ẩm", "value": "60%", "time": "2026-08-18T12:54:01" }
    ],
    "currentPage": 2,
    "totalPages": 5,
    "totalItems": 50
  }
}
```



Hình 4.29. Test api/sensors với postman



Hình 4.30. Test api/sensors với postman (tiếp)



Hình 4.31. Test api/sensors với postman (tiếp)

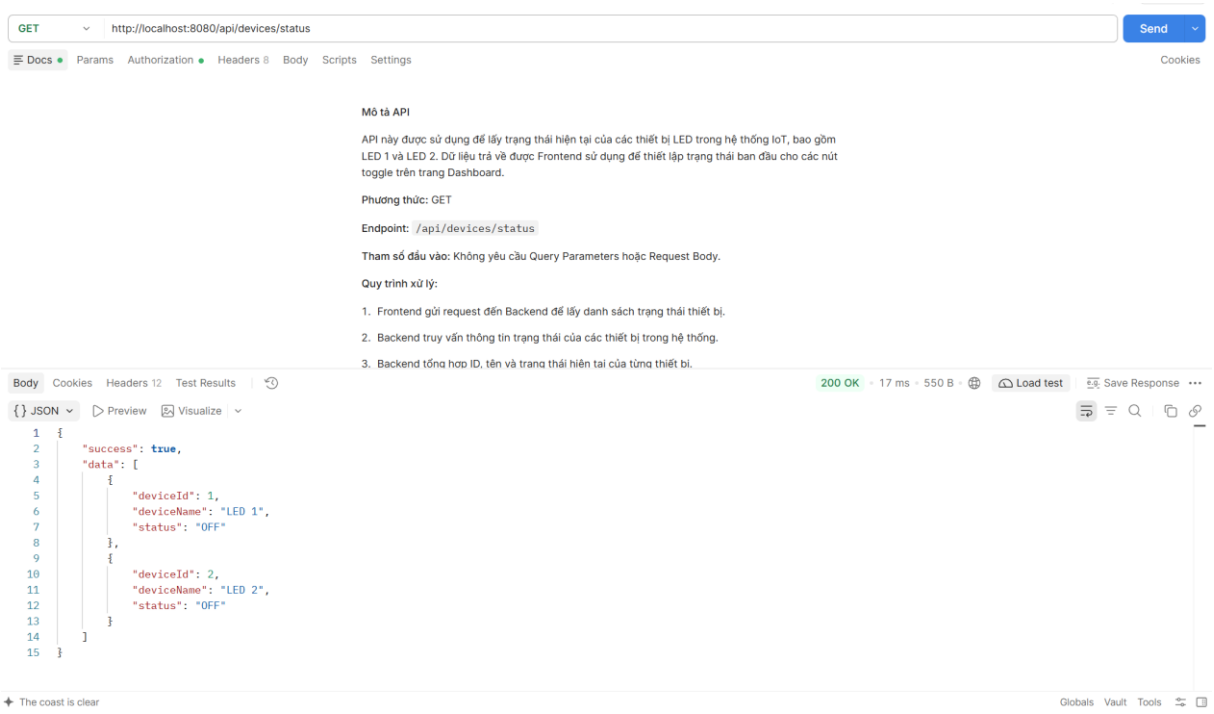
4.6.2. Device (LED) APIs

a. Lấy trạng thái hiện tại của tất cả thiết bị (Dashboard - toggle switch)

GET /api/devices/status

Response

```
{
  "success": true,
  "data": [
    { "deviceId": 1, "deviceName": "LED 1", "status": "OFF" },
    { "deviceId": 2, "deviceName": "LED 2", "status": "ON" }
  ]
}
```



Hình 4.32. Test api/devices/status với postman

b. Điều khiển bật/tắt thiết bị

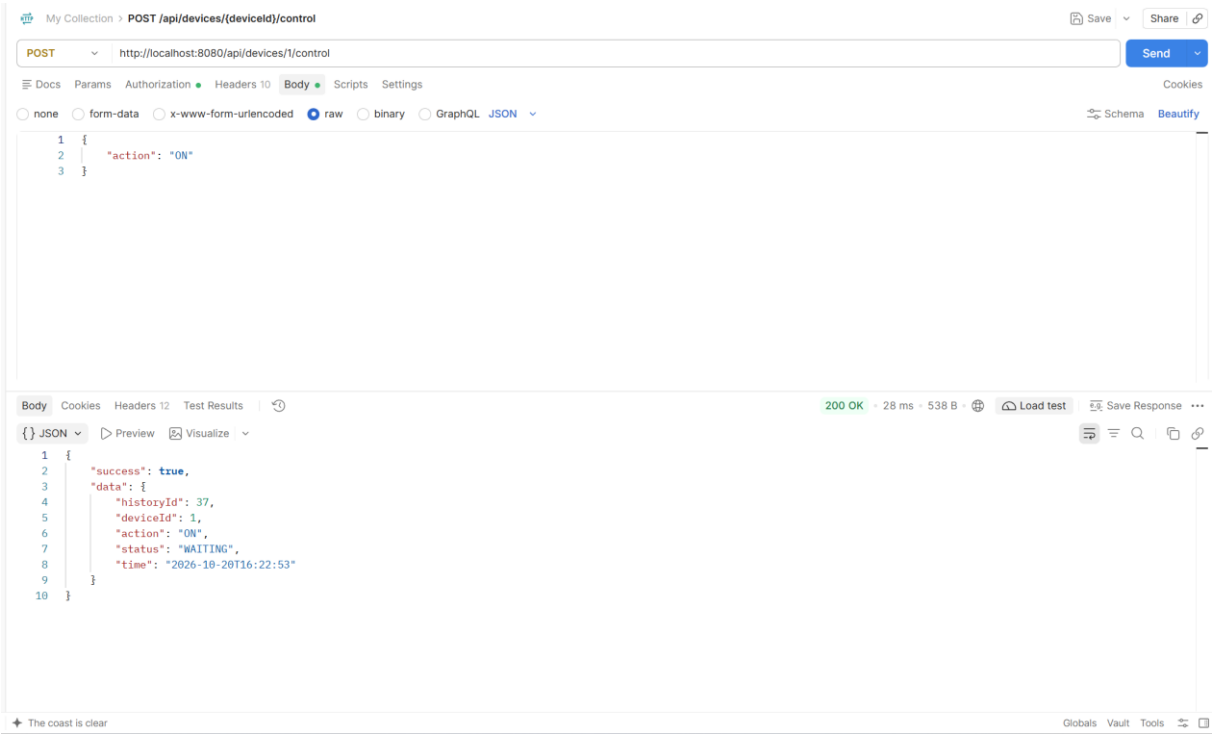
POST /api/devices/{deviceId}/control

Body

{"action": "ON"}

Response

{
"success": true,
"data": {
"historyId": 31,
"deviceId": 1,
"action": "ON",
"status": "WAITING",
"time": "2026-08-18T12:55:00"
}
}



Hình 4.33. Test api/devices/{deviceId}/control với postman

c. Kiểm tra trạng thái thực thi lệnh (FE polling sau khi control)

GET /api/devices/history/{historyId}/status

Response

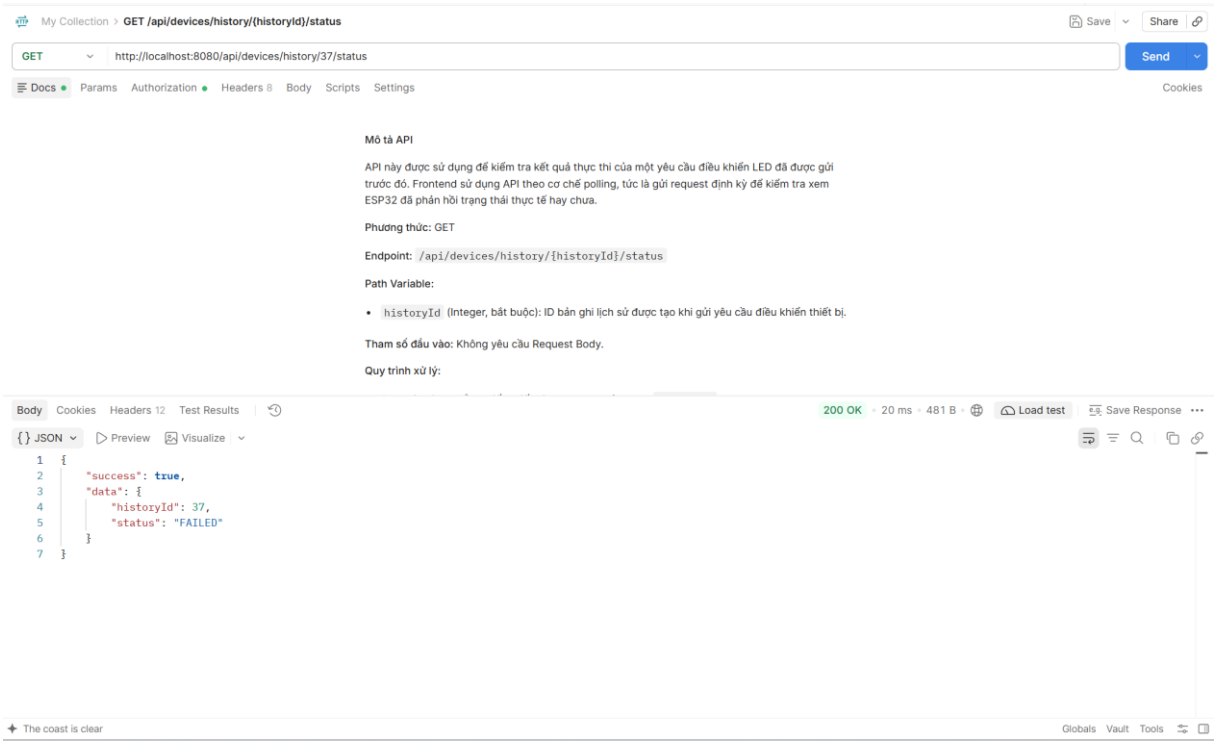
{

"success": true,

"data": { "historyId": 31, "status": "ON" }

}

status trả về: WAITING (chờ thiết bị phản hồi) | SUCCESS | FAILED (timeout không phản hồi)



Hình 4.34. Test api/devices/history/{historyId}/status với postman

4.6.3. History APIs

Lấy danh sách lịch sử điều khiển (trang History - có filter/sort/paging)

GET /api/devices/history

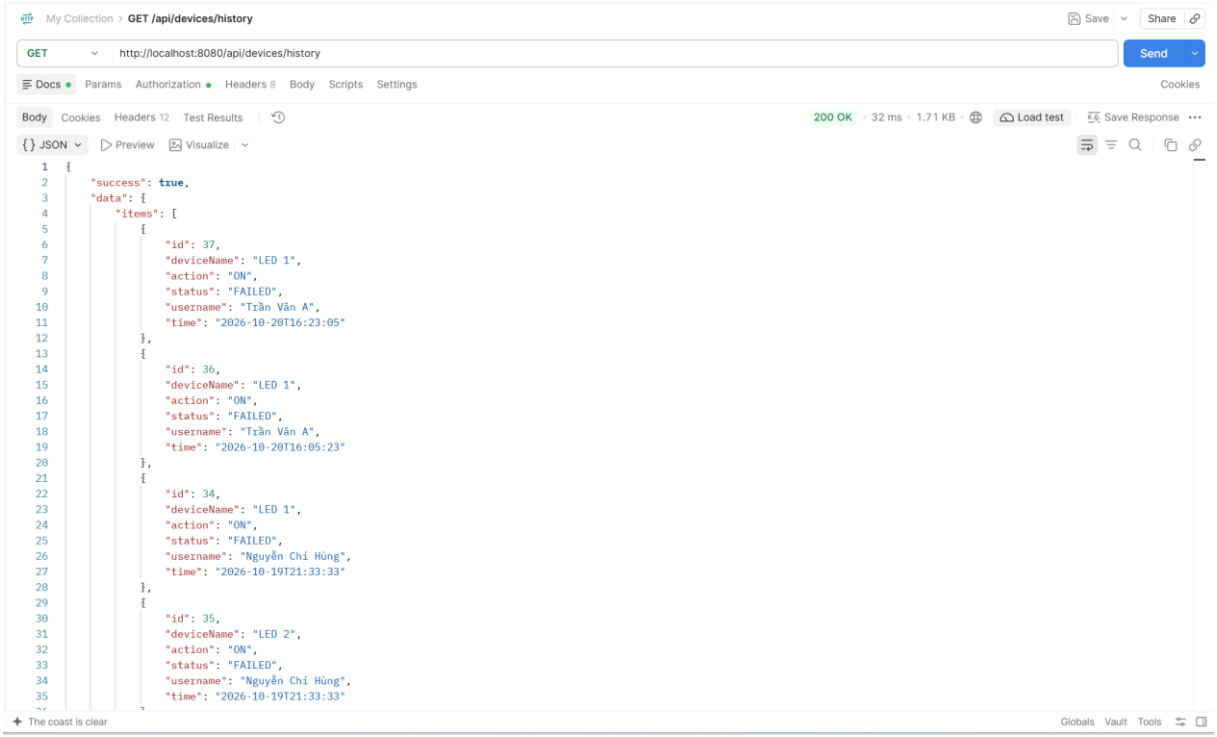
Param	Type	Bắt buộc	Mô tả
deviceName	string	không	LED 1 | LED 2 | (bỏ trống = tất cả)
time	string	không	lọc theo 1 mốc thời gian
action	string	không	lọc theo hành động (ON/OFF, có thể match text)
sort	string	không (default=desc)	asc | desc theo thời gian
page	int	không (default=1)	số trang
size	int	không (default=10)	số bản ghi/trang

Response

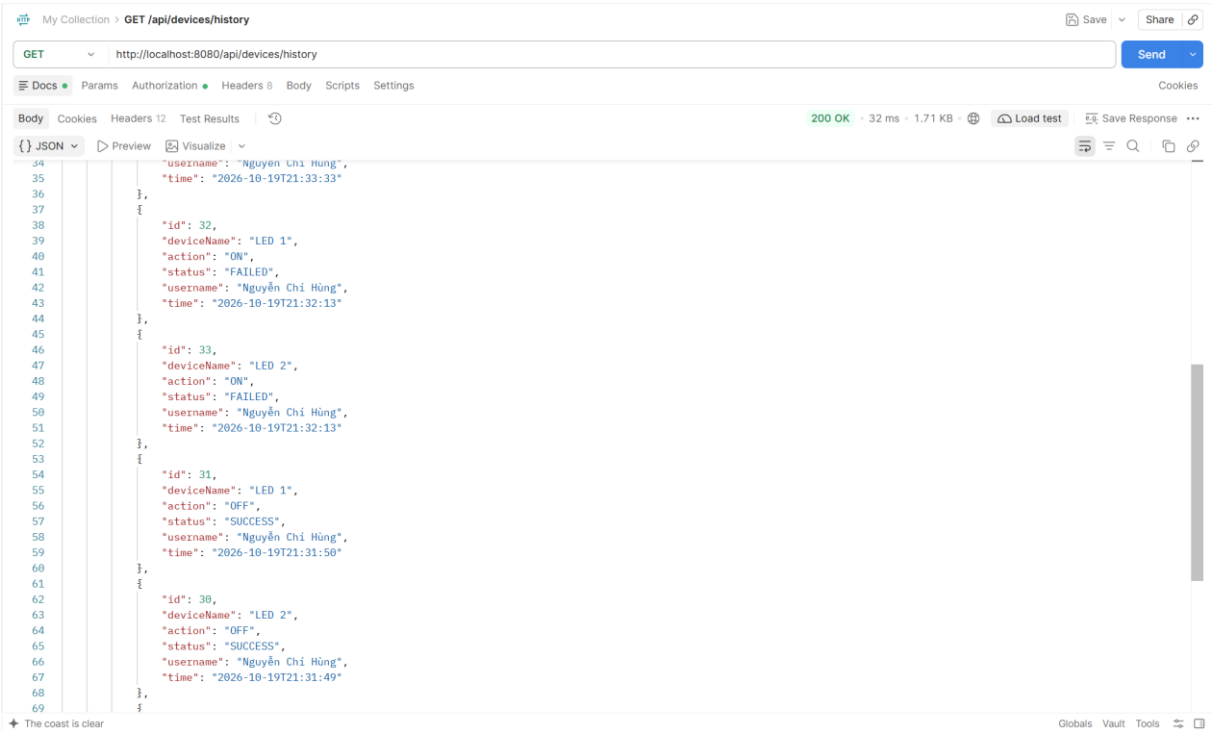
{

"success": true,

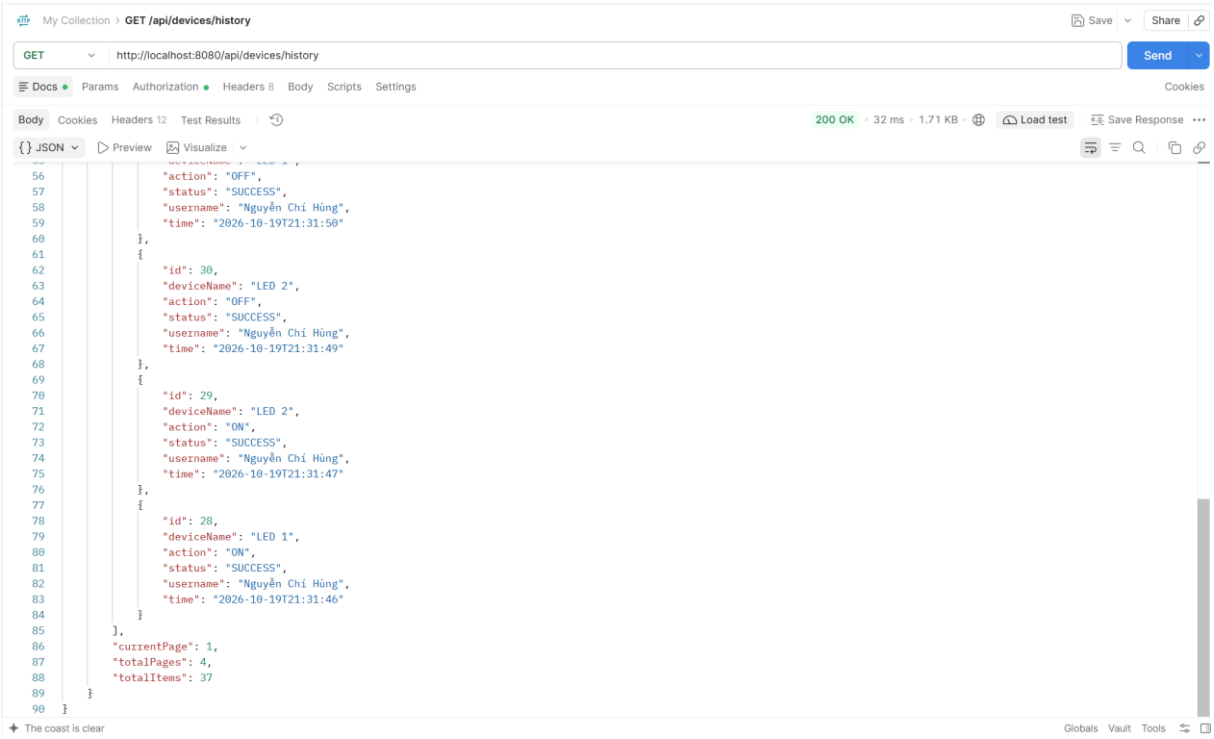
```
  "data": {
    "items": [
      { "id": 20, "deviceName": "LED 1", "action": "ON", "status": "ON", "time": "2026-08-18T12:54:01" },
      { "id": 22, "deviceName": "LED 2", "action": "ON", "status": "WAITING", "time": "2026-08-18T12:53:37" }
    ],
    "currentPage": 2,
    "totalPages": 5,
    "totalItems": 50
  }
}
```

Hình 4.35. Test api/devices/history với postman



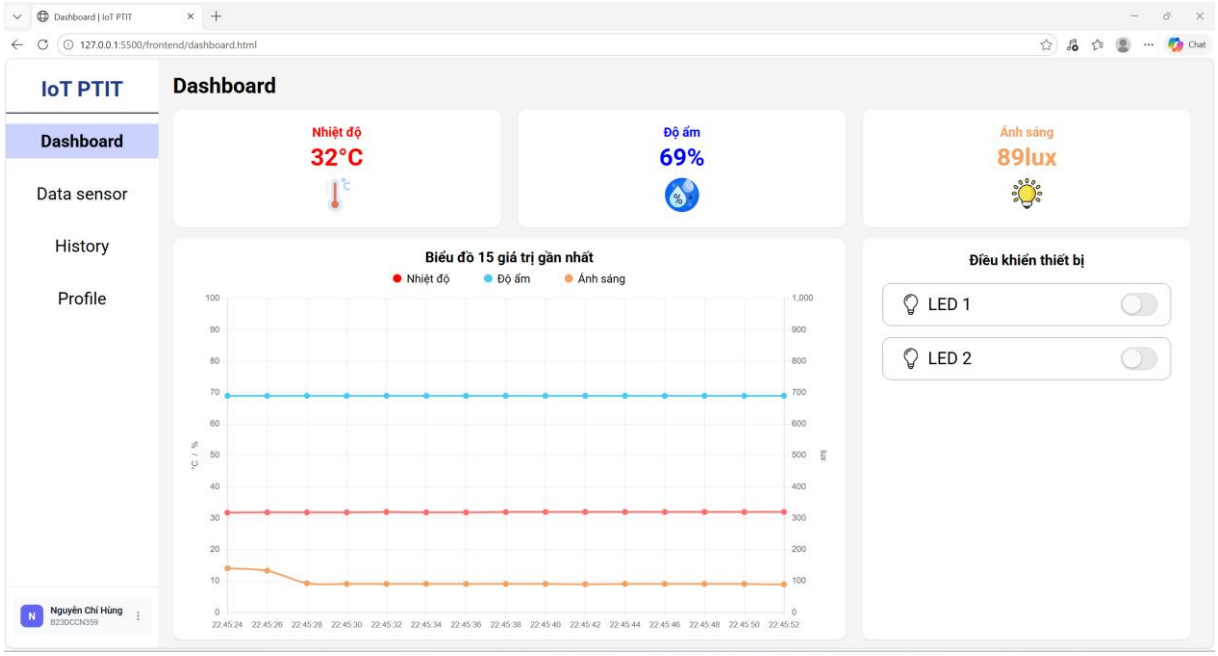
Hình 4.36. Test api/devices/history với postman (tiếp)



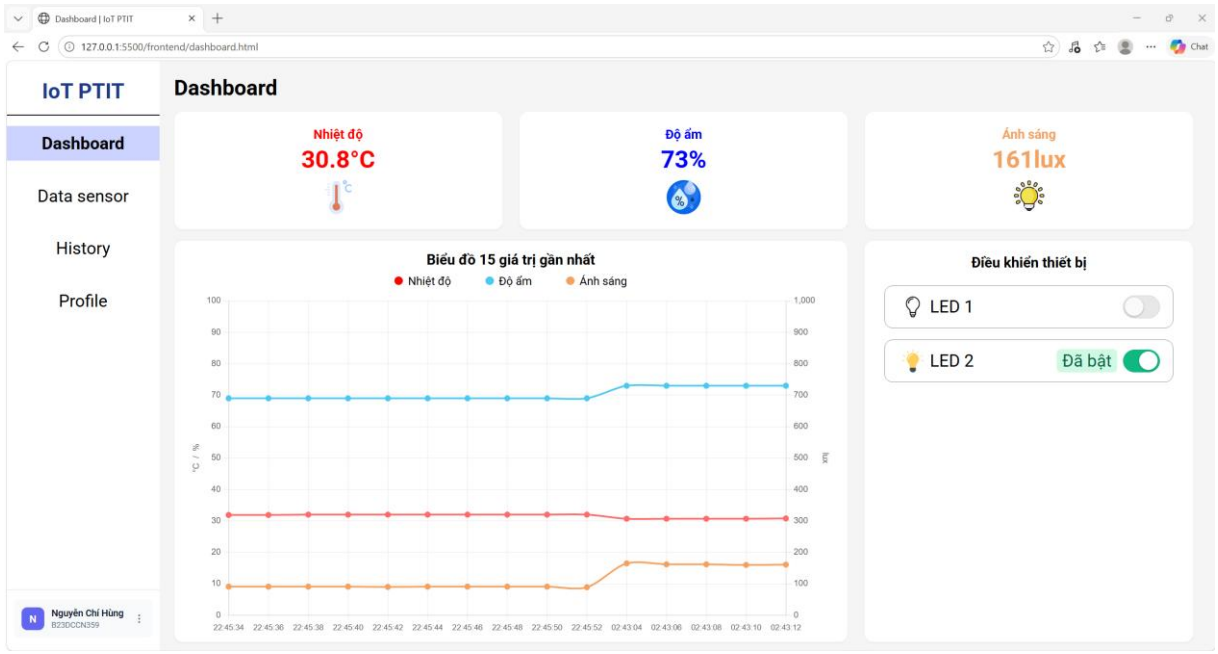
Hình 4.37. Test api/devices/history với postman (tiếp)

CHƯƠNG 5. KẾT QUẢ ĐẠT ĐƯỢC

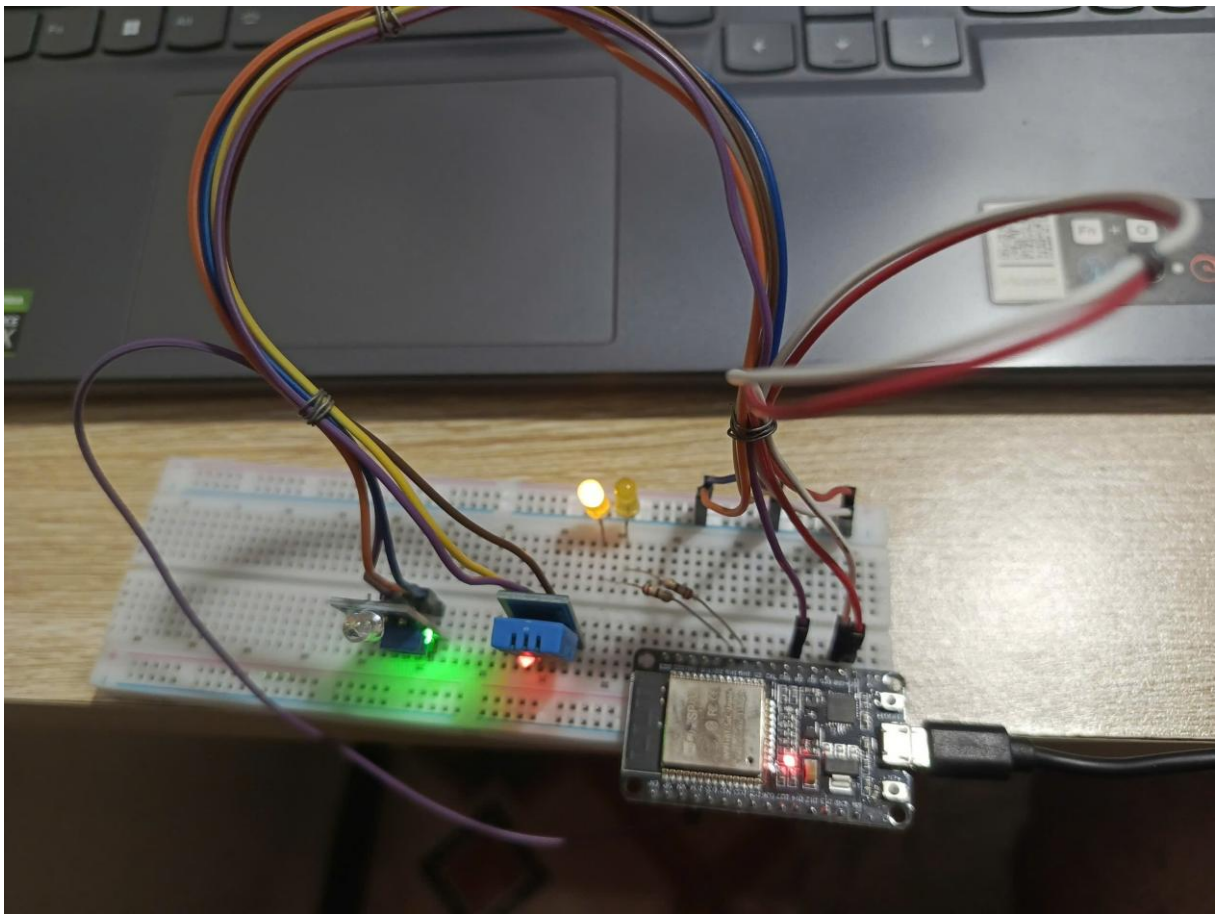
5.1. Trang Dashboard



Hình 5.1. Kết quả trang Dashboard



Hình 5.2. Giao diện khi bật đèn 2



Hình 5.3. Thiết bị khi bật đèn 2

Trang Dashboard được xây dựng nhằm cung cấp cái nhìn tổng quan về tình trạng hoạt động của hệ thống IoT, cho phép người dùng theo dõi các thông số môi trường và điều khiển thiết bị trực tiếp trên giao diện web.

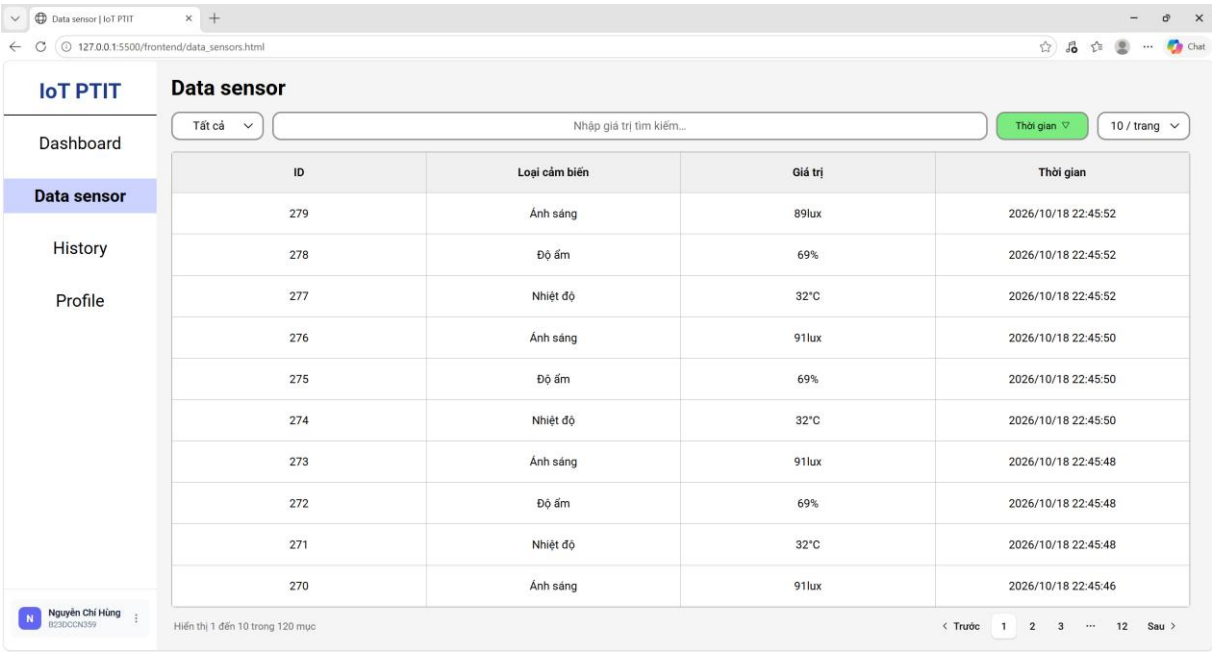
Các kết quả đạt được bao gồm:

- **Hiển thị thông số cảm biến:** Hệ thống hiển thị các giá trị đo được từ ba loại cảm biến gồm nhiệt độ, độ ẩm và ánh sáng. Các thông số được trình bày trực quan, giúp người dùng dễ dàng theo dõi và đánh giá điều kiện môi trường tại thời điểm hiện tại.
- **Hiển thị biểu đồ dữ liệu:** Trang Dashboard tích hợp biểu đồ để trực quan hóa 15 giá trị cảm biến gần nhất. Thông qua biểu đồ, người dùng có thể quan sát sự thay đổi của các thông số theo thời gian.
- **Cập nhật dữ liệu tự động:** Dữ liệu cảm biến được thu thập từ ESP32 và truyền về Backend thông qua giao thức MQTT. Giao diện Dashboard thực hiện lấy dữ liệu định kỳ 2 giây một lần, giúp thông tin hiển thị được cập nhật thường xuyên mà không cần người dùng tải lại trang.

- **Điều khiển thiết bị LED:** Người dùng có thể bật hoặc tắt LED1 và LED2 thông qua các nút chuyển đổi trạng thái trên giao diện. Yêu cầu điều khiển được gửi đến Backend, sau đó truyền qua MQTT đến ESP32 để thực thi. Trạng thái thiết bị được cập nhật dựa trên phản hồi thực tế từ phần cứng.

Nhìn chung, trang Dashboard đáp ứng yêu cầu theo dõi dữ liệu cảm biến theo thời gian và điều khiển thiết bị từ xa trên cùng một giao diện. Việc kết hợp hiển thị thông số, biểu đồ và trạng thái thiết bị giúp nâng cao tính trực quan, thuận tiện trong quá trình giám sát và vận hành hệ thống IoT.

5.2. Trang Data sensor



Hình 5.4. Kết quả trang Data sensor

Trang Data sensor được xây dựng nhằm hỗ trợ người dùng tra cứu và theo dõi lịch sử dữ liệu do các cảm biến thu thập và lưu trữ trong cơ sở dữ liệu. Trang này giúp người dùng tiếp cận dữ liệu chi tiết hơn so với các thông số tổng quan trên Dashboard.

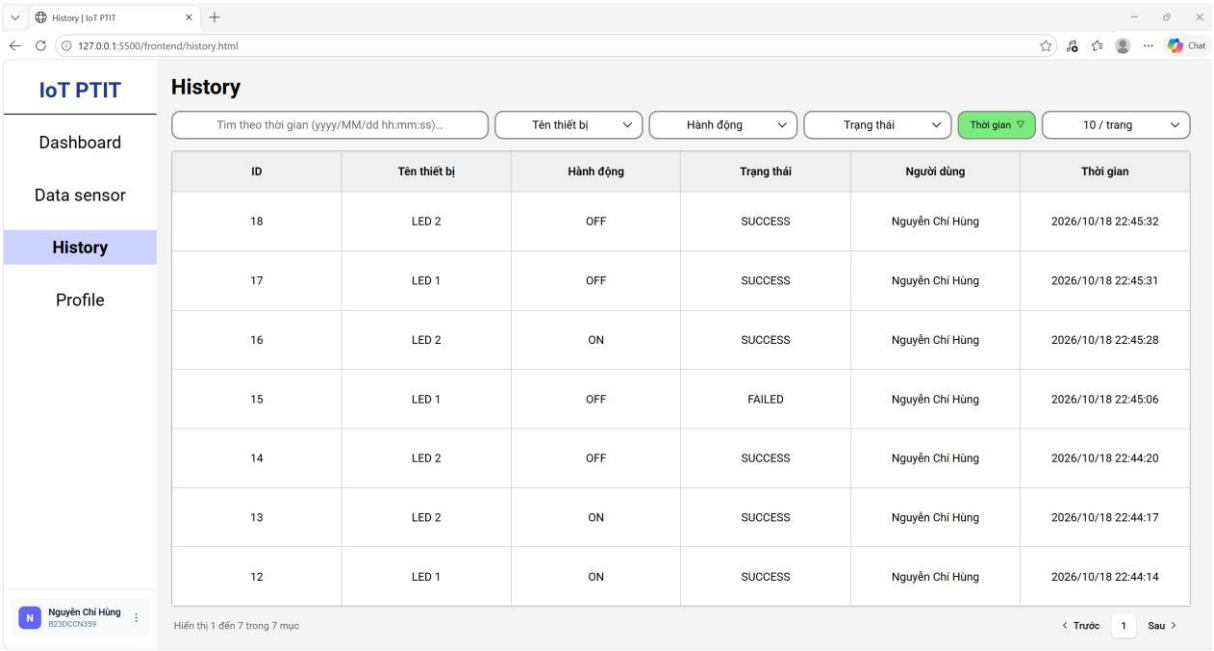
Các kết quả đạt được bao gồm:

- **Hiển thị danh sách dữ liệu cảm biến:** Hệ thống truy xuất dữ liệu từ bảng sensor_data trong cơ sở dữ liệu và hiển thị dưới dạng bảng. Các thông tin được trình bày bao gồm mã bản ghi (ID), loại cảm biến, giá trị đo và thời gian ghi nhận.
- **Tìm kiếm và lọc dữ liệu:** Người dùng có thể tìm kiếm dữ liệu theo thời gian hoặc giá trị của từng loại cảm biến để nhanh chóng xác định những bản ghi cần xem xét.

- **Sắp xếp dữ liệu:** Hệ thống hỗ trợ sắp xếp bản ghi theo thời gian, giúp người dùng theo dõi dữ liệu theo thứ tự mới nhất hoặc cũ nhất tùy theo nhu cầu.
- **Phân trang kết quả:** Danh sách dữ liệu được chia thành nhiều trang nhằm giúp giao diện dễ quan sát và thuận tiện khi xử lý số lượng bản ghi lớn. Người dùng có thể chuyển trang mà vẫn duy trì các điều kiện tìm kiếm và lọc đã áp dụng. Ngoài ra, người dùng có thể chọn giới hạn số bản ghi/trang.

Thông qua trang Data sensor, hệ thống đáp ứng nhu cầu lưu trữ, hiển thị và tra cứu lịch sử dữ liệu cảm biến. Các chức năng tìm kiếm, lọc, sắp xếp và phân trang giúp người dùng khai thác dữ liệu hiệu quả hơn, đồng thời hỗ trợ việc quan sát sự biến đổi của các thông số môi trường trong quá trình hệ thống hoạt động.

5.3. Trang History



Hình 5.5. Kết quả trang History

Trang History được xây dựng để quản lý và tra cứu lịch sử điều khiển các thiết bị trong hệ thống, cụ thể là LED1 và LED2. Thông qua trang này, người dùng có thể kiểm tra lại các thao tác đã thực hiện cũng như trạng thái xử lý của từng yêu cầu điều khiển.

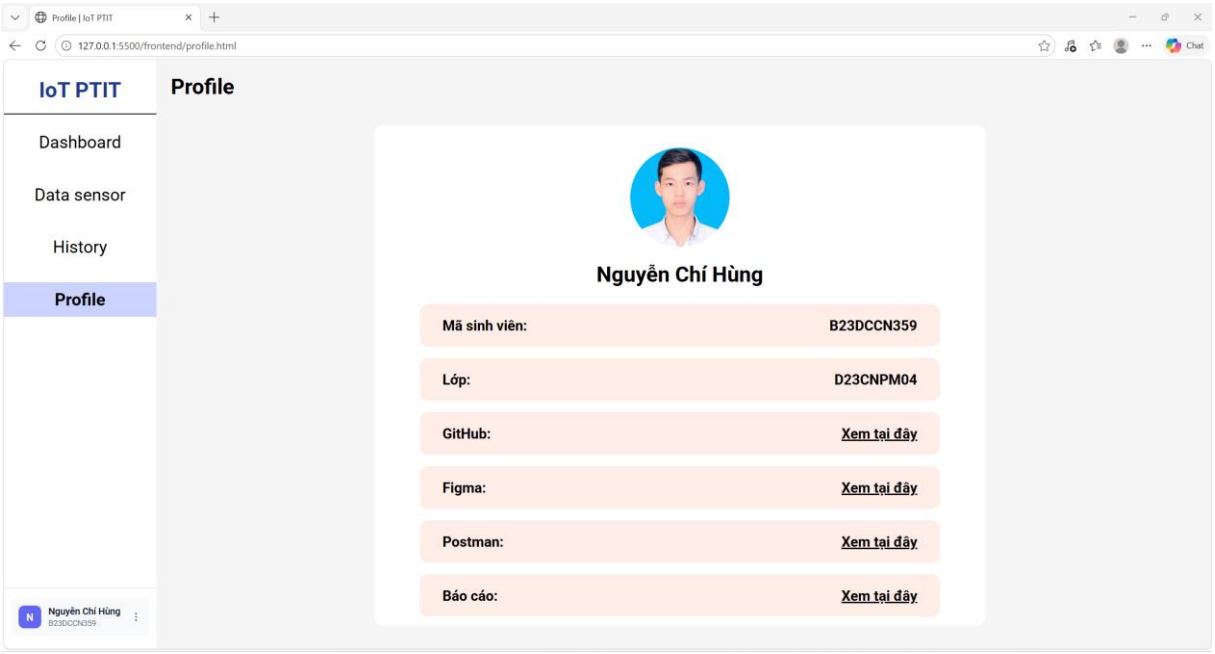
Các kết quả đạt được bao gồm:

- **Hiển thị lịch sử điều khiển thiết bị:** Hệ thống truy xuất dữ liệu từ bảng history trong cơ sở dữ liệu và hiển thị danh sách các bản ghi, bao gồm mã bản ghi (ID), tên thiết bị, hành động điều khiển, trạng thái, người dùng thực hiện hành động và thời gian thực hiện.

- **Ghi nhận yêu cầu điều khiển:** Khi người dùng thực hiện thao tác bật hoặc tắt LED trên Dashboard, Backend tạo một bản ghi lịch sử mới với hành động tương ứng và trạng thái ban đầu là WAITING, thể hiện yêu cầu đang chờ thiết bị phản hồi.
- **Cập nhật trạng thái thực tế:** Sau khi ESP32 nhận và thực thi lệnh, thiết bị gửi trạng thái phản hồi thông qua MQTT. Backend tiếp nhận thông tin và cập nhật bản ghi lịch sử tương ứng, giúp hệ thống lưu lại kết quả thực hiện thay vì chỉ ghi nhận yêu cầu ban đầu.
- **Tìm kiếm và lọc:** Người dùng có thể tra cứu các bản ghi theo thời gian, lọc theo tên thiết bị, lọc theo hoạt động, lọc theo trạng thái để xác định các thao tác cần kiểm tra.
- **Sắp xếp và phân trang:** Hệ thống hỗ trợ sắp xếp lịch sử theo thời gian và phân trang kết quả. Các điều kiện truy vấn được duy trì khi người dùng chuyển trang hoặc thay đổi cách hiển thị dữ liệu. Người dùng có thể chuyển trang mà vẫn duy trì các điều kiện tìm kiếm và lọc đã áp dụng. Ngoài ra, người dùng có thể chọn giới hạn số bản ghi/trang.
- **Hiển thị trạng thái chờ và kết quả xử lý:** Những bản ghi đang có trạng thái WAITING vẫn được hiển thị để người dùng nhận biết các yêu cầu chưa có kết quả xác nhận. Trường hợp thiết bị không phản hồi sau thời gian chờ, hệ thống có thể ghi nhận trạng thái lỗi như FAILED.

Như vậy, trang History đáp ứng yêu cầu theo dõi và quản lý lịch sử hoạt động của thiết bị. Việc lưu lại hành động điều khiển, thời gian và trạng thái phản hồi giúp tăng khả năng kiểm tra, đối chiếu và truy vết các thao tác, đồng thời hỗ trợ phát hiện những trường hợp thiết bị thực hiện lệnh không thành công.

5.4. Trang Profile



Hình 5.6. Kết quả trang profile

Trang Profile được xây dựng nhằm cung cấp thông tin cá nhân và các tài nguyên liên quan của người dùng hoặc thành viên thực hiện đề tài. Trang này giúp tập trung các thông tin cần thiết tại một giao diện thống nhất.

Các kết quả đạt được bao gồm:

- **Hiển thị thông tin cá nhân:** Hệ thống cung cấp các thông tin gồm họ tên, mã sinh viên và lớp, giúp xác định thông tin của thành viên thực hiện đề tài.
- **Cung cấp liên kết tài nguyên:** Trang Profile hiển thị các liên kết liên quan đến quá trình phát triển và thiết kế hệ thống, bao gồm GitHub, Figma, Postman và báo cáo đề tài.
- **Tổ chức thông tin trực quan:** Các thông tin được bố trí trên cùng một trang, giúp người dùng dễ dàng theo dõi và truy cập các tài nguyên cần thiết.

Kết quả đạt được của trang Profile là cung cấp giao diện hiển thị thông tin cá nhân và các liên kết phục vụ quá trình phát triển, thiết kế, kiểm thử và trình bày đề tài.

5.5. Đánh giá kết quả

Sau quá trình xây dựng và triển khai hệ thống IoT, các chức năng chính đã được phát triển nhằm đáp ứng yêu cầu giám sát dữ liệu cảm biến, điều khiển thiết bị từ xa và quản lý lịch sử hoạt động thông qua giao diện web. Hệ thống có sự kết hợp

giữa phần cứng ESP32, giao thức MQTT, Backend, cơ sở dữ liệu và Frontend để thực hiện quá trình thu thập, xử lý, lưu trữ và hiển thị dữ liệu. Tuy nhiên, bên cạnh những kết quả đạt được, hệ thống vẫn còn một số hạn chế cần được cải thiện để nâng cao tính ổn định, bảo mật và khả năng mở rộng trong tương lai.

5.5.1. Ưu điểm

Hệ thống đạt được một số ưu điểm nổi bật về chức năng, khả năng giám sát và quản lý dữ liệu như sau:

- **Giám sát dữ liệu cảm biến trực quan:** Trang Dashboard hiển thị các thông số nhiệt độ, độ ẩm và ánh sáng, kết hợp với biểu đồ 15 giá trị gần nhất. Dữ liệu được cập nhật định kỳ 2 giây một lần, giúp người dùng theo dõi sự thay đổi của môi trường một cách thuận tiện mà không cần tải lại trang thủ công.
- **Điều khiển thiết bị từ xa:** Người dùng có thể bật hoặc tắt LED1 và LED2 trực tiếp trên giao diện web. Việc sử dụng giao thức MQTT giúp truyền lệnh điều khiển giữa Backend và ESP32, đồng thời hỗ trợ thiết bị gửi phản hồi về hệ thống để cập nhật trạng thái thực tế.
- **Quản lý và tra cứu dữ liệu hiệu quả:** Các trang Data sensor và History cung cấp khả năng hiển thị dữ liệu dưới dạng bảng, hỗ trợ tìm kiếm, lọc, sắp xếp và phân trang. Những chức năng này giúp người dùng dễ dàng tra cứu dữ liệu cảm biến và lịch sử điều khiển khi số lượng bản ghi tăng lên.
- **Theo dõi lịch sử điều khiển:** Hệ thống ghi nhận các yêu cầu điều khiển và cập nhật trạng thái sau khi nhận phản hồi từ thiết bị. Việc lưu trữ thông tin về hành động, thiết bị, thời gian và trạng thái giúp hỗ trợ kiểm tra, đối chiếu và xác định kết quả của các thao tác đã thực hiện.
- **Kiến trúc hệ thống có tính phân tách:** Hệ thống được tổ chức thành các thành phần gồm phần cứng, MQTT Broker, Backend, cơ sở dữ liệu và Frontend. Mỗi thành phần đảm nhiệm một vai trò riêng, giúp quá trình phát triển, kiểm thử và bảo trì thuận tiện hơn.
- **Giao diện dễ sử dụng:** Các chức năng được phân chia thành những trang riêng biệt như Dashboard, Data sensor, History và Profile. Cách tổ chức này giúp người dùng dễ dàng tiếp cận các chức năng giám sát, điều khiển và tra cứu thông tin.
- **Có khả năng mở rộng:** Việc sử dụng MQTT và thiết kế các API riêng cho từng chức năng tạo điều kiện thuận lợi để bổ sung cảm biến, thiết bị điều khiển hoặc các tính năng quản lý mới khi có nhu cầu.

Nhìn chung, hệ thống đã đáp ứng được các chức năng cơ bản của một ứng dụng IoT giám sát và điều khiển từ xa. Sự kết hợp giữa thu thập dữ liệu tự động, giao diện trực quan và khả năng lưu trữ lịch sử góp phần nâng cao hiệu quả quản lý, đồng thời tạo nền tảng cho việc phát triển các tính năng thông minh hơn trong tương lai.

5.5.2. Nhược điểm

Bên cạnh những ưu điểm đã đạt được, hệ thống vẫn tồn tại một số hạn chế liên quan đến khả năng vận hành, xử lý dữ liệu và bảo mật.

- **Phụ thuộc vào kết nối mạng:** Quá trình truyền dữ liệu giữa ESP32, MQTT Broker, Backend và Frontend phụ thuộc vào kết nối mạng. Khi xảy ra sự cố kết nối, dữ liệu có thể bị gián đoạn, lệnh điều khiển có thể không đến được thiết bị hoặc trạng thái hiển thị trên giao diện chưa phản ánh kịp thời trạng thái thực tế.
- **Độ trễ trong cập nhật dữ liệu:** Dashboard thực hiện lấy dữ liệu định kỳ 2 giây một lần. Cách tiếp cận này đáp ứng nhu cầu giám sát cơ bản nhưng có thể phát sinh nhiều request HTTP khi có nhiều người dùng truy cập đồng thời. Ngoài ra, thời gian cập nhật thực tế còn phụ thuộc vào tốc độ xử lý của Backend, cơ sở dữ liệu và chất lượng mạng.
- **Khả năng xử lý lỗi điều khiển còn hạn chế:** Mặc dù hệ thống có cơ chế ghi nhận trạng thái WAITING và có thể đánh dấu FAILED khi thiết bị không phản hồi, việc xử lý các tình huống như phản hồi chậm, mất kết nối MQTT hoặc nhận phản hồi không đúng yêu cầu vẫn cần được hoàn thiện. Nếu không có cơ chế xác định chính xác từng yêu cầu điều khiển, hệ thống có thể gặp khó khăn khi liên kết phản hồi với bản ghi lịch sử tương ứng.
- **Khả năng mở rộng cơ sở dữ liệu:** Khi số lượng dữ liệu cảm biến và lịch sử điều khiển tăng lên theo thời gian, dung lượng lưu trữ và thời gian truy vấn có thể tăng. Nếu chưa tối ưu chỉ mục, truy vấn và cơ chế lưu trữ dữ liệu, hiệu suất hệ thống có thể giảm khi xử lý khối lượng dữ liệu lớn.
- **Khả năng bảo mật cần được tăng cường:** Hệ thống cần có cơ chế xác thực người dùng, phân quyền truy cập API và bảo vệ thông tin trao đổi giữa các thành phần. Nếu cấu hình xác thực MQTT, quyền truy cập cơ sở dữ liệu hoặc cơ chế bảo vệ API chưa đầy đủ, hệ thống có nguy cơ bị truy cập trái phép hoặc bị gửi lệnh điều khiển không hợp lệ.
- **Chưa khai thác chuyên sâu dữ liệu cảm biến:** Các chức năng hiện tại chủ yếu tập trung vào hiển thị thông số, biểu đồ và tra cứu lịch sử. Hệ thống

chưa tập trung vào các chức năng nâng cao như cảnh báo khi thông số vượt ngưỡng, phân tích xu hướng dài hạn hoặc dự đoán sự thay đổi của môi trường.

- **Khả năng giám sát tình trạng phần cứng còn hạn chế:** Việc hiển thị trạng thái LED chưa đồng nghĩa với việc giám sát toàn diện tình trạng hoạt động của ESP32 và các cảm biến. Hệ thống cần bổ sung cơ chế theo dõi thời gian kết nối cuối cùng, trạng thái thiết bị và tình trạng gửi dữ liệu để phát hiện sớm các sự cố phần cứng.

Những hạn chế trên chủ yếu liên quan đến khả năng vận hành trong điều kiện thực tế, độ tin cậy của quá trình điều khiển, bảo mật và khả năng mở rộng. Đây là những vấn đề cần được xem xét trong quá trình hoàn thiện hệ thống nhằm bảo đảm hoạt động ổn định khi số lượng thiết bị và người dùng tăng lên.

5.5.3. Hướng khắc phục và phát triển

Để khắc phục các hạn chế và nâng cao chất lượng hệ thống, có thể triển khai một số hướng cải tiến sau:

- **Nâng cao độ tin cậy của kết nối:** Bổ sung cơ chế tự động kết nối lại MQTT khi ESP32 hoặc Backend mất kết nối với Broker. Hệ thống nên theo dõi trạng thái kết nối của thiết bị và thời điểm nhận dữ liệu gần nhất để phân biệt giữa trạng thái hoạt động bình thường và trạng thái mất kết nối. Đối với các dữ liệu quan trọng, có thể bổ sung cơ chế lưu đệm tạm thời và gửi bù khi kết nối được khôi phục.

- **Cải thiện cơ chế cập nhật dữ liệu:** Nghiên cứu sử dụng WebSocket hoặc Server-Sent Events (SSE) để Backend chủ động gửi dữ liệu mới đến Frontend thay vì liên tục gửi request HTTP theo chu kỳ cố định. Giải pháp này có thể giảm số lượng request không cần thiết và cải thiện độ kịp thời của thông tin hiển thị. Tuy nhiên, cần đánh giá hiệu quả thực tế trước khi thay thế cơ chế cập nhật hiện tại.

- **Hoàn thiện quy trình điều khiển thiết bị:** Bổ sung mã định danh riêng cho từng yêu cầu điều khiển để liên kết chính xác lệnh gửi đi với phản hồi từ ESP32. Xây dựng cơ chế timeout, xử lý phản hồi trùng lặp và phân biệt rõ các trạng thái như WAITING, SUCCESS và FAILED. Ngoài ra, cần bảo đảm trạng thái trên giao diện được cập nhật theo phản hồi thực tế của thiết bị, tránh trường hợp giao diện hiển thị thành công khi lệnh chưa được thực thi.

- **Tối ưu cơ sở dữ liệu:** Bổ sung chỉ mục cho các trường thường xuyên được sử dụng trong tìm kiếm, lọc và sắp xếp như thời gian ghi nhận, loại cảm biến, tên thiết bị và trạng thái. Đồng thời, tối ưu truy vấn phân trang, giới hạn số lượng bản ghi trả về mỗi lần và xây dựng chính sách lưu trữ hoặc lưu trữ dài hạn đối với dữ liệu cũ để duy trì hiệu suất hệ thống.
- **Tăng cường bảo mật:** Hoàn thiện cơ chế xác thực và phân quyền người dùng; kiểm tra dữ liệu đầu vào của các API; sử dụng kết nối được mã hóa khi phù hợp; đồng thời cấu hình tài khoản và quyền truy cập MQTT theo nguyên tắc tối thiểu cần thiết. Các thông tin nhạy cảm như mật khẩu và khóa truy cập cần được quản lý an toàn, tránh lưu trực tiếp trong mã nguồn công khai.
- **Phát triển chức năng cảnh báo:** Bổ sung các ngưỡng cảnh báo cho nhiệt độ, độ ẩm và ánh sáng. Khi giá trị vượt ngưỡng được cấu hình, hệ thống có thể hiển thị cảnh báo trên Dashboard hoặc gửi thông báo đến người dùng thông qua các kênh phù hợp. Ngưỡng cảnh báo nên cho phép cấu hình để đáp ứng những môi trường sử dụng khác nhau.
- **Bổ sung chức năng phân tích dữ liệu:** Phát triển các biểu đồ theo ngày, tuần hoặc tháng; thống kê giá trị trung bình, lớn nhất và nhỏ nhất; đồng thời hỗ trợ xuất dữ liệu ra các định dạng như CSV hoặc Excel. Trong giai đoạn tiếp theo, có thể nghiên cứu các phương pháp phân tích chuỗi thời gian để nhận diện xu hướng và hỗ trợ dự đoán sự thay đổi của môi trường.
- **Mở rộng khả năng giám sát thiết bị:** Bổ sung thông tin về thời gian hoạt động, thời điểm gửi dữ liệu gần nhất và trạng thái kết nối của ESP32. Có thể áp dụng cơ chế heartbeat để xác định thiết bị còn hoạt động hay không, từ đó giúp người dùng phát hiện sớm sự cố và chủ động kiểm tra phần cứng.
- **Nâng cao chất lượng kiểm thử và triển khai:** Xây dựng các kịch bản kiểm thử cho từng chức năng, bao gồm trường hợp hoạt động bình thường, mất kết nối, dữ liệu không hợp lệ, phản hồi chậm và số lượng truy vấn lớn. Đồng thời, cần thực hiện kiểm thử tích hợp giữa phần cứng, MQTT, Backend, cơ sở dữ liệu và Frontend để đánh giá tính chính xác và độ ổn định của toàn hệ thống.

CHƯƠNG 6. KẾT LUẬN

Đề tài đã xây dựng được một hệ thống IoT sử dụng ESP32 nhằm giám sát các thông số môi trường gồm nhiệt độ, độ ẩm, ánh sáng và điều khiển hai đèn LED từ xa thông qua giao diện Web. Hệ thống áp dụng MQTT để trao đổi dữ liệu giữa ESP32 và Backend, kết hợp Spring Boot, RESTful API và MySQL để xử lý và lưu trữ dữ liệu.

Thông qua quá trình thực hiện, đề tài đã giúp vận dụng các kiến thức về IoT, MQTT, lập trình ESP32, phát triển Web và cơ sở dữ liệu vào một hệ thống thực tế. Mặc dù hệ thống còn ở quy mô nhỏ, đây là nền tảng để có thể phát triển thêm nhiều thiết bị, cảm biến và các chức năng tự động hóa trong tương lai.

PHỤ LỤC

Mã nguồn: https://github.com/Hunguyenobugs/BTL-IoT